

UNIVERSIDAD POLITÉCNICA DE MADRID

ESCUELA TÉCNICA SUPERIOR DE INGENIEROS DE TELECOMUNICACIÓN



GRADO EN INGENIERÍA DE TECNOLOGÍAS Y SERVICIOS DE
TELECOMUNICACIÓN

TRABAJO FIN DE GRADO

DESARROLLO Y OPTIMIZACIÓN DE UN
SISTEMA MODULAR DE CÓDIGO ABIERTO
PARA LA PRODUCCIÓN REMOTA DE
VÍDEO EN TIEMPO REAL

MARTÍN HERRANZ SÁNCHEZ

Junio 2026

GRADO EN INGENIERÍA DE TECNOLOGÍAS Y SERVICIOS DE TELECOMUNICACIÓN

TRABAJO FIN DE GRADO

Título: Desarrollo y optimización de un sistema modular de código abierto para la producción remota de vídeo en tiempo real

Autor: D. Martín Herranz Sánchez

Tutor: D. Álvaro Llorente Gómez

Ponente: D. David Jiménez Bermejo

Departamento: Electrónica física, Ingeniería Eléctrica y Física Aplicada

MIEMBROS DEL TRIBUNAL

Presidente:

Vocal:

Secretario:

Suplente:

Los miembros del tribunal arriba nombrados acuerdan otorgar la calificación de:

Madrid, a de de 2026

UNIVERSIDAD POLITÉCNICA DE MADRID

ESCUELA TÉCNICA SUPERIOR DE INGENIEROS DE TELECOMUNICACIÓN



GRADO EN INGENIERÍA DE TECNOLOGÍAS Y SERVICIOS DE
TELECOMUNICACIÓN

TRABAJO FIN DE GRADO

**Desarrollo y optimización de un
sistema modular de código abierto
para la producción remota de vídeo
en tiempo real**

Martín Herranz Sánchez

Junio 2026

RESUMEN

La producción de eventos en directo con múltiples cámaras, transiciones, rótulos y gestión de audio ha requerido históricamente equipamiento de radiodifusión profesional con costes de adquisición de varios miles de euros y personal técnico especializado desplazado al lugar del evento. Este modelo resulta inaccesible para eventos de mediana escala que necesitan producción de calidad.

Este Trabajo Fin de Grado presenta el diseño e implementación de Voctomix 2.0, una plataforma modular de código abierto para la producción remota de vídeo en tiempo real. La plataforma se construye sobre GStreamer y Python, siguiendo una arquitectura cliente-servidor en la que el núcleo de procesamiento (**voctocore**) puede ejecutarse de forma remota mientras el realizador opera la interfaz gráfica (**voctogui**) desde cualquier equipo de la red. Las funcionalidades implementadas incluyen cuatro modos de composición de imagen con transiciones animadas, *overlays* gráficos dinámicos, un gestor de estados de emisión con tres modos, sincronización automática del audio y un servicio de telemetría con publicación en tiempo real en la plataforma de mensajería RabbitMQ.

El sistema se ha contenerizado y desplegado tanto en Docker Compose como en Kubernetes con Minikube, integrándose en el proyecto europeo de investigación CyberNEMO de la UPM, donde opera en tiempo real como plataforma de producción audiovisual en un entorno de validación activa para diferentes componentes de ciberseguridad.

La plataforma se ha validado funcionalmente en cuatro escenarios de despliegue: un único equipo, dos equipos físicos en red local, Docker Compose y Kubernetes con Minikube. Sobre los dos últimos se realizaron además pruebas de rendimiento y consumo de recursos, mediciones de latencia de conmutación entre fuentes y de transición entre modos de composición, una prueba de estabilidad a largo plazo y ensayos de resiliencia ante fallos simulados de cámara.

El servicio de telemetría se validó de forma independiente, comprobando que los eventos de producción, el estado de las fuentes y las métricas de consumo de recursos se publican y almacenan correctamente en RabbitMQ mediante AMQP, posibilitando su análisis y auditoría posterior.

PALABRAS CLAVE

producción audiovisual en directo, producción remota de vídeo, mezclador de vídeo, *software* libre, GStreamer, Docker, Kubernetes, RabbitMQ, telemetría, *streaming*, AMQP.

SUMMARY

Live production of events with multiple cameras, transitions, graphics and audio management has historically required professional broadcast equipment costing several thousand euros and specialised technical crews deployed on site. This model is out of reach for medium-scale events that need quality production.

This Bachelor's Thesis presents the design and implementation of Voctomix 2.0, a modular open-source platform for remote video production in real time. The platform is built on GStreamer and Python, following a client-server architecture in which the processing core (**voccore**) can run remotely while the director operates the graphical interface (**vocgui**) from any machine on the network. The implemented features include four image composition modes with animated transitions, dynamic graphical overlays, an output state manager with three modes, automatic audio synchronisation and a telemetry service that publishes events in real time to the RabbitMQ message broker.

The system has been containerised and deployed on both Docker Compose and Kubernetes with Minikube, integrating into the CyberNEMO European research project at UPM, where it operates in real time as an audiovisual production platform in an active validation environment for different cybersecurity components.

Functional validation was carried out across four deployment scenarios: a single machine, two physical machines over a local network, Docker Compose and Kubernetes with Minikube. On the latter two, performance and resource consumption tests, switching latency and composition mode transition measurements, a long-term stability test and resilience trials with simulated camera failures were also conducted.

The telemetry service was validated independently, confirming that production events, source status and resource consumption metrics are correctly published and stored in RabbitMQ via AMQP, enabling their subsequent analysis and auditing.

KEYWORDS

live audiovisual production, remote video production, video mixer, free software, GStreamer, Docker, Kubernetes, RabbitMQ, telemetry, streaming, AMQP.

Agradecimientos

En primer lugar, quiero agradecer a mis tutores Álvaro Llorente y David Jiménez la oportunidad de desarrollar este proyecto, con el que he disfrutado a lo largo de todo el proceso. Realizarlo junto con la beca dentro del grupo GATV ha hecho el año mucho más llevadero.

A todos mis amigos de estos años de carrera, por los buenos momentos, las risas y también el sufrimiento, que ha sido una constante desde el primer día.

Por último, a mi familia, por haber estado siempre a mi lado y por apoyarme en todo momento. Gracias.

Índice

Resumen	II
Palabras clave	II
Summary	III
Keywords	III
Agradecimientos	IV
Lista de acrónimos	IX
1. Introducción y objetivos	1
1.1. Introducción	1
1.2. Objetivos	2
1.3. Estructura de la memoria	2
2. Estado del arte	4
2.1. Producción audiovisual	4
2.2. Contribución audiovisual	8
2.3. Distribución audiovisual	12
2.4. Casos de uso reales de producción remota	15
3. Diseño e implementación del sistema de producción remota de vídeo	18
3.1. Arquitectura del sistema	19
3.2. Fuentes de señal de cámaras	21
3.3. Fuentes de señal auxiliares	23
3.4. Composites	24
3.5. Transiciones	27
3.6. <i>Overlays</i> , logos y texto dinámico	27
3.7. Señal de programa final	29
3.8. Personalización de la interfaz gráfica	30
3.9. Sistema de telemetría	30
4. Despliegue del sistema	34
4.1. Despliegue nativo: un PC y dos PCs	34
4.2. Contenedorización con Docker Compose	35
4.3. Orquestación con Kubernetes	37
5. Pruebas de validación del sistema de producción remota de vídeo	39
5.1. Entorno de pruebas	39
5.2. Validación funcional	40

5.3. Rendimiento del sistema	40
5.4. Latencia del sistema	43
5.5. Estabilidad a largo plazo	44
5.6. Resiliencia y recuperación	46
6. Conclusiones y líneas futuras	48
6.1. Conclusiones	48
6.2. Líneas de trabajo futuro	50
Bibliografía	51
Anexo A. Aspectos éticos, económicos, sociales y ambientales	57
Anexo B. Presupuesto económico	61
Anexo C. Evolución del repositorio	62

Índice de figuras

2.1. Cadena de producción audiovisual tradicional.	5
2.2. Arquitectura del modelo REMI.	6
2.3. Cadena de producción audiovisual en directo.	8
3.1. Arquitectura global del sistema Voctomix 2.0.	19
3.2. Interfaz gráfica completa de Voctomix 2.0 (<i>voctogui</i>).	20
3.3. Panel de cámaras en <i>voctogui</i>	22
3.4. Estados NOSTREAM y PAUSE del <i>stream blanker</i> : vista del realizador (arriba) y señal recibida por el espectador (abajo).	24
3.5. Disposición geométrica de las fuentes A y B en los cuatro <i>composites</i> de Voctomix 2.0.	25
3.6. Señal de programa en los cuatro modos de <i>composite</i> de Voctomix 2.0.	26
3.7. Efecto espejo aplicado a los <i>composites</i> Picture in Picture y Lecture.	26
3.8. Señal de programa con <i>overlay</i> y rotulación activos (arriba) y panel de control correspondiente en la interfaz (abajo).	29
3.9. Salida del servicio de telemetría mostrando un evento STATE y un evento CHANGE consecutivos.	31
3.10. Colas RabbitMQ activas durante la integración con CyberNEMO.	33
4.1. Escenario de despliegue nativo con un único PC.	35
4.2. Escenario de despliegue nativo con dos PCs en red local.	35
4.3. Escenario de despliegue con Docker Compose.	36
4.4. Escenario de despliegue con Kubernetes.	38
5.1. Uso de CPU en función del número de cámaras activas.	41
5.2. Uso de RAM en función del número de cámaras activas.	41
5.3. Consumo energético neto medido por RAPL en función del número de cámaras activas.	42
5.4. Distribución de latencias de conmutación.	43
5.5. Latencia mediana de transición de modo de composición.	44
5.6. Uso de CPU y RAM durante la prueba de estabilidad en Docker Compose.	45
5.7. Uso de CPU y RAM durante la prueba de estabilidad en Kubernetes Minikube.	45
5.8. Tiempo de recuperación del contenedor cam1 en cada iteración en Docker Compose.	46
5.9. Tiempo de recuperación del contenedor cam1 en cada iteración en Kubernetes (Minikube).	46

6.1. Interfaz gráfica original de Voctomix al clonar el repositorio.	64
--	----

Índice de tablas

2.1. Beneficios y desafíos de la producción audiovisual remota (REMI).	7
2.2. Comparativa entre producción audiovisual tradicional y producción remota (REMI).	8
2.3. Comparativa de protocolos e interfaces de contribución audiovisual.	10
2.4. Comparativa de códecs de contribución audiovisual.	12
2.5. Comparativa de protocolos de distribución audiovisual en directo.	13
2.6. Comparativa de códecs de vídeo para distribución en directo.	15
2.7. Casos de uso representativos de producción remota audiovisual.	17
3.1. Puertos del sistema Voctomix 2.0.	18
4.1. Escenarios de despliegue de Voctomix 2.0.	34
5.1. Especificaciones del entorno de pruebas.	39
5.2. Funcionalidades verificadas durante las pruebas.	40
6.1. Consecución de los objetivos específicos del TFG.	49
6.2. Presupuesto económico del proyecto.	61

Lista de acrónimos

AAC	<i>Advanced Audio Coding</i>
ABR	<i>Adaptive Bitrate</i>
AES	<i>Advanced Encryption Standard</i>
AFV	<i>Audio Follows Video</i>
AKS	<i>Azure Kubernetes Service</i>
AMQP	<i>Advanced Message Queuing Protocol</i>
API	<i>Application Programming Interface</i>
ARQ	<i>Automatic Repeat Request</i>
ASIC	<i>Application-Specific Integrated Circuit</i>
AVC	<i>Advanced Video Coding</i>
CCC	<i>Chaos Communication Congress</i>
CDN	<i>Content Delivery Network</i>
CPU	<i>Central Processing Unit</i>
CSS	<i>Cascading Style Sheets</i>
DASH	<i>Dynamic Adaptive Streaming over HTTP</i>
EBU	<i>European Broadcasting Union</i>
EKS	<i>Elastic Kubernetes Service</i>
ETSIT	<i>Escuela Técnica Superior de Ingenieros de Telecomunicación</i>
GKE	<i>Google Kubernetes Engine</i>
GPU	<i>Graphics Processing Unit</i>
GUI	<i>Graphical User Interface</i>
HDR	<i>High Dynamic Range</i>
HEVC	<i>High Efficiency Video Coding</i>
HLS	<i>HTTP Live Streaming</i>
HTTP	<i>Hypertext Transfer Protocol</i>
IMF	<i>Interoperable Master Format</i>
IP	<i>Internet Protocol</i>
ITU-R	<i>International Telecommunication Union – Radiocommunication Sector</i>

ITU-T	<i>International Telecommunication Union – Telecommunication Standardization Sector</i>
JPEG	<i>Joint Photographic Experts Group</i>
JSON	<i>JavaScript Object Notation</i>
LOPD	<i>Ley Orgánica de Protección de Datos</i>
LTS	<i>Long Term Support</i>
MKV	<i>Matroska</i>
MPEG	<i>Moving Picture Experts Group</i>
MXF	<i>Material eXchange Format</i>
NIST	<i>National Institute of Standards and Technology</i>
OBS	<i>Open Broadcaster Software</i>
OTT	<i>Over The Top</i>
PCM	<i>Pulse-Code Modulation</i>
PIP	<i>Picture-in-Picture</i>
PNG	<i>Portable Network Graphics</i>
RAM	<i>Random Access Memory</i>
RAPL	<i>Running Average Power Limit</i>
REMI	<i>Remote Integration Model</i>
RGPD	<i>Reglamento General de Protección de Datos</i>
RTP	<i>Real-time Transport Protocol</i>
RTMP	<i>Real-Time Messaging Protocol</i>
RTSP	<i>Real Time Streaming Protocol</i>
RTVE	<i>Radiotelevisión Española</i>
SDI	<i>Serial Digital Interface</i>
SMPTE	<i>Society of Motion Picture and Television Engineers</i>
SRT	<i>Secure Reliable Transport</i>
SSH	<i>Secure Shell</i>
TCP	<i>Transmission Control Protocol</i>
TDP	<i>Thermal Design Power</i>
TDT	<i>Televisión Digital Terrestre</i>
TFG	<i>Trabajo Fin de Grado</i>
UDP	<i>User Datagram Protocol</i>

UPM	<i>Universidad Politécnica de Madrid</i>
VPN	<i>Virtual Private Network</i>
VVC	<i>Versatile Video Coding</i>
WAN	<i>Wide Area Network</i>
YAML	<i>YAML Ain't Markup Language</i>

1. Introducción y objetivos

1.1. Introducción

La producción audiovisual en directo ha experimentado en los últimos años una profunda transformación, impulsada por la democratización del *hardware* de bajo coste y el crecimiento exponencial del *streaming* (transmisión en flujo) en Internet [1]. Eventos técnicos de referencia como el Chaos Communication Congress (CCC), organizado por el Chaos Computer Club en Alemania, han puesto de manifiesto la necesidad de disponer de herramientas de realización profesional que sean al mismo tiempo económicas, fiables y auditables.

En este contexto surge Voctomix [2, 3], desarrollado por el equipo C3VOC (*Chaos Communication Congress Video Operations Crew*). Una solución previa, *dvswitch* [4], estaba limitada en resolución y escalabilidad. Otras herramientas como vMix [5] (comercial) u *Open Broadcaster Software* (OBS) Studio (código abierto) [6] concentran captura, procesamiento y emisión en un único nodo de operación, debido a su arquitectura monolítica. Voctomix, en cambio, adopta una arquitectura cliente-servidor desacoplada que separa el motor de mezcla multimedia de la interfaz de control, lo que permite distribuir la carga de procesamiento en distintos nodos de red.

El presente Trabajo Fin de Grado (TFG) toma como punto de partida la versión original de Voctomix y propone una serie de extensiones orientadas a su uso en producciones en directo de mediana escala: rotulación dinámica sobre el vídeo, un sistema de control de estados de emisión (*stream blanker*), *composites* avanzados de vídeo, telemetría en tiempo real mediante un *broker* de mensajería, y la contenerización completa del sistema mediante Docker Compose y Kubernetes. El resultado es Voctomix 2.0, una plataforma más versátil, desplegable y adaptada a flujos de trabajo profesionales.

Un elemento diferenciador de este proyecto es su validación en un entorno real: el sistema se integra en el proyecto europeo de investigación CyberNEMO [7], liderado por la Universidad Politécnica de Madrid (UPM), orientado a la ciberseguridad aplicada a la producción y distribución multimedia. Este trabajo se ha llevado a cabo en el marco de una beca de investigación en el grupo GATV de la UPM.

En ese contexto, Voctomix 2.0 actúa como motor de producción audiovisual del proyecto y el sistema de telemetría con RabbitMQ proporciona la monitorización en tiempo real del sistema. Además, responde a las necesidades de retransmisión de seminarios y eventos técnicos con calidad profesional sin depender de *hardware* costoso, y puede desplegarse de forma sencilla en cualquier equipo Linux estándar.

Las secciones siguientes detallan los objetivos concretos de este trabajo y la organización

de la memoria.

1.2. Objetivos

El objetivo principal de este trabajo es extender Voctomix hacia una plataforma de código abierto, modular y reproducible para la producción remota de vídeo en tiempo real. Partiendo de Voctomix como base, se busca construir un sistema completo que permita realizar producciones audiovisuales de calidad profesional sin depender de *hardware* propietario de alto coste, y que pueda desplegarse fácilmente mediante contenedores.

Para lograrlo, se plantean los siguientes objetivos específicos:

1. Ampliar las funcionalidades del mezclador con características propias de la producción profesional: modos de composición de imagen con transiciones animadas, rotulación dinámica con gráficos y textos en tiempo real, un gestor de estados de emisión y sincronización automática del audio con el vídeo.
2. Implementar un servicio de telemetría que exporte el estado del sistema en formato *JavaScript Object Notation* (JSON) y lo publique en el gestor de colas RabbitMQ, de forma que se pueda integrar con herramientas externas sin tocar el núcleo de la aplicación.
3. Contenerizar el sistema completo con Docker Compose, de modo que cualquiera pueda desplegarlo sin preocuparse por dependencias ni configuraciones del entorno.
4. Extender el despliegue a Kubernetes, validando que el sistema funciona también en infraestructuras más complejas y multinodo.
5. Validar el sistema en distintos escenarios de despliegue, desde un único ordenador hasta un clúster de Kubernetes, midiendo la latencia de conmutación y de transición entre modos de composición, la estabilidad del *pipeline* y el correcto funcionamiento del sistema de telemetría en cada configuración.
6. Demostrar la aplicabilidad del sistema en un contexto de producción real, mediante su integración en el proyecto europeo CyberNEMO de la Universidad Politécnica de Madrid, donde opera como motor de producción audiovisual en un entorno de investigación activo.

1.3. Estructura de la memoria

La presente memoria se organiza en seis capítulos y tres anexos. El presente capítulo recoge la introducción y motivación del trabajo, los objetivos específicos y la estructura de la memoria.

El Capítulo 2 presenta el estado del arte en producción audiovisual: la evolución desde la producción tradicional hasta el modelo remoto basado en *software*, los protocolos de red y los estándares de contribución y distribución de vídeo, y una comparativa de enfoques y herramientas de producción remota por *software*. Se incluyen también casos de uso reales de producción remota a escala industrial.

El Capítulo 3 describe el diseño e implementación del sistema Voctomix 2.0: la arquitectura cliente-servidor (*voccore* y *vocgui*), las fuentes de señal de cámaras y auxiliares, los modos de composición, las transiciones, el sistema de control de estados de emisión (*stream blanker*), el sistema de *overlays* y rotulación dinámica, y el servicio de telemetría mediante RabbitMQ.

El Capítulo 4 detalla el despliegue del sistema en sus distintos escenarios: en entorno nativo tanto en un único equipo como en dos equipos, el despliegue contenerizado con Docker Compose y el despliegue orquestado con Kubernetes.

El Capítulo 5 describe el entorno y la metodología de pruebas, la validación funcional del sistema y las pruebas de rendimiento en los escenarios de despliegue, con mediciones de latencia de conmutación, consumo de recursos, estabilidad a largo plazo, resiliencia ante fallos y registros de telemetría.

El Capítulo 6 presenta las conclusiones del trabajo y las líneas de trabajo futuro identificadas.

Los Anexos incluyen los aspectos éticos, económicos, sociales y ambientales del proyecto (Anexo A), el presupuesto económico detallado (Anexo B) y la evolución del repositorio a lo largo del desarrollo del proyecto (Anexo C).

2. Estado del arte

En este capítulo se presentan las bases tecnológicas sobre las que se sustenta Voctomix 2.0. El capítulo se estructura en cuatro secciones: el modelo de producción audiovisual y las herramientas de realización disponibles (Sección 2.1), los estándares, protocolos e interfaces de contribución de señal (Sección 2.2), los protocolos y códecs empleados en la distribución del contenido al espectador final (Sección 2.3), y los casos de uso reales de producción remota que contextualizan la adopción industrial de estas tecnologías (Sección 2.4). El objetivo de este capítulo es proporcionar el contexto técnico necesario para abordar el diseño e implementación del sistema en el Capítulo 3.

2.1. Producción audiovisual

La producción audiovisual en directo engloba el conjunto de procesos técnicos y organizativos que permiten capturar, procesar y distribuir contenido de vídeo y audio en tiempo real. Históricamente ligada a infraestructuras físicas costosas y al desplazamiento de equipos al lugar del evento, esta disciplina ha cambiado profundamente en la última década gracias a la integración de las redes *Internet Protocol* (IP) en los flujos de producción profesional [8].

2.1.1. Producción audiovisual tradicional

El modelo de producción tradicional se basa en el despliegue físico de toda la infraestructura técnica y del personal en el emplazamiento del evento. El componente central de este modelo es el mezclador *hardware*, denominado *switcher* (conmutador de vídeo), un dispositivo dedicado que selecciona en cada instante qué fuente de vídeo se emite a la salida de programa y gestiona las transiciones y los efectos. Fabricantes como Blackmagic Design (ATEM), Ross Video (Carbonite) o Grass Valley (Kayenne) dominan el mercado de estos equipos, que ofrecen latencias sub-imagen, múltiples buses de programa y previsualización simultáneos, y redundancia *hardware* certificada para operación continua [9].

En el modelo tradicional, todas las fuentes, cámaras, reproductores de vídeo y ordenadores de presentación, se interconectan mediante cableado SDI (*Serial Digital Interface*), el estándar histórico de la industria *broadcast* (radiodifusión) para el transporte de vídeo sin comprimir [10]. La señal resultante recorre el mezclador y los sistemas de rotulación y grabación, todo ello dentro del mismo emplazamiento físico, para ser enviada posteriormente a la etapa de codificación y distribución. La figura 2.1 ilustra esta cadena completa.

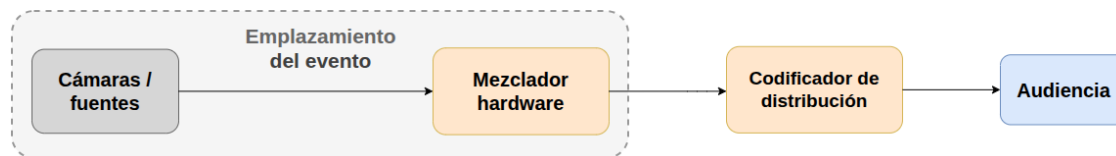


Figura 2.1: Cadena de producción audiovisual tradicional.

Este modelo ha demostrado su solidez durante décadas en las grandes producciones de televisión: retransmisiones deportivas, eventos parlamentarios y galas de premios emplean esta arquitectura con decenas de fuentes de cámara gestionadas en tiempo real. Sin embargo, su principal limitación es económica y logística: el coste de adquirir y transportar el equipamiento, más el personal técnico necesario para su instalación y operación, lo pone fuera del alcance de organizaciones de mediana escala.

Un caso ilustrativo es la retransmisión de un partido de La Liga de fútbol profesional en España. Este tipo de producción moviliza habitualmente entre 20 y 30 cámaras SDI, varias unidades móviles de producción equipadas con mezcladores *hardware* de gama alta y un equipo técnico de entre 50 y 100 personas desplazadas al estadio. El coste de una unidad móvil de estas características puede alcanzar varios millones de euros, cifra a la que se suma el gasto recurrente en desplazamientos, montaje y operación. Este modelo queda así reservado a grandes cadenas de televisión y organizaciones con infraestructura *broadcast* consolidada, resultando inaccesible para entidades académicas, asociaciones o eventos de mediana escala.

2.1.2. Producción audiovisual remota

La producción remota, conocida en la industria por el acrónimo **REMI** (*Remote Integration Model*), surge para resolver estas limitaciones permitiendo que la realización se lleve a cabo desde un centro de producción centralizado, mientras en el lugar del evento solo se mantiene el equipo técnico mínimo necesario para la captura de señal [8].

Arquitectura técnica del modelo REMI

La figura 2.2 ilustra la separación funcional entre el lugar del evento y el centro de producción remoto. En el emplazamiento solo permanecen las cámaras, los micrófonos y un sistema de contribución IP que codifica y transmite la señal hacia el centro remoto. El procesamiento completo, mezcla de fuentes, inserción de gráficos, control de audio, gestión de *overlays* y distribución al espectador, se realiza de forma centralizada.

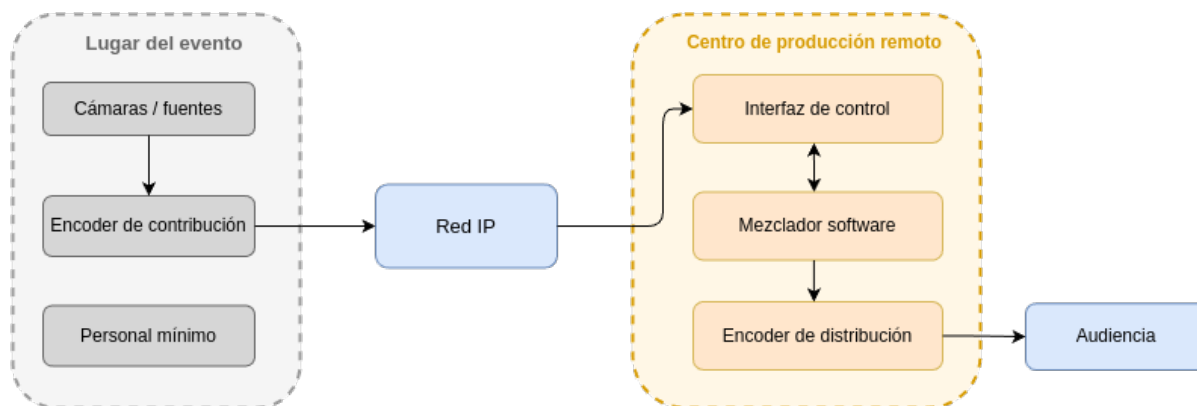


Figura 2.2: Arquitectura del modelo REMI.

Esta separación permite que el realizador opere el sistema desde cualquier ubicación con acceso a la red, y que el centro de producción pueda atender simultáneamente eventos en distintas localizaciones reutilizando la misma infraestructura de mezclado [8].

El modelo REMI se ha consolidado en los últimos años como referencia en eventos de gran escala. Los casos representativos, desde el Chaos Communication Congress hasta los Juegos Olímpicos de Tokio 2020, se analizan en la Sección 2.4.

Ventajas del modelo remoto

Las ventajas del modelo REMI frente a la producción tradicional son significativas [8]:

- **Reducción de costes:** se minimiza el desplazamiento de personal y equipos. Solo se requiere *in situ* el equipo de captura; la realización, mezcla y edición se realizan de forma centralizada.
- **Sostenibilidad:** la reducción de desplazamientos contribuye a disminuir la huella de carbono de las producciones, un factor cada vez más valorado por organizaciones y radiodifusores.
- **Escalabilidad:** el sistema puede adaptarse a producciones de diferente escala reutilizando la misma infraestructura centralizada.
- **Flexibilidad geográfica:** es posible retransmitir eventos desde cualquier parte del mundo con conectividad IP, sin necesidad de desplazar el equipo de producción.

La adopción del modelo REMI no está exenta de retos, entre ellos la latencia introducida al transportar la señal sobre una red de área amplia (*Wide Area Network* (WAN)). La tabla 2.1 resume los principales beneficios y desafíos identificados en entornos de producción reales [8].

Tabla 2.1: Beneficios y desafíos de la producción audiovisual remota (REMI).

Dimensión	Beneficio	Desafío
Coste	Reducción de desplazamientos y equipos	Inversión inicial en red y <i>software</i>
Latencia	Viable con SRT (<1 s en WAN)	Coordinación realizador-cámaras difícil
Escalabilidad	Un centro atiende múltiples eventos	Número de fuentes de cámara simultáneas limitado por ancho de banda y capacidad de procesamiento
Red	Producción desde cualquier ubicación IP	Corte de red detiene la producción
Calidad	<i>hardware</i> ilimitado en el centro remoto	Artefactos con ancho de banda insuficiente
Sincronía A/V	Procesamiento centralizado facilita la sincronía entre fuentes	<i>jitter</i> (fluctuación) puede desincronizar fuentes
Seguridad	Cifrado AES-256 disponible en transporte SRT	Mayor superficie de ataque expuesta
Complejidad	Herramientas maduras (FFmpeg, GStreamer)	Diagnóstico de <i>pipelines</i> distribuidos
Sostenibilidad	Menor huella de carbono	Consumo energético del centro permanente

Herramientas de producción remota por *software*

La disponibilidad de herramientas de mezclado por *software* ha democratizado el acceso al modelo remoto incluso para organizaciones sin presupuesto de radiodifusión profesional:

- **OBS Studio** [6]: solución de código abierto ampliamente utilizada para *streaming* y grabación. Su diseño todo en uno concentra captura, composición y emisión en una única aplicación para un único operador. Soporta fuentes de entrada variadas, como cámaras USB, captura de pantalla o fuentes IP, con soporte para transiciones y filtros. Es la opción más utilizada por *streamers* independientes, aunque su diseño centralizado limita su uso en entornos distribuidos.
- **vMix** [5]: solución comercial profesional para Windows con soporte *multiview* (vista múltiple) y producción 4K. Permite gestionar múltiples fuentes, aplicar *chroma key* (croma) y transiciones, y emitir a varias plataformas simultáneamente. Su coste de licencia, que puede superar los 1.000 €, y su restricción a Windows la excluyen de entornos de código abierto sobre Linux.
- **Voctomix** [2]: herramienta de código abierto con arquitectura cliente-servidor que separa el motor de mezcla (*voctocore*) de la interfaz de control (*voctogui*), permitiendo que el procesamiento se realice en un servidor mientras el operador trabaja desde otro equipo conectado por red. Construido sobre GStreamer [11], un *framework* (entorno de *software*) multimedia de código abierto, proporciona composición de vídeo con múltiples fuentes, transiciones animadas, *overlays* y un protocolo de control TCP extensible. Es la herramienta objeto del presente trabajo (véase el

Capítulo 3).

La tabla 2.2 sintetiza los parámetros diferenciadores entre ambos modelos.

Tabla 2.2: Comparativa entre producción audiovisual tradicional y producción remota (REMI).

Parámetro	Tradicional	REMI / Software
Infraestructura en lugar del evento	Completa	Mínima (cámaras + <i>encoder</i>)
Coste de equipamiento	Muy alto	Bajo (<i>hardware</i> convencional)
Personal <i>in situ</i>	Numeroso	Reducido
Latencia extremo a extremo	Sub-imagen	1 fotograma – varios segundos
Escalabilidad	Limitada	Alta
Huella de carbono	Alta	Baja
Ejemplo	Blackmagic ATEM	OBS, vMix, Voctomix

La figura 2.3 ilustra los bloques funcionales de la cadena completa: captura y contribución de señal, mezcla y procesamiento, codificación y distribución al espectador final.

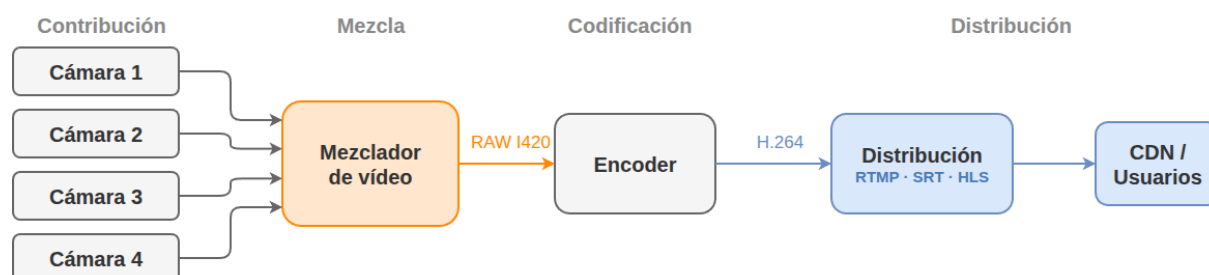


Figura 2.3: Cadena de producción audiovisual en directo.

2.2. Contribución audiovisual

La contribución audiovisual designa el transporte de la señal de vídeo y audio desde las fuentes de captura, cámaras, ordenadores de presentación y reproductores, hasta el mezclador o núcleo del sistema de producción. Es el primer eslabón de la cadena y su elección determina la calidad máxima disponible, la latencia de extremo a extremo y el coste de la infraestructura. En esta sección se describen los protocolos e interfaces, los contenedores multimedia y los códecs empleados en esta etapa.

2.2.1. Protocolos e interfaces de contribución

Los mecanismos de contribución audiovisual se articulan en torno a dos familias: las interfaces físicas basadas en SDI, históricamente dominantes en el ámbito *broadcast* profesional, y los protocolos IP, que transportan la señal sobre redes Ethernet estándar. Los protocolos

IP de contribución se apoyan en *User Datagram Protocol* (UDP) (RFC 768 [12]) para flujos en tiempo real donde la temporización es prioritaria sobre la integridad, como en RTP y SRT, o en *Transmission Control Protocol* (TCP) (RFC 793 [13]) cuando se requiere entrega ordenada garantizada, como en el transporte de fuentes de cámara o el protocolo de control del sistema. La elección entre una y otra familia depende del entorno: estudio con infraestructura SDI existente, red local de alta velocidad o contribución remota sobre Internet.

- **SDI:** estándar histórico de la industria *broadcast* para el transporte de vídeo sin comprimir sobre cable coaxial o fibra óptica [10]. Ofrece latencia sub-imagen y sincronismo garantizado, pero está limitado a distancias de hasta 100 m sin regeneradores y requiere infraestructura de alto coste.
- ***Society of Motion Picture and Television Engineers* (SMPTE) ST 2022-6 (SDI sobre IP):** encapsula la señal SDI íntegra en paquetes UDP/IP, facilitando la migración de infraestructuras SDI existentes hacia redes Ethernet estándar [14]. La integridad del flujo puede reforzarse opcionalmente mediante el estándar complementario SMPTE ST 2022-7, que implementa conmutación sin corte (*hitless failover*) entre dos flujos redundantes.
- **SMPTE ST 2110 (producción IP nativa):** estándar publicado en 2017 que transporta vídeo, audio y metadatos como flujos IP independientes, proporcionando mayor flexibilidad de encaminamiento que SDI [15]. La sincronización entre flujos se consigue mediante PTP (IEEE 1588) [16], con precisión de microsegundos.
- **SRT (*Secure Reliable Transport*):** protocolo de código abierto desarrollado por Haivision con corrección de errores *Automatic Repeat Request* (ARQ) y cifrado *Advanced Encryption Standard* (AES)-128/256 [17]. Permite transportar la señal desde el lugar del evento hasta el centro de producción remoto sobre Internet pública con latencia inferior a 1 s. FFmpeg lo soporta como entrada y salida, lo que permite integrarlo directamente en *pipelines* de Voctomix.
- **RTP (*Real-time Transport Protocol*):** protocolo de transporte en tiempo real definido en la RFC 3550 [18], utilizado junto con RTSP para la distribución de flujos multimedia desde cámaras IP profesionales.
- **RTSP (*Real Time Streaming Protocol*):** protocolo de señalización definido en la RFC 7826 [19] que se emplea para establecer y controlar sesiones de *streaming* multimedia. Actúa como mando a distancia del flujo RTP: permite iniciar, pausar y detener la transmisión de una fuente IP.

La tabla 2.3 recoge comparativamente los parámetros clave de cada protocolo e interfaz.

Desarrollo y optimización de un sistema modular de código abierto
para la producción remota de vídeo en tiempo real

Tabla 2.3: Comparativa de protocolos e interfaces de contribución audiovisual.

Protocolo	Medio	Velocidad máx.	Latencia	Uso típico
SDI (12G)	Coaxial/fibra	11,88 Gbit/s	Sub-imagen	Estudio <i>broadcast</i>
SMPTE ST 2022-6	UDP/IP	3 Gbit/s	Sub-imagen	Migración SDI/IP
SMPTE ST 2110	UDP/IP	Variable	Sub-imagen	Producción IP nativa
SRT	UDP/IP (ARQ)	Variable	<1 s	Contribución REMI
RTP/RTSP	UDP/IP	Variable	Sub-imagen	Cámaras IP

2.2.2. Contenedores multimedia de contribución

En producción *broadcast* profesional, el contenedor multimedia no solo encapsula los flujos de vídeo y audio, sino que transporta también metadatos de producción, timecode y descriptores técnicos de cada pista. Los formatos más empleados en la etapa de contribución son:

- **MXF (*Material eXchange Format*, SMPTE ST 377-1)** [20]: estándar de la industria *broadcast* definido por la SMPTE y adoptado por la *European Broadcasting Union* (EBU) y la mayoría de fabricantes de cámaras y sistemas de archivo profesionales. Encapsula vídeo comprimido (*Moving Picture Experts Group* (MPEG)-2, AVC-Intra, ProRes), audio multicanal y metadatos de producción en un único fichero con soporte de *timecode frame-accurate*. Fabricantes como Sony (XDCAM), Panasonic (P2), Canon (XF-AVC [21]) y Grass Valley utilizan MXF como formato de grabación y contribución nativo, convirtiéndolo en el contenedor de intercambio estándar entre el lugar del evento y el centro de producción.
- **Matroska (MKV)** [22]: contenedor de código abierto empleado por Voctomix como formato de transporte interno para señal RAW. Las fuentes envían vídeo I420 y audio PCM S16LE a 48 kHz encapsulados en MKV sobre conexiones TCP, garantizando sincronía audio-vídeo sin compresión intermedia. A diferencia de MXF, no es un estándar *broadcast* certificado, pero su nula sobrecarga de codificación lo hace idóneo para la contribución de señal sin comprimir en redes locales de alta velocidad.

2.2.3. Códecs de contribución

En la etapa de contribución se prioriza la calidad y la mínima degradación sobre la eficiencia de compresión. Los códecs utilizados son de alta calidad, generalmente intra-frame, para evitar la acumulación de artefactos en cadenas de producción con múltiples etapas de procesamiento.

- **Vídeo sin comprimir (RAW I420):** el vídeo se transporta sin ningún tipo de compresión, preservando íntegramente toda la información de la señal original. El formato I420 (YUV 4:2:0 planar) organiza los datos en tres planos separados: el plano de luminancia Y a resolución completa y los dos planos de crominancia Cb

y Cr a resolución reducida a la mitad en ambas dimensiones. Este submuestreo de crominancia aprovecha la menor sensibilidad del sistema visual humano a las variaciones de color, resultando en 1,5 B/píxel de promedio frente a los 3 B/píxel de RGB sin comprimir. El ancho de banda de un flujo I420 a 1080p25 es determinista e independiente del contenido visual:

$$622,08 \text{ Mbit/s} = \frac{1920 \times 1080 \times 1,5 \text{ B/píxel} \times 25 \text{ fps} \times 8}{10^6}$$

Voctomix emplea I420 en todo el procesamiento interno del motor GStreamer mediante cápsulas `video/x-raw,format=I420`, lo que elimina cualquier conversión de formato durante el procesamiento y garantiza que la latencia de mezcla dependa únicamente del tiempo de cálculo sobre los píxeles. El coste del formato RAW es que los 622 Mbit/s por fuente obligan a que todo el tráfico de vídeo circule en bucle local, ya sea mediante la interfaz de loopback del servidor en instalación monopuesto o a través de redes Docker internas de alta velocidad en el despliegue containerizado.

- **Audio PCM (*Pulse-Code Modulation*)**: el audio se transporta sin compresión en formato PCM S16LE (muestras enteras de 16 bits en orden *little-endian*) a 48 kHz estéreo. Voctomix utiliza este formato para todo el procesamiento interno de audio, encapsulado junto al vídeo I420 en el mismo contenedor MKV. La ausencia de compresión garantiza la máxima calidad y elimina cualquier latencia de codificación.
- **Apple ProRes** [23]: familia de códecs de alta calidad desarrollados por Apple para producción profesional. Utilizan exclusivamente compresión intra-frame, lo que minimiza la degradación acumulativa en múltiples etapas de procesamiento. ProRes 422 HQ está ampliamente implantado en producción televisiva, especialmente en flujos de trabajo basados en Final Cut Pro.
- **VC-3 / DNxHD (SMPTE ST 2019)** [24, 25]: códec intra-frame estandarizado por SMPTE como VC-3 y distribuido comercialmente por Avid bajo el nombre DNxHD (y su variante de alta resolución DNxHR). Equivalente a ProRes en su propósito, está ampliamente implantado en flujos de trabajo de postproducción *broadcast* y en sistemas de edición Avid Media Composer.
- **Joint Photographic Experts Group (JPEG) 2000** [26]: códec que ofrece compresión sin pérdidas o con pérdidas mínimas y latencia muy reducida gracias a su arquitectura de codificación por bloques independientes. Utilizado en sistemas de contribución *broadcast* de alta calidad y en señalización digital.
- **JPEG XS (ISO/IEC 21122)** [27]: códec intra-frame de nueva generación publicado en 2019 específicamente diseñado para producción profesional en tiempo real. Su latencia de codificación es inferior a un milisegundo con compresiones de 4:1 a 6:1 en entornos de producción profesional típicos, sin pérdidas visualmente relevantes, lo que lo hace adecuado para contribución 4K y 8K en vivo. Ha sido adoptado en iniciativas de producción IP de alta calidad donde JPEG 2000 resulta excesivamente exigente computacionalmente.

La tabla 2.4 resume las características principales de cada códec en términos de compresión y latencia.

Tabla 2.4: Comparativa de códecs de contribución audiovisual.

Códec	Compresión	Latencia enc.	Uso típico
RAW (I420)	Sin compresión	Nula	Vídeo interno Voctomix / estudio
PCM S16LE (audio)	Sin compresión	Nula	Audio interno Voctomix
Apple ProRes	4:1 – 8:1	<1 fotograma	Producción TV, postproducción
VC-3 / DNxHD	4:1 – 7:1	<1 fotograma	Broadcast, Avid workflows
JPEG 2000	4:1 – 20:1	<5 ms	Contribución <i>broadcast</i>
JPEG XS	4:1 – 6:1	<1 ms	Contribución 4K/8K en vivo

2.3. Distribución audiovisual

Una vez generada la señal de programa por el mezclador, esta debe distribuirse hasta la audiencia. Esta etapa difiere fundamentalmente de la contribución: mientras que en la contribución se prioriza la calidad y la latencia mínima, en la distribución el objetivo es llegar al máximo número de espectadores con los recursos de red disponibles, lo que obliga a comprimir la señal de forma agresiva y a equilibrar el compromiso entre latencia, calidad y escalabilidad.

2.3.1. Protocolos de distribución

Los protocolos de distribución gobiernan cómo la señal de programa viaja desde el mezclador hasta las plataformas de *streaming* y, desde ahí, hasta el reproductor del espectador. Dos de ellos, HLS y MPEG-DASH, delegan su transporte en HTTP [28], lo que les permite aprovechar la infraestructura CDN existente sin configuración específica de *media*.

- **RTMP (*Real-Time Messaging Protocol*)**: protocolo TCP de Adobe, estándar de ingesta para plataformas como YouTube Live o Twitch con latencia de 2–5 s. Aunque obsoleto en reproducción desde la retirada de Adobe Flash en 2020, mantiene una adopción del 58 % entre profesionales en 2025 [29] y sigue siendo el protocolo de ingesta aceptado de forma universal por las principales plataformas *Content Delivery Network* (CDN) (YouTube, Twitch, Facebook Live). Voctomix lo soporta como salida mediante FFmpeg.
- **HLS (*HTTP Live Streaming*)**: protocolo de Apple que divide el vídeo en segmentos servidos mediante HTTP [30]. Latencia de 10–30 s en modo estándar, reducible a 2–5 s con *Low-Latency HLS*. Es el protocolo de reproducción más compatible, con soporte nativo en prácticamente todos los dispositivos y CDN.
- **MPEG-DASH (*Dynamic Adaptive Streaming over HTTP*)**: estándar abierto ISO/IEC 23009-1 [31] equivalente a HLS pero sin dependencia de proveedor. Divide el contenido en segmentos HTTP con ajuste adaptativo de calidad (*Adaptive Bitrate* (ABR)) y es empleado por YouTube, Netflix y Amazon Prime. Puede generarse con FFmpeg como alternativa abierta a HLS, siendo compatible con el *pipeline* de salida del sistema.
- **WebRTC (*Web Real-Time Communication*)**: estándar W3C para comunicación multimedia en tiempo real desde el navegador, con latencias de 100–500 ms.

Su escalabilidad para audiencias masivas es limitada, pero resulta ideal para monitorización remota del sistema de producción y retornos entre el equipo técnico.

La tabla 2.5 compara los parámetros clave de cada protocolo.

Tabla 2.5: Comparativa de protocolos de distribución audiovisual en directo.

Protocolo	Latencia	Transporte	Cifrado	Uso principal
RTMP	2–5 s	TCP	No (RTMPS sí)	Ingesta a plataformas
HLS	2–30 s	HTTP	HTTPS	Reproducción masiva (CDN)
MPEG-DASH	2–30 s	HTTP	HTTPS	<i>Streaming</i> adaptativo abierto
WebRTC	100–500 ms	UDP	Sí (integrado)	Monitorización, retornos

2.3.2. Contenedores multimedia de distribución

El formato contenedor de distribución define cómo se empaquetan los flujos de vídeo y audio codificados para su transporte hasta el reproductor final.

- **MPEG-TS (*Transport Stream*, ISO 13818-1)** [32]: estándar diseñado específicamente para transmisión *broadcast* con tolerancia a errores. Es el contenedor asociado a HLS, la emisión por satélite y la *Televisión Digital Terrestre* (TDT). Su robustez frente a pérdidas de paquetes lo hace el formato de referencia para distribución *broadcast*.
- **MP4 / ISOBMFF (*ISO Base Media File Format*)** [33]: contenedor de referencia para distribución por HTTP progresivo y *streaming* adaptativo (MPEG-DASH, HLS fragmentado). Es el formato de almacenamiento estándar para grabaciones y distribución bajo demanda.
- **IMF (*Interoperable Master Format*, serie SMPTE ST 2067)** [34]: estándar de mastering desarrollado por la SMPTE y adoptado por la EBU. Construido sobre MXF, incorpora una *Composition Playlist* en XML que permite gestionar variantes lingüísticas, de resolución y de rango dinámico como paquetes suplementarios sobre un maestro base, sin duplicar la esencia de vídeo. Soporta vídeo JPEG 2000 o HEVC, audio PCM y subtítulos TTML (*Timed Text Markup Language*). Plataformas como Netflix, Amazon Prime Video y Disney+ lo exigen para la entrega de contenido original, consolidándolo como referencia en distribución *Over The Top* (OTT) de alta calidad.
- **MOV (*QuickTime Movie*)** [35]: contenedor desarrollado por Apple, ampliamente utilizado en cámaras de producción profesional como formato de grabación nativo (Sony, Canon, Blackmagic). Comparte estructura con MP4/ISOBMFF al ser su antecedente directo. Es habitual en flujos de distribución hacia plataformas de edición y archivado.
- **FLV (*Flash Video*)**: contenedor legado asociado a RTMP. Aunque técnicamente obsoleto como formato de reproducción desde la retirada de Adobe Flash en 2020,

sigue siendo el contenedor utilizado internamente en el protocolo RTMP para la ingesta hacia plataformas de *streaming* en directo.

2.3.3. Códecs de distribución

En la etapa de distribución se emplean códecs de alta eficiencia que comprimen la señal para reducir el ancho de banda necesario hasta el espectador final, asumiendo cierta pérdida de calidad a cambio de viabilizar el transporte.

- **H.264 / AVC (*Advanced Video Coding*)** [36]: el códec de vídeo más extendido en la actualidad y la línea de base de la industria del *streaming*. Introdujo herramientas avanzadas de predicción inter-fotograma que establecieron en 2003 el estándar de eficiencia vigente durante dos décadas. H.264 se organiza en perfiles que determinan el conjunto de herramientas de codificación disponibles; en producción audiovisual se emplea principalmente el perfil *High*, soportado por la totalidad de plataformas CDN y decodificadores *hardware* desde 2010.

Voctomix emplea H.264 como códec de salida por defecto a través de FFmpeg y la biblioteca de *software* libre `libx264`. La señal de programa RAW I420 que emite voctocore es consumida directamente por FFmpeg para codificación en tiempo real, sin etapa de conversión de formato intermedia. La salida se encapsula en MPEG-TS para ingesta mediante RTMP o se segmenta en HLS para distribución adaptativa. La universalidad del perfil *High* garantiza compatibilidad con YouTube Live, Twitch y cualquier CDN comercial sin configuración adicional, mientras que la velocidad de codificación de `libx264` permite realizar la codificación en tiempo real sin aceleración *hardware* en CPU de gama media.

- **H.265 / HEVC (*High Efficiency Video Coding*)**: sucesor de H.264 que aproximadamente duplica la eficiencia de compresión, permitiendo la misma calidad perceptual a la mitad del *bitrate* [37]. Su codificación es 5–8 veces más lenta que H.264, lo que lo hace adecuado para *streaming* en tiempo real cuando se dispone de aceleración *hardware* (*Graphics Processing Unit* (GPU) o *Application-Specific Integrated Circuit* (ASIC)).
- **AV1**: códec abierto desarrollado por la Alliance for Open Media que mejora la eficiencia de H.265 en aproximadamente un 50 % adicional [38]. Su limitación histórica en producción en directo ha sido la velocidad de codificación: AV1 tarda entre 5 y 10 veces más que H.265 en *encoders* de referencia, aunque el *encoder* SVT-AV1 (Intel/Netflix) permite ya codificación en tiempo real en *software* sobre CPU modernas de gama alta. YouTube, Netflix y Meta ya codifican gran parte de su contenido en AV1 para *streaming* bajo demanda.
- **H.266 / VVC (*Versatile Video Coding*)** [39]: sucesor de H.265 publicado en 2020 por el Joint Video Experts Team (JVET) de *International Telecommunication Union – Telecommunication Standardization Sector* (ITU-T) e ISO/IEC. Aproximadamente duplica la eficiencia de compresión de H.265, permitiendo la misma calidad perceptual a la mitad del *bitrate* [38]. Diseñado para contenidos 4K, 8K, vídeo de 360° y *High Dynamic Range* (HDR). Su principal limitación actual es la velocidad de codificación, significativamente superior a la de H.265, lo que restringe su uso en tiempo real a entornos con aceleración *hardware* especializada. Plataformas como

Netflix y Disney+ lo evalúan para distribución OTT de próxima generación.

- **AAC / Opus:** códecs de audio para distribución por Internet. AAC (*Advanced Audio Coding*) es el estándar en plataformas de *streaming* (YouTube, Twitch, Apple Music) y en HLS; Opus es el códec de audio obligatorio en WebRTC (RFC 7587) y el preferido en aplicaciones de comunicación en tiempo real por su baja latencia y alta eficiencia a *bitrates* reducidos [40].
- **AC-3 / AC-4 (Dolby):** AC-3 (*Dolby Digital*, ATSC A/52) [41] es el códec de audio dominante en distribución *broadcast* lineal: estándar en la norma ATSC de televisión digital en Norteamérica y ampliamente empleado en cable, satélite, DVD y Blu-ray. Soporta hasta 5.1 canales a tasas de entre 192 kbit/s y 640 kbit/s y es soportado de forma nativa por FFmpeg para la generación de flujos MPEG-TS con audio envolvente. AC-4 (ETSI TS 103 190) [42] es su sucesor de nueva generación, adoptado en el estándar ATSC 3.0 (*NextGen TV*), con soporte para audio inmersivo de hasta 7.1.4 canales compatible con Dolby Atmos y mayor eficiencia de compresión que AC-3 a calidad perceptual equivalente.

La tabla 2.6 resume la eficiencia de compresión relativa y la viabilidad en tiempo real de cada códec de vídeo.

Tabla 2.6: Comparativa de códecs de vídeo para distribución en directo.

Códec	Eficiencia vs H.264	Vel. encoding	Compatibilidad	RT posible
H.264/AVC	Base	Muy rápido	Universal	Sí
H.265/ HEVC	×2	5–8× más lento	Alta (post 2015)	Sí (con hardware)
AV1	×3	5–10× más lento	Media (creciendo)	Sí (hardware o SVT-AV1)
H.266/VVC	×4	Muy lento	Emergente (2024+)	No

2.4. Casos de uso reales de producción remota

La producción remota ha dejado de ser una propuesta experimental para convertirse en el modelo operativo de referencia en eventos de gran escala. En esta sección se recogen cinco casos de uso representativos que ilustran el rango de aplicaciones, desde eventos masivos con presupuesto de televisión pública hasta proyectos europeos de investigación con participación de la UPM.

2.4.1. Chaos Communication Congress (C3VOC)

El Chaos Communication Congress (CCC) es uno de los mayores congresos de tecnología y seguridad informática del mundo, celebrado anualmente en Alemania con más de 17 000 asistentes en sus ediciones de mayor afluencia [43] y múltiples salas en emisión simultánea. El equipo de producción de vídeo del congreso, C3VOC, desarrolló Voctomix [2] precisamente para cubrir esta necesidad: un mezclador *software* de código abierto capaz de gestionar varias salas en paralelo sin depender de *hardware broadcast* propietario.

Desarrollo y optimización de un sistema modular de código abierto
para la producción remota de vídeo en tiempo real

Cada sala del congreso dispone de su propio servidor con una instancia de voctocore, alimentado por cámaras que envían señal RAW I420 sobre TCP/MKV y operado remotamente por un realizador a través de voctogui. Las grabaciones finales y la emisión en directo se publican en el portal de media del congreso (media.ccc.de). Este caso de uso es el origen del proyecto: Voctomix 2.0 hereda la arquitectura cliente-servidor de voctocore originalmente diseñada para este entorno.

2.4.2. Juegos Olímpicos de Tokio 2020

Los Juegos Olímpicos de Tokio 2020, celebrados en 2021, constituyeron uno de los mayores despliegues de producción remota en la historia del olimpismo. Olympic Broadcasting Services implementó un modelo REMI a gran escala en el que la señal de las cámaras de todas las sedes olímpicas se transportó a un centro de producción centralizado en Tokio, reduciendo drásticamente el número de equipos y personal técnico desplazado a cada instalación [44].

Este despliegue validó la viabilidad técnica del modelo REMI para eventos con decenas de sedes simultáneas, señales de alta resolución y requisitos de fiabilidad *broadcast*. La reducción de la huella de carbono asociada al transporte de equipos fue uno de los argumentos adicionales que aceleraron la adopción del modelo remoto como estándar para ediciones futuras.

2.4.3. LaLiga (Telefónica Servicios Audiovisuales)

Telefónica Servicios Audiovisuales fue seleccionada junto con Appear para desplegar la nueva red de contribución y distribución de LaLiga desde Madrid, basada en SRT [45]. Las cámaras de los estadios envían la señal mediante SRT sobre Internet pública hasta el centro de producción remoto, donde los realizadores operan el sistema como si estuvieran físicamente presentes en el estadio.

Este caso de uso demuestra la madurez del protocolo SRT para contribución *broadcast* en condiciones de red reales: el cifrado AES-256 garantiza la seguridad del contenido deportivo durante el transporte, y la corrección ARQ recupera los paquetes perdidos sin impacto perceptible en la latencia. La operación resultó transparente para el equipo de realización, que mantuvo su flujo de trabajo habitual sin adaptaciones significativas.

2.4.4. Proyecto 5G Media: Producción desde el Edge (Telefónica/RTVE/UPM)

El proyecto europeo 5G Media, financiado por la Comisión Europea con la participación de Telefónica, Radiotelevisión Española (RTVE) y la Universidad Politécnica de Madrid, desarrolló el servicio denominado *Producción desde el Edge* [46]. El 4 de diciembre de 2019 se realizó en la Cineteca del Matadero de Madrid el primer piloto operativo: tres cámaras conectadas mediante fibra óptica al *edge* de Telefónica transmitieron la ficción sonora *La Radio es Sueño* de Radio 3, mientras el equipo de realización operaba desde las instalaciones de RTVE en Torrespaña, suprimiendo el desplazamiento de la unidad móvil de producción.

La latencia de transporte medida en la red de acceso de Telefónica fue de 3–5 ms, cum-

pliendo el requisito de tiempo real para producción en directo. Este resultado demostró que la infraestructura de *edge computing* puede sustituir funcionalmente a la unidad móvil en eventos de cobertura local. La industria ha documentado reducciones de coste relevantes en implantaciones REMI de escala comparable [8].

2.4.5. Proyectos europeos de investigación de la UPM

La Universidad Politécnica de Madrid ha liderado una línea de investigación en producción audiovisual remota a través de sucesivos proyectos europeos que han evolucionado desde la validación de tecnologías de red hasta la ciberseguridad aplicada a entornos multimedia distribuidos:

- **5G Media (2017–2020):** proyecto de la Comisión Europea que exploró la viabilidad del *edge computing* para la producción audiovisual remota. El piloto con RTVE y Telefónica descrito en la sección anterior constituyó su demostración operativa más representativa.
- **NEMO (*Next Generation Meta Operating System*)** [47]: proyecto europeo orientado a la orquestación de servicios multimedia en infraestructuras de red distribuidas, en cuyo marco se desplegó Voctomix 1.0 en escenarios de producción distribuida. La experiencia en arquitecturas de microservicios acumulada en NEMO sirvió de antecedente técnico para el posterior desarrollo de Voctomix 2.0 en el proyecto CyberNEMO.
- **CyberNEMO** [7]: proyecto europeo centrado en la ciberseguridad aplicada a entornos de producción y distribución multimedia. Voctomix 2.0 fue desplegado en el clúster Kubernetes del proyecto, donde el *broker* RabbitMQ del sistema fue integrado con las colas *vproduction-uc-media* y *vqprobe-uc-media*, publicando en tiempo real eventos de mezclador durante sesiones de producción audiovisual técnica. Este despliegue constituyó la primera validación del sistema de telemetría de Voctomix 2.0 en un entorno real.

La tabla 2.7 resume los cinco casos de uso presentados, indicando la escala de despliegue, la tecnología clave empleada y la aportación de cada uno al contexto del presente trabajo.

Tabla 2.7: Casos de uso representativos de producción remota audiovisual.

Caso de uso	Escala	Tecnología clave	Aportación
CCC / C3VOC	Múltiples salas	Voctomix, MKV/TCP	Origen del proyecto
JJ.OO. Tokio 2020	Decenas de sedes	REMI cloud	Validación olímpica
LaLiga (Telefónica)	Red de estadios	SRT + AES-256	REMI sobre Internet
5G Media (UPM/RTVE)	Red de acceso	Edge computing, fibra	Latencia de red 3–5 ms
CyberNEMO (UPM)	Clúster Kubernetes	RabbitMQ, telemetría	Validación académica

3. Diseño e implementación del sistema de producción remota de vídeo

Este capítulo describe el diseño e implementación de Voctomix 2.0: la arquitectura del sistema, sus componentes principales y las decisiones técnicas adoptadas. Para cada módulo se exponen los retos encontrados y las soluciones implementadas.

El punto de partida fue el repositorio oficial de Voctomix [2], alojado en GitHub bajo la organización *voc*, a partir del cual se realizó el clon inicial sobre el que se aplicaron todas las extensiones descritas en este capítulo. El trabajo comenzó con una fase de exploración sobre Voctomix 1.0, que permitió familiarizarse con el *pipeline* GStreamer [11], la comunicación cliente-servidor y el funcionamiento de los *composites*. La evolución completa de la estructura del repositorio, desde el estado inicial hasta el estado final de Voctomix 2.0, se documenta en el Anexo C.

La tabla 3.1 recoge los puertos TCP/UDP del sistema y su función.

Tabla 3.1: Puertos del sistema Voctomix 2.0.

Puerto	Protocolo	Función
9998	UDP	Sincronización de reloj de red GStreamer
9999	TCP	Protocolo de control (GUI, scripts, telemetría)
10000–10003	TCP	Entradas de cámara 1–4
10004	TCP	Entrada fuente break (vídeo de pausa)
10005	TCP	Entrada fuente intro (vídeo introductorio)
11000	TCP	Salida mezcla en bruto (grabación, monitor sin JPEG)
12000	TCP	Salida previsualización de mezcla comprimida (JPEG)
14000–14005	TCP	Salidas de previsualización comprimidas (GUI)
15000	TCP	Salida de <i>streaming</i>
17000	TCP	Entrada <i>stream blanker</i> PAUSE
17001	TCP	Entrada <i>stream blanker</i> NOSTREAM
18000	TCP	Entrada audio de fondo
8080	HTTP	Recepción del estado de la interfaz gráfica (telemetría)

La figura 3.1 muestra una visión global del sistema: las fuentes de señal, el núcleo de procesamiento, las salidas y la capa de control y telemetría. Este diagrama sirve de hilo conductor para los apartados que siguen.

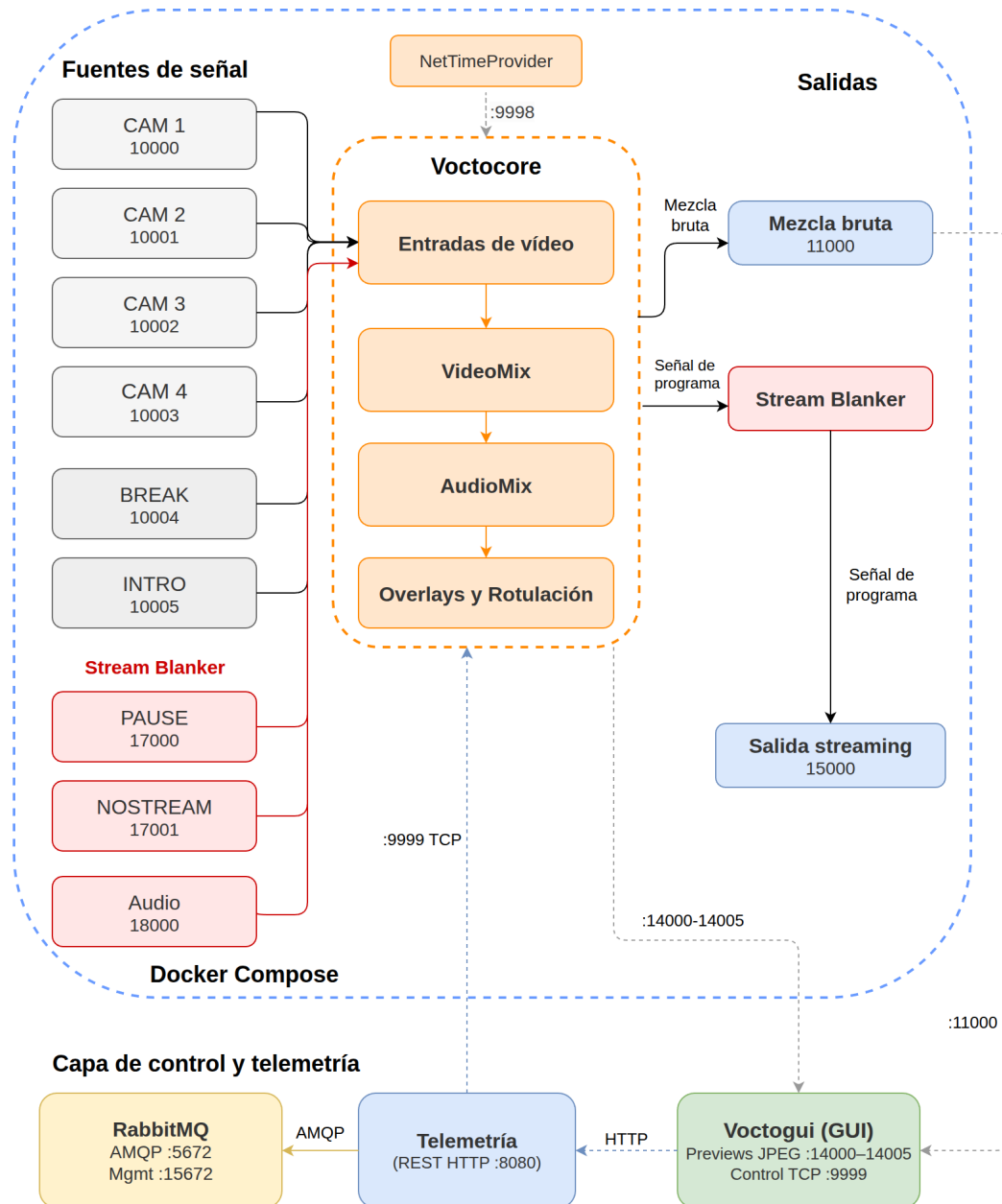


Figura 3.1: Arquitectura global del sistema Voctomix 2.0.

3.1. Arquitectura del sistema

3.1.1. voctocore: núcleo multimedia

voctocore es el servidor central del sistema. Recibe las fuentes de vídeo entrantes, realiza la mezcla y expone los resultados en distintos puertos TCP. Al arrancar construye el *pipeline* GStreamer a partir del fichero de configuración INI, crea una instancia de **AVSource** por cada fuente declarada, instancia el **VideoMix** y el **AudioMix**, conecta el **StreamBlanker**

y abre los puertos de salida. La configuración por defecto (`default-config.ini`) fija los parámetros globales del sistema: vídeo a 1920×1080 píxeles en formato I420 a 25 fotogramas por segundo, y audio PCM S16LE a 48 kHz estéreo. Todas las fuentes de señal deben ajustarse a estos parámetros para garantizar la compatibilidad con el *pipeline*. El contenido funcional de ambos ficheros de configuración se reproduce en la sección 6.2 del Anexo C.

El puerto 9999 expone el protocolo de control mediante comandos de texto plano como `set_video_a cam2` o `set_composite_mode pip`. Cada cambio de estado se difunde a todos los clientes conectados, de modo que la interfaz gráfica y el servicio de telemetría mantienen siempre una vista coherente del sistema.

El módulo AudioMix implementa la función *Audio Follows Video* (AFV): cuando el realizador conmuta a una fuente de vídeo diferente, el sistema ejecuta automáticamente un fundido cruzado en el audio hacia la pista de esa fuente. El audio y el vídeo en antena corresponden siempre a la misma fuente sin necesidad de intervención manual del operador.

3.1.2. voctogui: interfaz gráfica de control

voctogui es el cliente de control. Implementado en Python con GTK3 mediante PyGObject, proporciona al realizador la interfaz gráfica de usuario (*Graphical User Interface* (GUI)) para operar el sistema en tiempo real. Se conecta al núcleo por TCP y recibe las previsualizaciones de cada fuente en formato JPEG comprimido desde los puertos 14000–14005. El monitor principal recibe la mezcla por el puerto 12000 en JPEG (configuración predeterminada) o por el puerto 11000 en bruto cuando las previsualizaciones están desactivadas. El estado de la interfaz en el repositorio original, antes de iniciar el desarrollo, se recoge en la figura 6.1 del Anexo C. La figura 3.2 muestra la interfaz gráfica completa resultante tras el desarrollo del sistema.

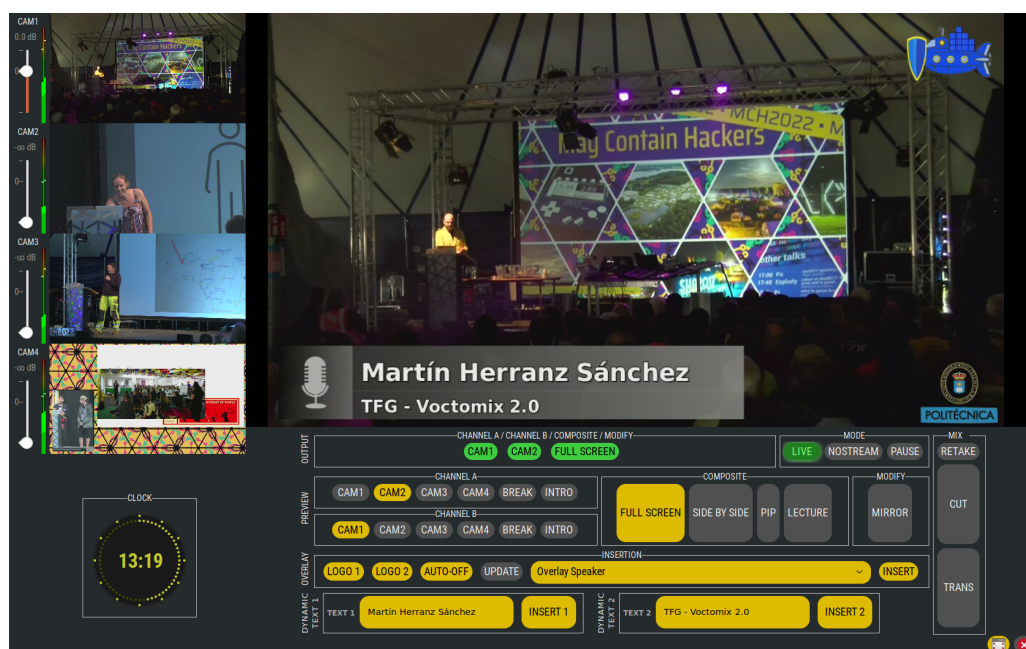


Figura 3.2: Interfaz gráfica completa de Voctomix 2.0 (voctogui).

3.2. Fuentes de señal de cámaras

El sistema acepta hasta cuatro fuentes de cámara simultáneas (cam1–cam4), que constituyen las entradas principales del mezclador. Cada cámara envía su señal al núcleo mediante una conexión TCP en formato Matroska con vídeo en bruto (I420) y audio PCM S16LE a 48 kHz estéreo.

3.2.1. Conexión de las fuentes

Cada fuente de cámara se conecta al puerto TCP correspondiente de **votocore**:

- **cam1** → puerto 10000
- **cam2** → puerto 10001
- **cam3** → puerto 10002
- **cam4** → puerto 10003

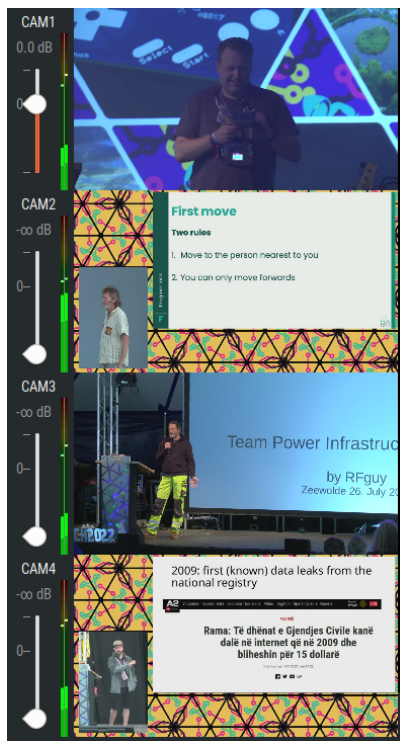
El envío de la señal se realiza mediante FFmpeg [48], herramienta de código abierto para el procesamiento multimedia, que encapsula la señal en un flujo Matroska y lo transmite por TCP hacia el puerto correspondiente de **votocore**. En el entorno de desarrollo y pruebas, FFmpeg inyecta vídeos pregrabados de un evento real, grabaciones de distintas cámaras colocadas en diferentes puntos de un auditorio durante unas conferencias técnicas, que se reproducen en bucle para emular la presencia de cámaras en directo. En un despliegue con cámaras físicas, el mismo proceso FFmpeg se ejecuta desde el equipo servidor conectado a la tarjeta de captura.

3.2.2. Previsualización en la interfaz gráfica

Una vez conectadas, las fuentes de cámara aparecen automáticamente en el panel de previsualizaciones de **votogui**. La interfaz muestra cuatro miniaturas JPEG de las cuatro cámaras, recibidas desde los puertos 14000–14003 a una resolución reducida (1024×576) para minimizar el consumo de ancho de banda entre el servidor y el cliente de control. Cada miniatura incluye:

- Un indicador visual del nombre de la fuente (cam1, cam2, cam3, cam4).
- Botones de selección para asignar la fuente al bus A o al bus B del mezclador.
- Un indicador del nivel de audio asociado a esa fuente.

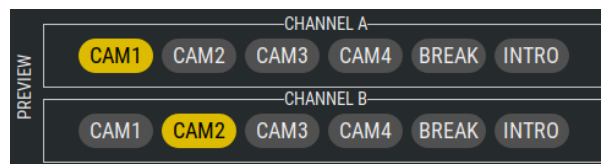
El realizador selecciona la fuente activa haciendo clic sobre la miniatura correspondiente, lo que envía el comando `set_video_a <fuente>` o `set_video_b <fuente>` al núcleo mediante el protocolo de control TCP. La figura 3.3 muestra el aspecto de este panel.



(a) Previsualizaciones y niveles de audio.



(b) Detalle de CAM1 con su medidor.



(c) Selectores de canal A y canal B.

Figura 3.3: Panel de cámaras en *voctogui*.

3.2.3. Requisitos de formato de la señal de entrada

Durante el desarrollo se comprobó que *votocore* es estricto con los metadatos del flujo de vídeo entrante. Al intentar inyectar un archivo *.mp4* convencional como fuente de prueba, el núcleo rechazaba la conexión sin emitir un mensaje de error claro. Tras el diagnóstico se identificaron un requisito obligatorio y una condición de calidad que toda fuente debe satisfacer:

1. **Pista de audio obligatoria:** *votocore* exige que cada flujo Matroska contenga tanto vídeo como audio. Los vídeos sin audio deben complementarse con una pista de silencio generada en tiempo real por FFmpeg mediante el filtro `-f lavfi -i aevalsrc=0`. Sin este ajuste, el motor GStreamer cierra la conexión en el *handshake* inicial.
2. **Colorimetría televisiva (BT.709):** [49] se recomienda que los metadatos del flujo declaren el espacio de color estándar de alta definición (`colorimetry=bt709`) y el rango de luz televisivo (`tv range`). Los archivos grabados con cámaras convencionales o editados con software de consumo suelen emplear el rango `pc` (full range). Sin la conversión adecuada, los colores presentan una desviación perceptible en la mezcla, aunque la conexión se establece correctamente. Para convertirlos, se aplican los filtros FFmpeg `scale=out_range=tv` y `colorspace=bt709`, que recalculan los valores de píxel en tiempo real sin degradación visual perceptible.

En definitiva, todos los parámetros del flujo entrante, resolución, frecuencia de imagen, espacio de color y formato de audio, deben coincidir con los valores declarados en el

fichero de configuración INI de **votocore** (**default-config.ini**), que fija los parámetros globales del sistema: 1920×1080, I420, 25 fps y 48 kHz S16LE estéreo. Es el propio *pipeline* GStreamer el que impone esta restricción: cualquier discrepancia entre la fuente y la configuración provoca un rechazo de conexión o artefactos en la mezcla, como se comprobó durante el desarrollo.

3.3. Fuentes de señal auxiliares

Además de las cuatro fuentes de cámara, el sistema incorpora varias fuentes de señal auxiliares que permiten gestionar el estado de la emisión en momentos en los que la mezcla principal no debe ser visible para el espectador, así como una entrada de audio independiente para música de fondo.

3.3.1. Fuente **break** e **intro**

break e **intro** son fuentes de vídeo pregrabadas que **votocore** trata como dos entradas adicionales en los puertos 10004 y 10005. Al no aportar información relevante durante la producción, se ocultan en la interfaz gráfica sin perder funcionalidad en el núcleo.

- **break**: vídeo en bucle continuo (**-stream_loop -1**) empleado como pantalla animada durante los descansos. El realizador puede seleccionarlo directamente en el mezclador, al igual que cualquier cámara.
- **intro**: vídeo de reproducción única. Al asignar **intro** a cualquier canal del mezclador (A o B), **votogui** detiene el proceso FFmpeg que alimenta esa entrada y lanza uno nuevo que reproduce el vídeo desde el principio sin bucle. Una vez que la nueva señal está estabilizada, la interfaz envía el comando de corte al núcleo para que la **intro** comience a emitirse desde el primer fotograma.

3.3.2. Entrada de audio

El sistema incorpora una entrada de audio independiente en el puerto 18000, pensada para inyectar música de fondo durante los estados de pausa o fuera de emisión. Esta entrada acepta un flujo Matroska de solo audio (PCM S16LE, 48 kHz, estéreo) y su nivel de salida puede controlarse desde el protocolo de control.

3.3.3. *stream blanker*: estados de emisión

El *stream blanker* es el mecanismo que permite interrumpir la señal de programa hacia el exterior sin interrumpir el flujo de trabajo interno del realizador. Se implementa en **streamblanker.py** como un elemento GStreamer adicional interpuesto entre la mezcla y el puerto de salida de *streaming* (puerto 15000).

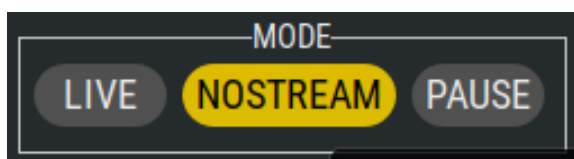
El sistema define tres estados de emisión, controlables desde la barra de herramientas de **votogui**:

- **LIVE**: se emite la mezcla activa producida por **votocore**. El realizador controla en todo momento lo que ve el espectador.

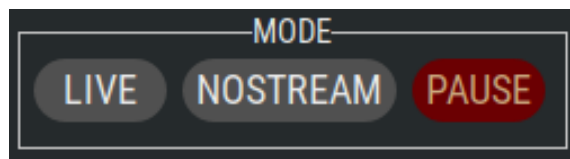
- **PAUSE:** se emite el vídeo de pausa procedente del puerto 17000, acompañado de la música de fondo del puerto 18000. Se usa durante los descansos del evento para que los espectadores vean una pantalla animada mientras el equipo de producción prepara la siguiente sesión.
- **NOSTREAM:** se emite un vídeo de “fuera de emisión” procedente del puerto 17001, con el audio de la mezcla en vivo bloqueado. Indica al espectador que la transmisión ha finalizado o aún no ha comenzado.

El cambio entre estados se aplica sobre la señal de salida. Desde la interfaz gráfica, el realizador dispone de tres botones (LIVE, PAUSE, NOSTREAM) en la barra de herramientas.

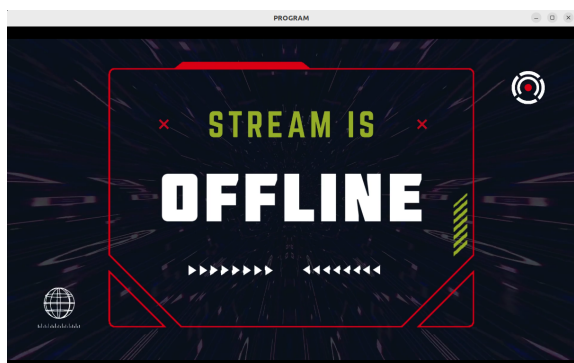
La figura 3.4 muestra la interfaz del realizador y la señal vista por el espectador en los estados NOSTREAM y PAUSE.



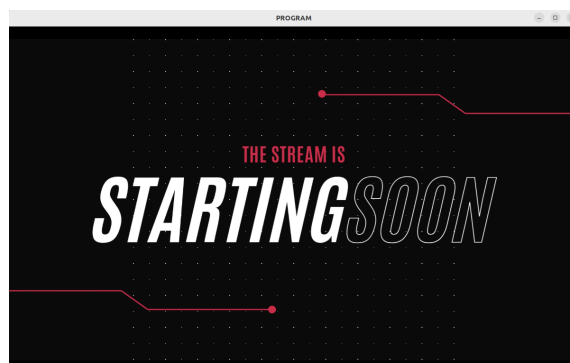
(a) GUI en estado NOSTREAM.



(c) GUI en estado PAUSE.



(b) Señal del espectador en NOSTREAM.



(d) Señal del espectador en PAUSE.

Figura 3.4: Estados NOSTREAM y PAUSE del *stream blanker*: vista del realizador (arriba) y señal recibida por el espectador (abajo).

3.4. Composites

Los *composites* definen cómo se combinan visualmente las dos fuentes de vídeo activas (bus A y bus B) en el fotograma de salida. Voctomix implementa los *composites* mediante el elemento GStreamer *compositor*, que sitúa y escala cada fuente de acuerdo con los parámetros geométricos definidos en la sección [composites] del fichero de configuración (reproducida en su totalidad en la sección 6.2 del Anexo C). El compositor de GStreamer soporta la modificación de estos parámetros en caliente sin interrumpir el *pipeline*, base técnica que el motor de transiciones (sección 3.5) aprovecha para interpolar de forma animada entre *composites* consecutivos.

La figura 3.5 ilustra la disposición visual de las dos fuentes en cada uno de los *composites* principales.

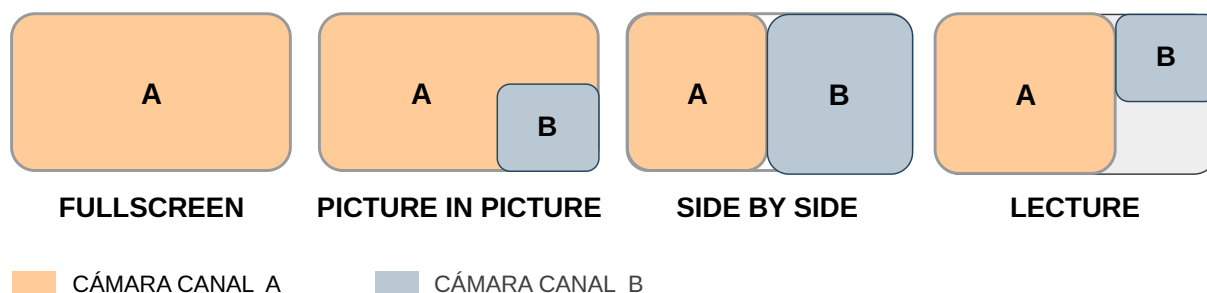
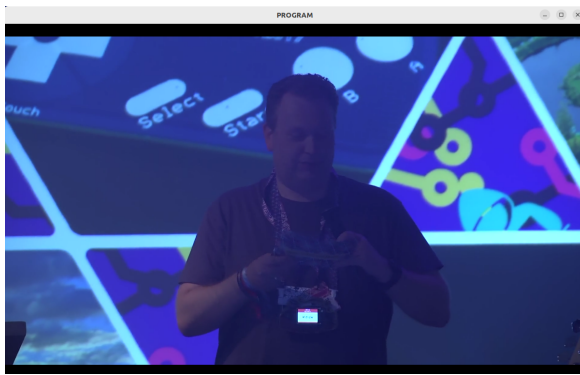


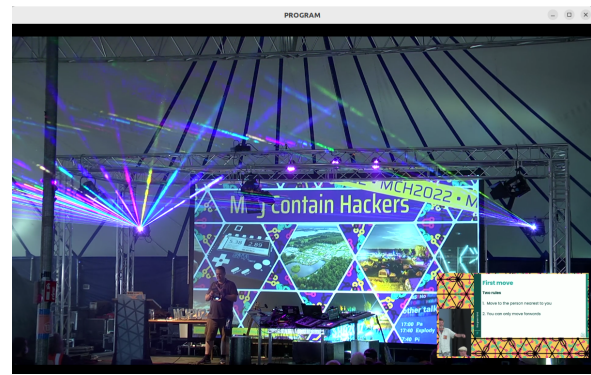
Figura 3.5: Disposición geométrica de las fuentes A y B en los cuatro *composites* de Voctomix 2.0.

El sistema ofrece cuatro modos de *composite*, cuyo resultado sobre la señal de programa se muestra en la figura 3.6:

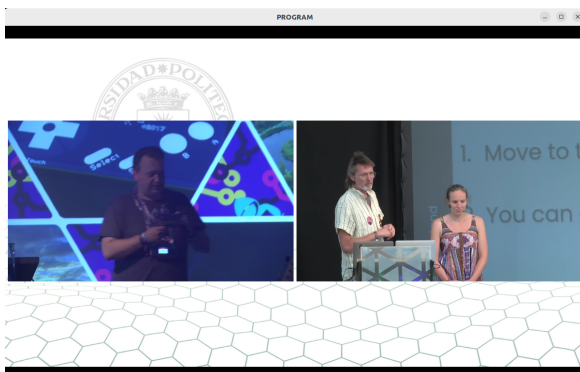
- **Full Screen:** la fuente A ocupa toda la pantalla y la fuente B queda oculta. Es el modo por defecto de una realización convencional, adecuado para la emisión de una ponencia individual, una demostración en pantalla o cualquier situación en la que una única fuente deba ocupar todo el encuadre.
- **Picture in Picture:** la fuente A se muestra a pantalla completa mientras la fuente B aparece reducida en una esquina del encuadre a una escala aproximada del 26 % del ancho del fotograma. Es el *composite* clásico de ponente con presentación: la cámara ocupa la pantalla completa y las diapositivas aparecen en la esquina, o viceversa.
- **Side by Side:** ambas fuentes se muestran a igual tamaño, situadas una junto a la otra y ocupando cada una aproximadamente la mitad del fotograma. Resulta útil para comparaciones visuales, debates con dos participantes o para mostrar simultáneamente la cámara del ponente y sus diapositivas con el mismo peso visual.
- **Lecture:** la fuente A ocupa el 75 % del encuadre y la fuente B aparece reducida al 23 % en la parte derecha. Está diseñado para retransmisiones de ponencias o clases: las diapositivas centran la atención del espectador mientras la cámara del ponente permanece visible en un recuadro lateral, sin comprometer la legibilidad del contenido proyectado.



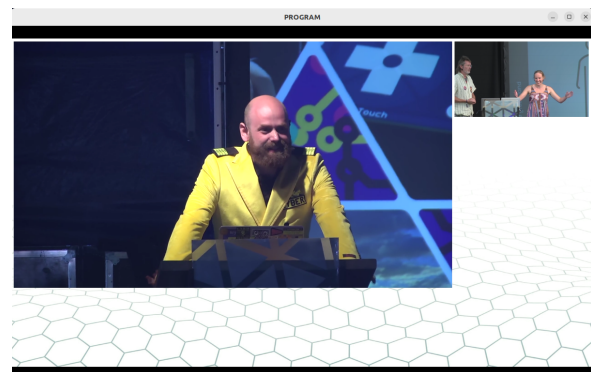
(a) Full Screen.



(b) Picture in Picture.



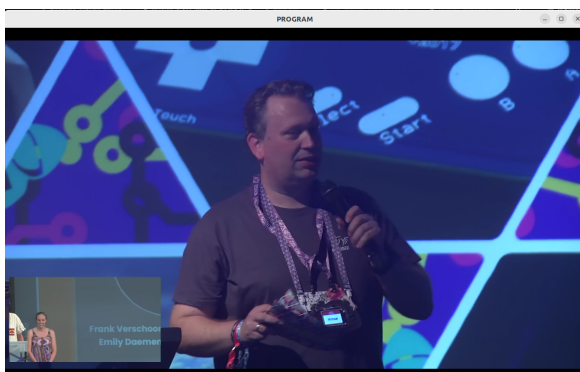
(c) Side by Side.



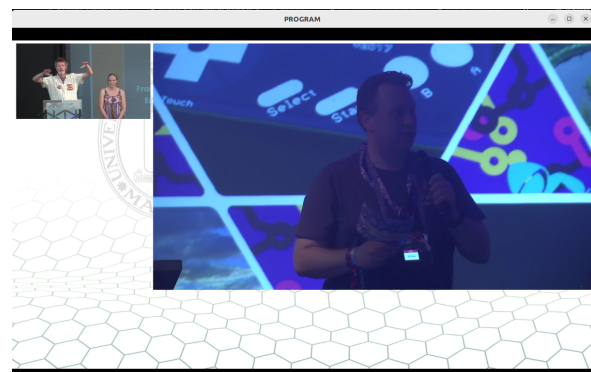
(d) Lecture.

Figura 3.6: Señal de programa en los cuatro modos de *composite* de Voctomix 2.0.

El sistema incorpora además un modo *mirror*, disponible como variante para Picture in Picture y Lecture. Al activarlo, el núcleo invierte la disposición horizontal del *composite*: los elementos situados a la derecha se desplazan a la izquierda y viceversa. Por ejemplo, el recuadro del PIP pasa de la esquina inferior derecha a la inferior izquierda, y en Lecture el bloque de diapositivas cambia de lado. Esta variante se genera automáticamente a partir de los parámetros geométricos del *composite* original, sin coste adicional en el procesamiento de vídeo. La figura 3.7 muestra el resultado visual en ambos modos.



(a) Mirror activo en Picture in Picture.



(b) Mirror activo en Lecture.

Figura 3.7: Efecto espejo aplicado a los *composites* Picture in Picture y Lecture.

3.5. Transiciones

Las transiciones controlan cómo se realiza el cambio de una fuente o *composite* a otro en el flujo de programa. Voctomix 2.0 ofrece tres modos de transición que el realizador activa desde la barra de herramientas de la interfaz gráfica.

3.5.1. Cut (corte instantáneo)

El botón **Cut** realiza una transición instantánea: en el siguiente fotograma tras la pulsación, la fuente de programa pasa a ser la fuente de *preview*. No existe ningún efecto intermedio. Es la transición estándar en producción *broadcast*: el corte imita el montaje cinematográfico y resulta imperceptible cuando el *timing* es correcto.

3.5.2. Trans (transición suave)

El botón **Trans** realiza una transición animada entre el estado actual y el nuevo estado de *composite* o fuente, interpolando los parámetros geométricos y de transparencia durante un tiempo configurable (por defecto 750 ms, definido en la sección [transitions] del fichero de configuración). La transición se implementa modificando progresivamente los valores de los *pads* del compositor de GStreamer.

Voctomix 2.0 define múltiples trayectorias de transición según el estado de origen y destino (por ejemplo, de **fs** a **pip** la transición pasa por el *composite* intermedio **fs-pip** antes de llegar a **pip**), garantizando que la animación sea visualmente coherente independientemente del cambio solicitado.

3.5.3. Retake

El botón **Retake** permite al realizador deshacer el último cambio de fuente o *composite* y volver al estado inmediatamente anterior sin interrumpir la señal de programa. Su funcionamiento es equivalente a un **Cut** inverso: en el siguiente fotograma, el sistema intercambia programa y *preview*, restaurando la fuente que estaba en antena antes del cambio.

En la práctica de la producción en directo, el flujo de trabajo más habitual consiste en alternar entre dos fuentes o dos *composites*, como ocurre en una retransmisión de ponencia donde el realizador conmuta continuamente entre la cámara del ponente y sus diapositivas. En este contexto, el estado anterior es siempre la opción más probable para el siguiente corte, lo que convierte al **Retake** en un atajo eficaz para ese patrón de trabajo. Además, la función resulta útil cuando el realizador comete un corte erróneo y necesita corregirlo de inmediato sin generar un segundo corte visible en la emisión.

3.6. Overlays, logos y texto dinámico

Voctomix 2.0 incorpora un sistema completo de superposición de elementos gráficos sobre la señal de vídeo: logos corporativos, *overlays Portable Network Graphics* (PNG) con canal alfa y texto dinámico renderizado en tiempo real. Todos estos elementos se gestionan desde la interfaz gráfica sin interrumpir la retransmisión.

3.6.1. Logos

El sistema soporta dos logos simultáneos superpuestos en posiciones fijas del encuadre:

- **logo1**: posicionado en la esquina inferior derecha del fotograma.
- **logo2**: posicionado en la esquina superior izquierda del fotograma.

La posición de cada logo no la determina el *pipeline* de GStreamer, sino el diseño del fichero PNG: cada imagen cubre el fotograma completo con una zona transparente excepto en la esquina donde aparece el logotipo. Los ficheros PNG de los logos se definen en las secciones [logo1] y [logo2] del fichero de configuración, lo que permite cambiar la identidad visual del evento sin modificar el código. La visibilidad de cada logo se puede activar o desactivar individualmente desde la interfaz gráfica.

3.6.2. *Overlays* PNG y texto dinámico

Los *overlays* son imágenes PNG que se superponen sobre la señal de vídeo mediante el elemento GStreamer `gdkpixbufoverlay`. El módulo `overlay.py` gestiona el ciclo de vida completo de un *overlay*: carga del fichero PNG, fundido de entrada (*blend-in*), mantenimiento en pantalla y fundido de salida (*blend-out*). El tiempo de fundido se controla mediante el parámetro `blend-time` en la sección [overlay] del fichero de configuración (en milisegundos).

El elemento `gdkpixbufoverlay` está diseñado para leer archivos de imagen estáticos desde una ruta del sistema de ficheros. Para cambiar el *overlay* en caliente, la clase `Overlay` actualiza la propiedad `location` del elemento mediante el comando `set_overlay` del protocolo de control, lo que fuerza la recarga del nuevo fichero PNG sin interrumpir el *pipeline*.

3.6.3. Rotulación dinámica

El sistema de rotulación permite al operador escribir el nombre del ponente o el título de la sesión en el panel de la interfaz gráfica. Al confirmar el texto, la interfaz lo envía al núcleo a través del protocolo de control, donde `votocore` actualiza directamente el elemento `textoverlay` del *pipeline* GStreamer. El rótulo aparece en antena en el siguiente fotograma, sin escritura a disco ni procesamiento externo.

El sistema dispone de dos canales de rotulación independientes, lo que permite mostrar simultáneamente dos textos en posiciones distintas del encuadre sin solapamientos. Cada canal tiene su propio campo de texto en la interfaz, con gestión de foco de teclado independiente para evitar colisiones con los atajos del mezclador. Su implementación requirió extender el *pipeline* con un segundo elemento `textoverlay`, ampliar el protocolo de control con un segundo comando de texto y añadir el campo correspondiente en la interfaz gráfica.

3.6.4. Botón UPDATE: recarga dinámica de recursos

El botón UPDATE del panel de *overlays* permite recargar el directorio de recursos gráficos durante la emisión. Esta funcionalidad es crítica en entornos de directo: si el equipo de grafismo corrige un error ortográfico en una plantilla PNG o añade un nuevo rótulo de

último minuto, el operador puede incorporarlo al sistema sin interrumpir la señal de vídeo ni reiniciar ningún proceso.

3.6.5. Auto-off de *overlays*

La funcionalidad *auto-off* resuelve un problema frecuente en producción en directo: el operador realiza un corte de cámara olvidando retirar el *overlay* activo, que queda superpuesto sobre la nueva fuente de forma inapropiada (por ejemplo, el nombre de un ponente apareciendo sobre la imagen de otro).

Con esta opción activa, cada vez que el realizador efectúa un corte de cámara el sistema retira automáticamente el *overlay* y el texto dinámico con un fundido de salida, sin que el operador tenga que hacerlo manualmente. La funcionalidad puede activarse de forma permanente en la configuración o exponerse como un botón de alternancia en la interfaz para que el operador decida en tiempo real si la quiere habilitada. La figura 3.8 ilustra el resultado visual y el panel de control.

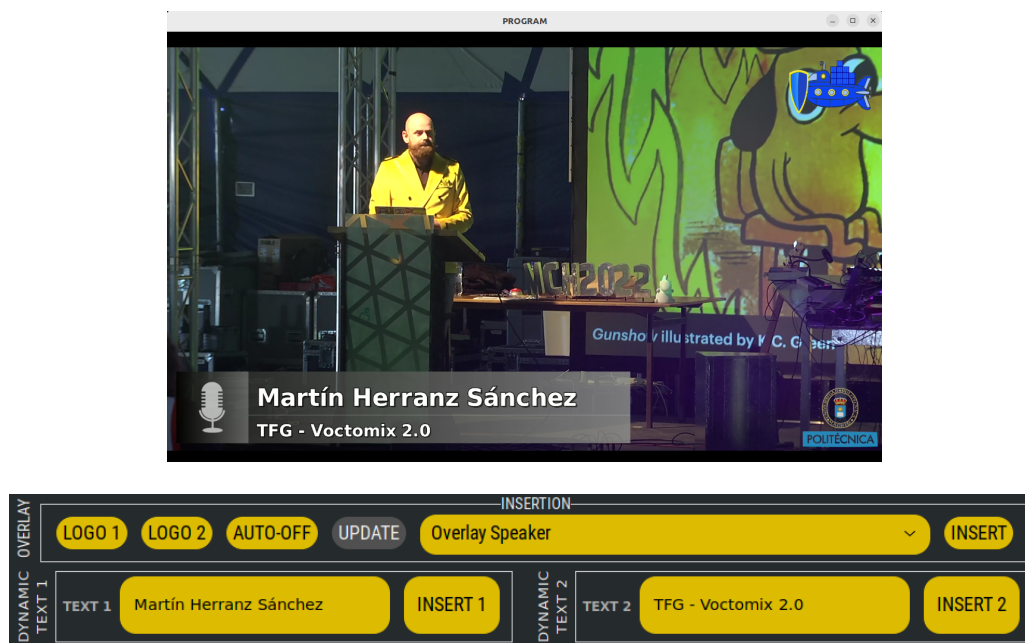


Figura 3.8: Señal de programa con *overlay* y rotulación activos (arriba) y panel de control correspondiente en la interfaz (abajo).

3.7. Señal de programa final

Una vez procesada la mezcla, con el *composite* activo, las transiciones aplicadas y los *overlays* superpuestos, *voctocore* emite la señal de programa final de forma simultánea por dos puertos:

- **Puerto 11000:** salida de la mezcla en bruto (vídeo sin comprimir en formato I420), utilizada principalmente por encoders externos como FFmpeg para grabación local. La interfaz gráfica solo se conecta aquí cuando las previsualizaciones están desactivadas; en la configuración predeterminada utiliza el puerto 12000 (JPEG comprimido).

Esta salida refleja siempre el estado real de la mezcla, independientemente del estado del *stream blanker*.

- **Puerto 15000:** salida de *streaming*, que pasa previamente por el *stream blanker*. Cuando el estado es LIVE, esta salida es idéntica al puerto 11000; cuando el estado es PAUSE o NOSTREAM, emite el vídeo alternativo correspondiente en lugar de la mezcla real.

El encoder de salida (típicamente FFmpeg) se conecta al puerto 15000 y produce el flujo H.264/AAC que se transmite a la plataforma de distribución mediante los protocolos RTMP o SRT, ampliamente soportados por las plataformas de *streaming* en directo. La separación entre los puertos 11000 y 15000 garantiza que la grabación local refleje siempre la mezcla real del realizador, mientras que la audiencia remota solo recibe la señal que el operador autoriza explícitamente a través del *stream blanker*.

3.8. Personalización de la interfaz gráfica

La interfaz gráfica *votogui* incorpora las siguientes adaptaciones visuales para su uso en producción profesional:

- **Tema oscuro.** La interfaz dispone de una hoja de estilos *Cascading Style Sheets* (CSS) que aplica un fondo #252a2c y texto claro en todos los paneles. Los botones activos se iluminan en amarillo (#ddbb00), proporcionando al realizador una retroalimentación visual inmediata sobre el estado de cada control.
- **Reloj de estudio.** El panel de cámaras incluye un *widget* de reloj en su parte inferior que muestra la hora del sistema en tiempo real, lo que permite al realizador sincronizar los cortes de cámara con los tiempos de la escaleta sin apartar la vista del monitor de mezcla.

3.9. Sistema de telemetría

El sistema de telemetría registra en tiempo real todo lo que ocurre durante una sesión de producción: cambios de fuente activa, cambios de *composite*, activaciones del *stream blanker*, estado de *overlays* y niveles de audio. Esta información permite tanto monitorizar la sesión mientras está en curso como auditarla en detalle una vez terminada la emisión.

3.9.1. Arquitectura del servicio

El servicio (`telemetry_service.py`) es un proceso independiente que se conecta a *votocore* en el puerto 9999 como un cliente TCP ordinario y consulta su estado cada segundo. Sin embargo, *votocore* no tiene acceso a la información gestionada por la interfaz gráfica: qué fuentes están en *preview*, qué logos están activos o el contenido de los rótulos.

Para capturar también esta información, el módulo `gui_state_exporter.py` de *votogui* exporta el estado completo de la interfaz cada segundo. En el escenario de un PC lo escribe en un fichero local; en escenarios en red (Docker o dos PCs) lo envía al servicio mediante HTTP POST al puerto 8080.

El servicio fusiona las dos fuentes y construye una vista unificada de la sesión, que guarda en formato JSON (`sessionN.jsonl`): un objeto JSON completo por línea, con numeración automática por sesión. Este formato permite analizar el log de forma incremental con herramientas estándar como Python, sin necesidad de cargar el fichero completo en memoria (figura 3.9).

```
[05:58:07] STATE CPU: 62.9% RAM: 16.3% uptime: 11:53:20 session: 00:00:18 [cam1/ cam2/ cam3/ cam4/]
full → {"preview": {"channel a": "cam1", "channel b": "cam2"}, "output": {"channel a": "cam1", "channel b": "cam2"}, "mode": "pause",
"composite": {"name": "full screen", "fullscreen": true, "side by side": false, "pip": false, "lecture": false}, "mirror": false, "insertion":
{"overlay": false, "auto off": true, "overlay selected": "Overlay Speaker", "logo1": false, "logo2": false, "text1 active": false, "text1 value":
"", "text2 active": false, "text2 value": ""}, "audio": {"cam1 db": -20.0, "cam2 db": -20.0, "cam3 db": -20.0, "cam4 db": -20.0, "break db": n
ull, "intro db": null}, "sources": {"cam1": true, "cam2": true, "cam3": true, "cam4": true, "break": true, "intro": true}, "stream info": {"vide
o_resolution": "1920x1080", "video_fps": "25", "video_codec": "rawvideo", "video_pix_fmt": "I420", "calc bandwidth mbps_per_cam": 622.08, "audio
_sample rate": "48000", "audio channels": "2", "audio_fmt": "S16LE"}, "system health": {"cpu usage percent": 62.9, "ram usage percent": 16.3, "r
am available_mb": 107494, "uptime": "11:53:20", "session_duration": "00:00:18"}, "network": {"core_ip": "172.19.0.4", "gui_local_ip": "192.168.0
.42"}}

[05:58:08] CHANGE CPU: 62.9% RAM: 16.3% uptime: 11:53:20 session: 00:00:18 [cam1/ cam2/ cam3/ cam4/]
full → {"preview": {"channel a": "cam1", "channel b": "cam2"}, "output": {"channel a": "cam1", "channel b": "cam2"}, "mode": "pause",
"composite": {"name": "full screen", "fullscreen": true, "side by side": false, "pip": false, "lecture": false}, "mirror": false, "insertion":
{"overlay": false, "auto off": true, "overlay selected": "Overlay Speaker", "logo1": false, "logo2": false, "text1 active": false, "text1 value":
"", "text2 active": false, "text2 value": ""}, "audio": {"cam1 db": -20.0, "cam2 db": -20.0, "cam3 db": -20.0, "cam4 db": -20.0, "break db": n
ull, "intro db": null}, "sources": {"cam1": true, "cam2": true, "cam3": true, "cam4": true, "break": true, "intro": true}, "stream info": {"vide
o_resolution": "1920x1080", "video_fps": "25", "video_codec": "rawvideo", "video_pix_fmt": "I420", "calc bandwidth mbps_per_cam": 622.08, "audio
_sample rate": "48000", "audio channels": "2", "audio_fmt": "S16LE"}, "system health": {"cpu usage percent": 62.9, "ram usage percent": 16.3, "r
am available_mb": 107494, "uptime": "11:53:20", "session_duration": "00:00:18"}, "network": {"core_ip": "172.19.0.4", "gui_local_ip": "192.168.0
.42"}}}
```

Figura 3.9: Salida del servicio de telemetría mostrando un evento STATE y un evento CHANGE consecutivos.

3.9.2. Registro de eventos

El registro combina dos tipos de entrada:

- **Evento Change (CHANGE):** se escribe inmediatamente cuando se detecta cualquier variación de estado, como un corte de cámara o un cambio de *composite*. Captura el instante exacto en que ocurre el cambio.
- **Evento State (STATE):** un hilo independiente escribe el estado completo del sistema cada 5 segundos, con independencia de que haya habido cambios. Sirve como mecanismo de redundancia para reconstruir la sesión aunque se pierda algún evento.

Cada entrada incluye una marca de tiempo, el estado completo del mezclador (fuentes en programa y *preview*, *composite*, modo de *stream*, *overlays* y niveles de audio) y métricas del servidor (*Central Processing Unit* (CPU), *Random Access Memory* (RAM) y ancho de banda por fuente). El ejemplo siguiente muestra una entrada de tipo CHANGE completa tras un corte de cámara:

```
{"timestamp": "2025-03-28 20:14:32",
"type": "change",
"output": {"channel_a": "cam2", "channel_b": "cam1"},
"preview": {"channel_a": "cam1", "channel_b": "cam2"},
"mode": "live",
"composite": {"name": "full_screen", "fullscreen": true,
"side_by_side": false, "pip": false, "lecture": false},
"mirror": false,
"insertion": {"overlay": false, "auto_off": true,
"overlay_selected": "Overlay Speaker",
"logo1": false, "logo2": false,
"text1_active": false, "text1_value": ""},
```

```
"text2_active": false, "text2_value": "{}",
"audio": {"cam1_db": -12.0, "cam2_db": 0.0, "cam3_db": -20.0,
          "cam4_db": -20.0, "break_db": null, "intro_db": null},
"sources": {"cam1": true, "cam2": true, "cam3": true,
            "cam4": true, "break": true, "intro": true},
"stream_info": {"video_resolution": "1920x1080", "video_fps": "25",
                "video_codec": "rawvideo", "video_pix_fmt": "I420",
                "calc_bandwidth_mbps_per_cam": 622.08,
                "audio_sample_rate": "48000", "audio_channels": "2",
                "audio_fmt": "S16LE"},
"system_health": {"cpu_usage_percent": 34.2, "ram_usage_percent": 8.7,
                  "ram_available_mb": 107494, "uptime": "11:53:20",
                  "session_duration": "00:05:47"},
"network": {"core_ip": "172.19.0.4", "gui_local_ip": "192.168.0.42"}}
```

3.9.3. Publicación en RabbitMQ

RabbitMQ [50] es un servidor de mensajería que actúa como intermediario: los eventos se publican en él y cualquier aplicación externa interesada puede suscribirse para recibirlos, sin que el servicio de telemetría necesite saber quién los consume. Voctomix 2.0 lo utiliza para hacer accesibles los eventos de producción a sistemas externos mediante el protocolo *Advanced Message Queuing Protocol* (AMQP) [51], el estándar abierto de mensajería entre aplicaciones.

La función `publish_to_rabbit()` es la responsable del envío. Recibe los datos de cada evento (los mismos que ya se guardan en el log local), los serializa a texto JSON y los publica en el *exchange* (punto de distribución de mensajes) `voctomix_events`. Los mensajes no se persisten en disco, priorizando la velocidad sobre la entrega garantizada, lo que es apropiado en producción en directo.

La conexión con el servidor se configura mediante cuatro variables de entorno. `RABBITMQ_HOST` contiene la dirección del servidor RabbitMQ: puede ser una dirección IP, un nombre de dominio o, en entornos Docker y Kubernetes, el nombre del servicio, que el sistema DNS interno del clúster traduce automáticamente a la IP del contenedor correspondiente. `RABBITMQ_PORT` fija el puerto TCP de escucha, cuyo valor estándar AMQP es 5672. Las variables `RABBITMQ_USER` y `RABBITMQ_PASS` aportan las credenciales de autenticación: RabbitMQ requiere usuario y contraseña en todas sus conexiones y desactiva la cuenta de invitado por defecto en entornos en red por razones de seguridad. La integración solo se activa cuando `RABBITMQ_HOST` está definida; en su ausencia, el servicio opera únicamente con el log local sin penalización de rendimiento.

El servicio mantiene una conexión TCP persistente al servidor y reconecta automáticamente ante cualquier caída de red, sin bloquear el procesamiento de eventos. El *exchange* es de tipo *fanout* (difusión a todos los suscriptores): cualquier consumidor externo que se suscriba recibe todos los eventos sin necesidad de modificar el código del servicio de telemetría ni el de `voctocore`. En el marco del proyecto europeo CyberNEMO (UPM), el servidor RabbitMQ de Voctomix 2.0 fue integrado en el clúster Kubernetes, donde la cola `vproduction-uc-media` recibió los eventos del mezclador en tiempo real, como muestra la figura 3.10.

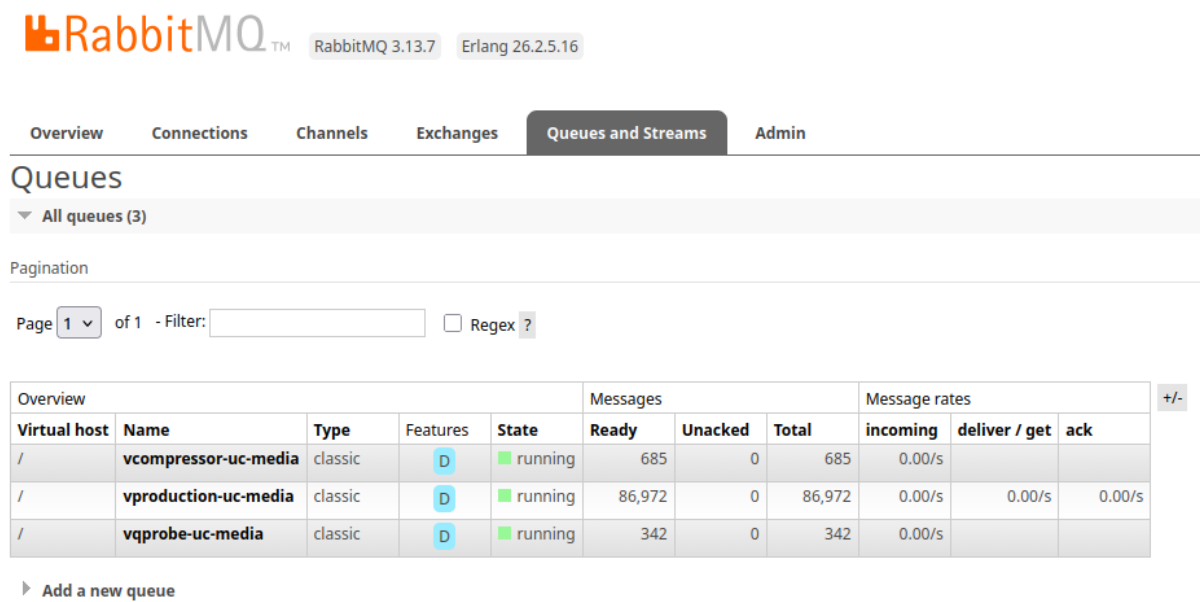


Figura 3.10: Colas RabbitMQ activas durante la integración con CyberNEMO.

4. Despliegue del sistema

El sistema puede desplegarse en cuatro escenarios distintos según el contexto de uso: desde un único ordenador para desarrollo y pruebas hasta un clúster Kubernetes para producción en la nube. La tabla 4.1 resume cada opción.

Tabla 4.1: Escenarios de despliegue de Voctomix 2.0.

Escenario	Requisito	Caso de uso
Un PC (nativo)	Linux, GStreamer y Python	Pruebas y desarrollo
Dos PCs (nativo)	Dos equipos Linux en red local	Pruebas y desarrollo
Docker Compose	Docker instalado	Entorno portable sin instalación
Kubernetes	Minikube o clúster externo	Infraestructura orquestada

Los dos primeros se denominan escenarios nativos porque ejecutan los procesos directamente sobre el sistema operativo del equipo, sin virtualización. Los dos siguientes emplean contenedores, lo que elimina la necesidad de instalar GStreamer, Python o sus dependencias en la máquina anfitriona y garantiza un comportamiento idéntico en cualquier equipo. La contenerización fue un objetivo prioritario del proyecto por dos razones complementarias. Docker simplifica la adopción del sistema al reducir el despliegue a un único comando, sin configuración manual del entorno. Kubernetes, por su parte, responde a una necesidad concreta del contexto en el que se desarrolla este trabajo: los proyectos de investigación CyberNEMO de la UPM utilizan Kubernetes como infraestructura de producción, por lo que Voctomix 2.0 debe poder integrarse en ese entorno.

4.1. Despliegue nativo: un PC y dos PCs

El despliegue nativo ejecuta los procesos de `votocore` y `votogui` directamente sobre Linux, sin ninguna capa intermedia. Es el escenario de referencia para el desarrollo y las pruebas funcionales del sistema.

Escenario de un PC. Este fue el escenario utilizado durante el desarrollo del proyecto y el que permitió comprender en profundidad la arquitectura del sistema. El arranque sigue tres pasos: primero se lanza `votocore.py`, que abre el puerto de control (9999) y los puertos de entrada de señal (10000–10005, uno por fuente), y comienza a escuchar conexiones; a continuación se inyectan las señales de vídeo y audio en esos puertos mediante FFmpeg; por último se arranca `votogui.py`, que se conecta a `votocore` a través

de `localhost` (configurado como `host=localhost` en el fichero de configuración de `votogui`), y muestra en la interfaz todas las fuentes activas. Al cerrar la interfaz, los procesos auxiliares se terminan automáticamente (figura 4.1).

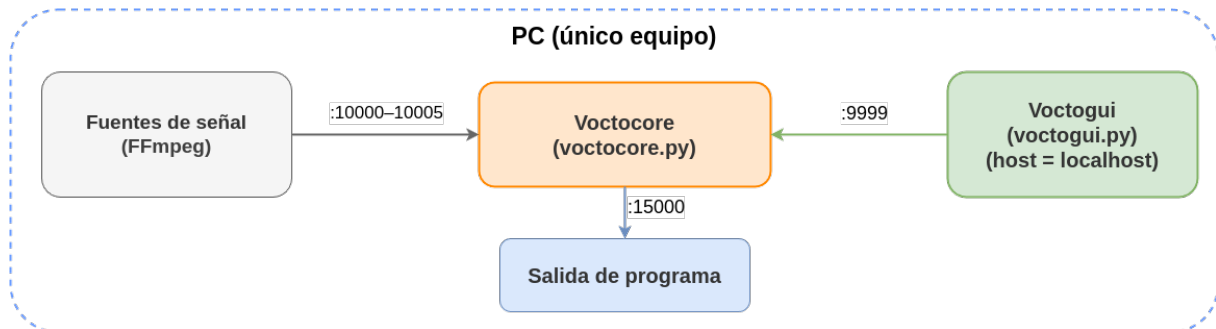


Figura 4.1: Escenario de despliegue nativo con un único PC.

Escenario de dos PCs. En este escenario la carga de trabajo se reparte entre dos equipos conectados por red local. El proceso de arranque es análogo al de un PC pero distribuido: en el PC servidor se lanza `votocore.py` y se inyectan todas las señales de vídeo y audio mediante FFmpeg, exactamente igual que en el escenario anterior. En el PC del realizador se lanza únicamente `votogui.py`, que recibe como parámetro la dirección IP del servidor (`IP_SERVER`) para conectarse al puerto de control 9999 a través de la red local, en lugar de `localhost`. Además, el PC del realizador abre un monitor de programa conectándose al puerto de salida 15000 del servidor, lo que permite supervisar la señal de programa en tiempo real desde el puesto del operador (figura 4.2).

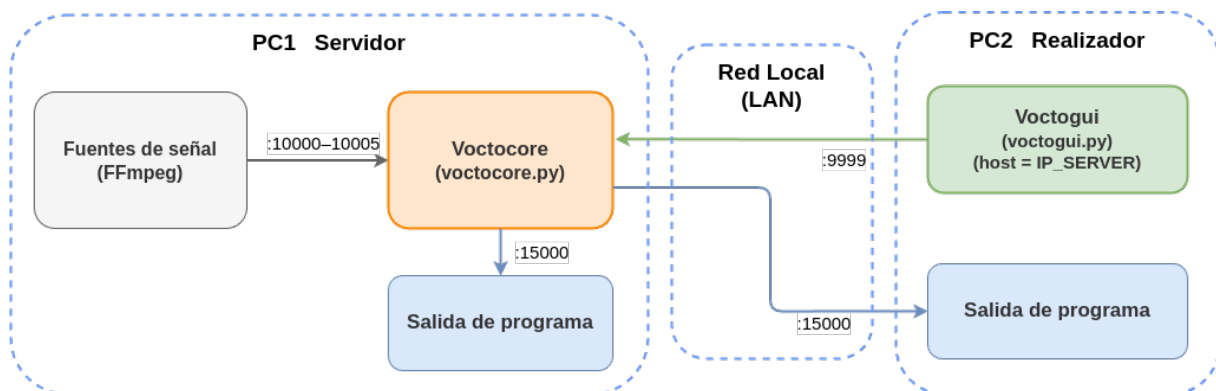


Figura 4.2: Escenario de despliegue nativo con dos PCs en red local.

4.2. Contenedorización con Docker Compose

El despliegue con Docker [52, 53] se basa en **una única imagen personalizada**, construida a partir del `Dockerfile` sobre Ubuntu 22.04 *Long Term Support* (LTS). Esa imagen incluye todo lo necesario para ejecutar el sistema: GStreamer 1.0 con sus *plugins*, FFmpeg [48], las librerías Python y `pika` para RabbitMQ. No hay una imagen por servicio ni una imagen separada para el núcleo y para la interfaz: los once servicios del conjunto de servicios parten de esa misma imagen y ejecutan un proceso diferente sobre ella, `votocore` como núcleo de mezcla, los contenedores de cámara inyectando señal con FFmpeg,

el servicio de telemetría registrando eventos, etc. Voctogui **no tiene imagen Docker**: corre directamente en el equipo anfitrión porque necesita acceder al servidor gráfico del sistema para mostrar la interfaz. La única imagen externa que se utiliza es la oficial de RabbitMQ (`rabbitmq:3-management`), descargada automáticamente de Docker Hub. La imagen propia se construye de forma automática la primera vez que se arranca el sistema. El contenido funcional de ambos ficheros se reproduce en la sección 6.2 del Anexo C.

Para arrancar el sistema completo basta con ejecutar `make docker-up`. Internamente, el comando detiene cualquier contenedor previo para partir de un estado limpio, levanta todos los servicios en segundo plano y espera a que voctocore supere su verificación de salud (puerto 9999 disponible) antes de continuar. Una vez el núcleo está listo, lanza voctogui de forma nativa en el equipo y abre un monitor de programa en el puerto 15000. Al cerrar la sesión con Ctrl+C, todos los procesos, tanto los contenedores como los procesos nativos, se detienen de forma coordinada.

El principal problema técnico que hubo que resolver fue el aislamiento de red. En Docker, cada contenedor está aislado y cuando se conecta a `localhost` solo se comunica consigo mismo. Esto suponía un problema porque las cámaras necesitan enviar la señal a voctocore a través de `localhost`. La solución fue hacer que todos los contenedores compartan la red de voctocore mediante la opción `network_mode: "service:voctocore"` en el fichero Compose, de forma que todos los procesos se ven entre sí como si estuvieran en la misma máquina, sin modificar ninguna línea del código original.

Voctogui se conecta a voctocore a través del puerto 9999 y comparte los recursos visuales como logos e imágenes de fondo mediante un volumen Docker, por lo que se pueden modificar sin tener que reconstruir la imagen. Durante las pruebas se detectó que algunos contenedores intentaban conectarse a voctocore antes de que estuviera listo, lo que causaba fallos en el arranque. Para resolverlo se aplican dos medidas independientes: Docker reinicia automáticamente cualquier contenedor que falle; además, antes de lanzar cada servicio se comprueba que el puerto 9999 está disponible. Ambas garantizan un despliegue robusto y repetible en cualquier máquina con Docker instalado (figura 4.3).

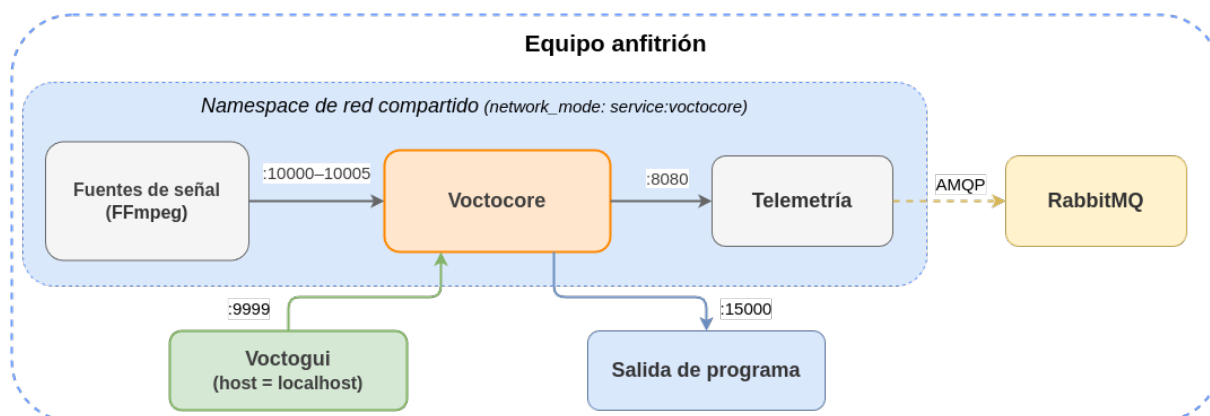


Figura 4.3: Escenario de despliegue con Docker Compose.

La Sección 4.3 describe cómo este mismo sistema se despliega sobre un clúster Kubernetes para entornos de investigación como el proyecto CyberNEMO.

4.3. Orquestación con Kubernetes

Kubernetes [54] es un sistema de orquestación de contenedores que gestiona de forma declarativa el ciclo de vida de los servicios: arranque, reinicio ante fallos y escalado. A diferencia de Docker Compose, que está pensado para un único equipo, Kubernetes puede ejecutarse sobre infraestructura en la nube sin modificar el código de la aplicación, lo que lo convierte en el entorno de producción natural para proyectos de investigación como CyberNEMO. Para las pruebas locales se utilizó Minikube [55], que levanta un clúster Kubernetes completo en el propio equipo de desarrollo sin necesidad de infraestructura en la nube.

La unidad mínima de despliegue en Kubernetes es el *pod*. Cuando varios contenedores se agrupan en el mismo pod, comparten automáticamente la interfaz de red, es decir, todos ven el mismo `localhost`. Este comportamiento resuelve de forma nativa el mismo problema de aislamiento de red que en Docker se solucionó con el espacio de red compartido. El pod principal, llamado `studio`, agrupa voctocore, todas las fuentes de vídeo (`cam1–4`, `break`, `intro`, *stream blanker* y *audio manager*) y el servicio de telemetría. Al compartir pod, todas las conexiones TCP internas funcionan sin modificar una sola línea de código. RabbitMQ se despliega en un pod independiente; la telemetría le publica sus eventos mediante AMQP.

El despliegue se define en cinco ficheros *YAML Ain't Markup Language* (YAML): `studio.yaml` define el pod principal con todos sus contenedores y los puertos expuestos; `rabbitmq.yaml` despliega RabbitMQ como pod independiente; `configmap.yaml` centraliza todos los parámetros de configuración (resolución, puertos, fuentes de vídeo) como variables de entorno accesibles por todos los contenedores, de forma que un solo cambio aquí se aplica a todo el clúster; `pvc.yaml` reserva 1 GiB de almacenamiento persistente para los registros de telemetría, que así sobreviven al reinicio del pod; y `secret.yaml` almacena las credenciales de RabbitMQ de forma segura, sin escribirlas directamente en los manifiestos. A diferencia del escenario Docker, donde la imagen se construye en el propio equipo, Kubernetes descarga la imagen directamente de Docker Hub (`martin1210235/voctomix:latest`), lo que permite desplegar el sistema en cualquier nodo del clúster sin necesidad de construirla localmente. El contenido funcional de los manifiestos se reproduce en la sección 6.2 del Anexo C.

El orden de arranque también requiere coordinación. Si el pod `studio` intentase arrancar antes de que RabbitMQ estuviera listo, la telemetría fallaría al intentar conectarse. Para evitarlo, un pequeño contenedor auxiliar se ejecuta primero y simplemente espera hasta que RabbitMQ responde; solo entonces arranca el resto del pod. Del mismo modo, una comprobación periódica sobre el puerto 9999 confirma que voctocore está operativo antes de que el sistema empiece a aceptar conexiones externas.

Para arrancar el sistema, el operador lanza en PC1 un comando que primero comprueba si Minikube ya está en marcha y, si no, lo inicia. A continuación aplica todos los manifiestos YAML al clúster y espera a que los pods estén completamente listos. Una vez que el sistema está operativo, establece un reenvío de puertos (`kubect1 port-forward --address 0.0.0.0`) que hace accesibles desde la red local los puertos de voctocore (`control`, *previews*, salida de programa). En PC2, el operador lanza voctogui apuntando a la IP de PC1, exactamente igual que en el escenario de dos PCs nativo. Para detener el sistema basta con cerrar el terminal de PC1, lo que cierra automáticamente el reenvío de puertos;

el clúster y los pods permanecen activos y pueden retomarse en cualquier momento sin necesidad de volver a desplegarlos (figura 4.4).

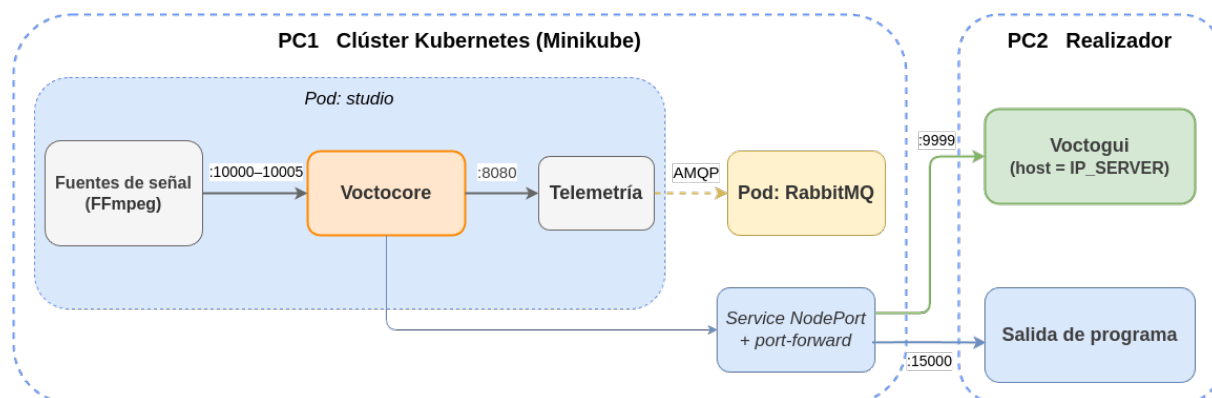


Figura 4.4: Escenario de despliegue con Kubernetes.

Los cuatro escenarios de despliegue cubren un rango amplio de necesidades: el escenario de un PC es el punto de entrada natural para el desarrollo y las pruebas; el de dos PCs permite separar el servidor del puesto del realizador en una red local; Docker elimina la instalación de dependencias y hace el sistema portable a cualquier máquina; y Kubernetes lo integra en infraestructuras de producción como la del proyecto CyberNEMO. En todos los casos el código de Voctomix 2.0 es idéntico, y solo cambia el entorno de ejecución.

5. Pruebas de validación del sistema de producción remota de vídeo

A lo largo de este capítulo se presentan los resultados obtenidos durante la validación de Voctomix 2.0. Se verifica el funcionamiento completo del sistema, se mide el consumo de CPU, RAM y energía en función del número de cámaras activas, se analizan las latencias de conmutación y de transición entre modos de composición, y se evalúa la estabilidad a largo plazo y la capacidad de recuperación ante fallos. Todo ello se compara entre los dos escenarios de despliegue instrumentados.

Los escenarios sobre los que se ha realizado el análisis son:

- **Docker Compose.** Todos los servicios contenerizados y orquestados con Docker Compose en un único equipo.
- **Kubernetes con Minikube.** Despliegue sobre un clúster local Minikube.

Los escenarios de un PC y dos PCs en red local fueron validados funcionalmente (arranque, conmutación, composites, telemetría y latencia), pero no se incluyeron en el análisis de rendimiento instrumentado al quedar fuera del alcance de este trabajo.

5.1. Entorno de pruebas

Todas las pruebas se realizaron en el mismo equipo, comparando dos escenarios de despliegue: Docker Compose y Kubernetes con Minikube. La señal de vídeo fue generada con FFmpeg [48]: cuatro fuentes a 1920×1080 a 25 fps en formato RAW I420 sin compresión, cuyo tráfico circula íntegramente por la red interna del equipo sin atravesar ninguna interfaz física. La tabla 5.1 recoge las especificaciones del entorno.

Tabla 5.1: Especificaciones del entorno de pruebas.

Componente	Especificación
CPU	Intel Core i9-10900X @ 3,70 GHz (10 núcleos / 20 hilos)
RAM	128 GB DDR4
Sistema operativo	Ubuntu 22.04.5 LTS (kernel 6.8.0-111)
Docker	Engine 29.1.3 / Compose 2.37.1
Kubernetes	Minikube v1.38.1 / kubectl v1.36.0
Red interna	Loopback (sin interfaz física)

5.2. Validación funcional

En este apartado se comprueba que cada módulo del sistema funciona correctamente con señal activa. Para ello se ejecutaron las acciones correspondientes desde la interfaz de control y se verificó la respuesta del sistema, tanto en Docker Compose como en Kubernetes. La tabla 5.2 resume lo que se comprobó en cada caso.

Tabla 5.2: Funcionalidades verificadas durante las pruebas.

Módulo	Qué se verificó
Composites	Los cuatro modos de combinación visual y la variante mirror se aplican correctamente sobre la señal.
Transiciones	El cambio entre modos de composición es animado y suave, sin salto brusco en la señal de programa.
<i>Overlay</i>	El rótulo de texto aparece y desaparece con animación, y se desactiva automáticamente en cada corte de cámara.
<i>Stream blanker</i>	La señal de programa cambia entre los estados LIVE, PAUSE y NOSTREAM al pulsar el botón correspondiente.
Audio Follows Video	Al cambiar la cámara activa, el audio hace un fundido cruzado automático hacia la pista de la nueva fuente, sin intervención del operador.
Telemetría (STATE)	Cada 5 s se publica en RabbitMQ un mensaje con el estado completo del mezclador, con una latencia inferior a 50 ms.
Telemetría (CHANGE)	Cada acción sobre la interfaz genera de forma inmediata un evento en RabbitMQ, con una latencia inferior a 50 ms.

Todas las funcionalidades se verificaron con éxito en los dos escenarios de despliegue, lo que confirma que el sistema está operativo y listo para las pruebas de rendimiento y latencia que se presentan a continuación. Las herramientas de medición empleadas en las secciones siguientes se reproducen, en su contenido funcional, en la sección 6.2 del Anexo C.

5.3. Rendimiento del sistema

Para garantizar que Voctomix puede funcionar durante una retransmisión larga sin saturar el equipo, se midió el consumo de CPU, RAM y energía en los dos entornos de despliegue. Se ejecutó una sesión de cinco minutos con 1, 2, 3 y 4 cámaras activas; los valores de CPU y RAM que aparecen en las gráficas son la mediana de las muestras recogidas cada 5 s por la telemetría. El consumo energético se midió con el sensor *Running Average Power Limit* (RAPL) [56] del procesador; los valores de las gráficas descuentan el consumo en reposo de cada entorno para mostrar únicamente el coste atribuible a Voctomix. La figura 5.1 muestra el uso de CPU en cada entorno.

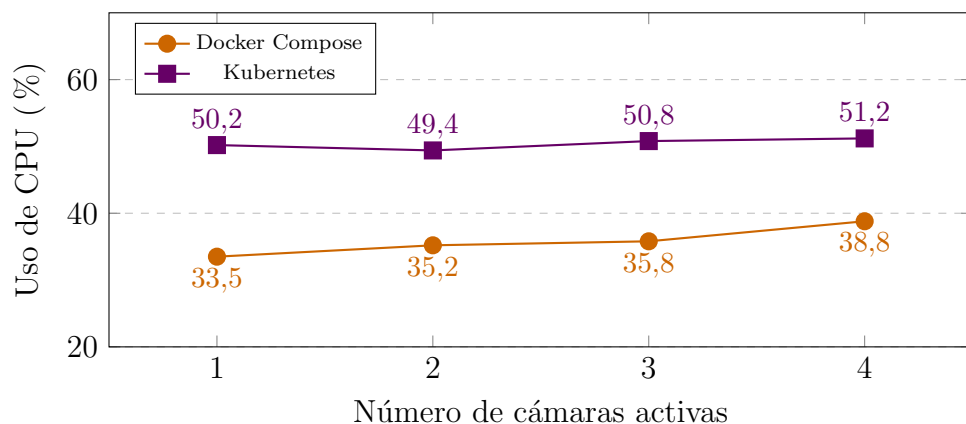


Figura 5.1: Uso de CPU en función del número de cámaras activas.

En Docker la CPU escala con el número de cámaras, de 33,5 % con una a 38,8 % con cuatro (+5,3pp), porque cada fuente añade trabajo real al pipeline de GStreamer. En Kubernetes, en cambio, la CPU se mantiene fija en torno al 50 % independientemente del número de cámaras. Esto ocurre porque Minikube ejecuta en el mismo equipo los cuatro procesos de gestión del clúster, que consumen ese 50 % por sí solos incluso con una sola cámara activa. El trabajo de mezcla de vídeo es idéntico en los dos entornos, pero en Kubernetes queda oculto detrás de ese consumo fijo del plano de control. El incremento de 5,3pp distribuido en tres cámaras adicionales supone un coste medio de 1,8pp por fuente activa, lo que refleja la eficiencia del procesamiento de GStreamer: con cuatro cámaras, el sistema ocupa únicamente el 38,8 % de la capacidad del procesador, dejando margen amplio para incorporar más fuentes o efectos en tiempo real sin comprometer la estabilidad. En un despliegue en la nube, donde el plano de control de Kubernetes se ejecuta en servidores del proveedor sin compartir recursos con **votocore**, el perfil de escalado de CPU sería equivalente al observado en Docker Compose. La figura 5.2 muestra el uso de RAM.

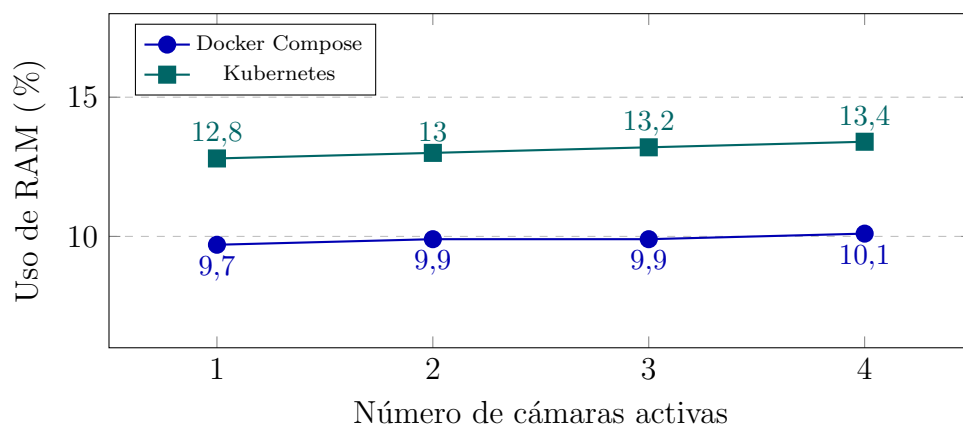


Figura 5.2: Uso de RAM en función del número de cámaras activas.

El uso de RAM es prácticamente plano en los dos entornos: Docker se mantiene entre 9,7 % y 10,1 %, y Kubernetes entre 12,8 % y 13,4 %. La diferencia de unos tres puntos entre ambos corresponde a la memoria que ocupa el plano de control de Kubernetes.

Añadir cámaras no incrementa el consumo de forma significativa porque voctocore reserva al arrancar las cuatro entradas del pipeline de GStreamer independientemente de cuántas cámaras estén activas; cada cámara adicional solo incorpora un proceso FFmpeg de captura que ocupa entre 50 y 80 MB. La figura 5.3 muestra el consumo energético neto.

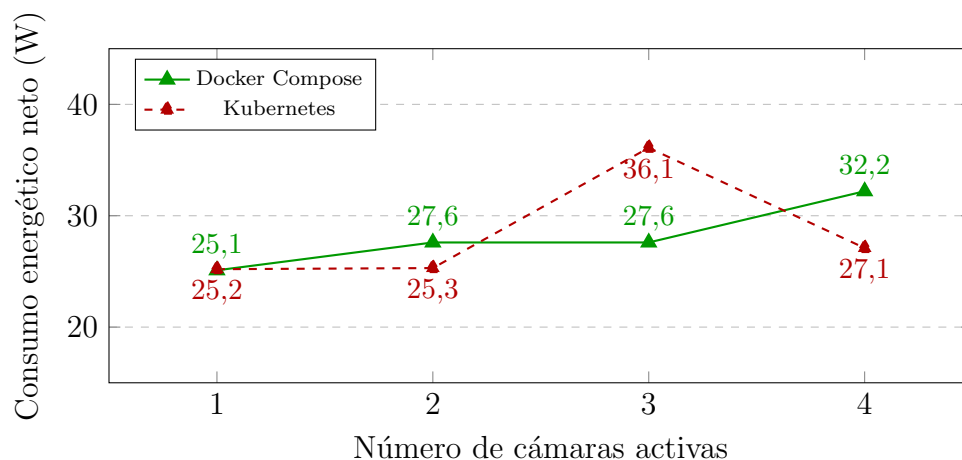


Figura 5.3: Consumo energético neto medido por RAPL en función del número de cámaras activas.

El consumo neto de Voctomix es similar en los dos entornos: entre 25 y 32 W en Docker, y entre 25 y 27 W en Kubernetes si se excluye el dato atípico de tres cámaras. Ese valor atípico (36,1 W) se explica por un problema en la medición de la línea base de esa sesión concreta: el consumo del clúster de Minikube en reposo se midió en 59,2 W en lugar de los 62 W que registró en el resto de sesiones. El consumo neto se calcula restando esa línea base al total medido con Voctomix en marcha; si la referencia es más baja de lo habitual, el resultado de la resta queda inflado. Corrigiendo con los 62 W habituales, el valor para tres cámaras sería de unos 33 W, más acorde con las otras mediciones. Las tres sesiones restantes de Kubernetes (N=1, 2 y 4) dan valores de entre 25 y 27 W, lo que confirma que 36,1 W no es representativo del comportamiento real del sistema.

En cuanto al margen disponible, el pico de 32 W con cuatro cámaras en Docker representa menos del 20 % del *Thermal Design Power* (TDP) del procesador (165 W), lo que indica que el sistema dispone de un amplio margen de capacidad para añadir más fuentes o efectos sin comprometer la estabilidad. En Kubernetes el consumo total del equipo es mayor porque el plano de control de Minikube añade una carga fija que no existe en Docker. Este sobrecoste desaparecería en un despliegue en la nube (*Elastic Kubernetes Service* (EKS), *Google Kubernetes Engine* (GKE) o *Azure Kubernetes Service* (AKS)), donde el plano de control corre en servidores del proveedor y no consume recursos del equipo local.

Respecto al ancho de banda, cada cámara RAW I420 a 1080p25 genera 622 Mbps, lo que supone 2,49 Gbps con cuatro cámaras activas, todo ello circulando por la red virtual interna sin salir al exterior.

5.4. Latencia del sistema

Docker Compose tarda unos 20 s en arrancar porque levanta los contenedores directamente sobre el equipo, sin ningún paso previo. Kubernetes, en cambio, necesita alrededor de 2 minutos porque antes de arrancar los servicios de Voctomix tiene que poner en marcha el propio clúster de Minikube: el nodo del clúster, la base de datos de estado y los procesos de gestión. Solo cuando todo eso está listo empieza a desplegar la aplicación. Este tiempo mayor es asumible porque el sistema se despliega una sola vez antes del evento y permanece activo durante toda la retransmisión. El parámetro relevante durante la emisión es la rapidez con la que el sistema responde al operador, aspecto que se analiza a continuación.

5.4.1. Latencia de conmutación

Durante una retransmisión en directo el operador cambia de cámara continuamente. Si ese cambio tarda demasiado, el espectador percibe un retraso o un desajuste entre la imagen y el audio. Por ello resulta necesario medir cuánto tarda el sistema en ejecutar ese cambio desde que el operador pulsa el botón.

Para medirlo se enviaron 30 órdenes de cambio de fuente seguidas al mezclador y se cronometró el tiempo entre el envío de la orden y la confirmación de `votocore` de que GStreamer había aplicado el cambio. Al no haber ningún proceso de codificación que introduzca retardo adicional, las latencias medidas reflejan únicamente el tiempo de procesamiento interno de GStreamer. La figura 5.4 muestra la distribución de los resultados.

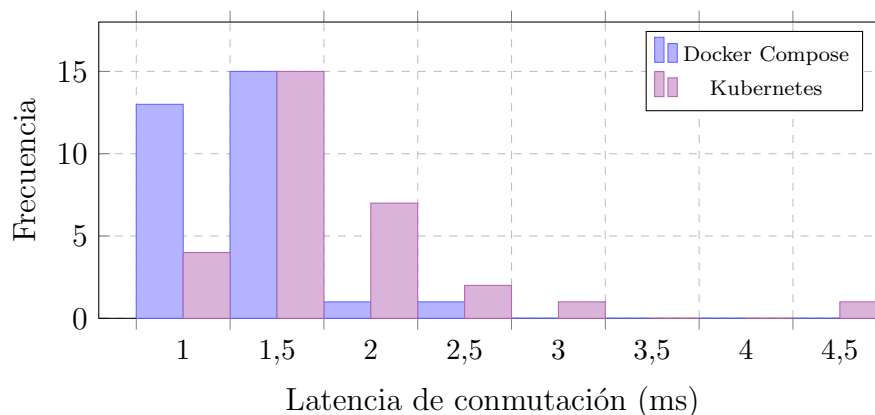


Figura 5.4: Distribución de latencias de conmutación.

Los resultados obtenidos son satisfactorios: todos los cambios tardaron menos de 5 ms en los dos entornos. La mediana fue de 1,5 ms en Docker y 1,8 ms en Kubernetes, lo que está muy por debajo de los 45 ms que fija el estándar *International Telecommunication Union – Radiocommunication Sector* (ITU-R) BT.1359-1 [57] como umbral máximo tolerable de desincronía entre audio y vídeo. En la práctica, cambiar de cámara es instantáneo y el espectador no percibe ningún salto.

5.4.2. Latencia de transición de modo de composición

Además de medir cuánto tarda el sistema en cambiar de cámara, resulta relevante conocer cuánto tarda en cambiar el modo de composición, es decir, pasar por ejemplo de ver

una sola cámara en pantalla completa a verlas en pantalla dividida. Esta operación es diferente a un corte de cámara porque GStreamer tiene que animar la mezcla fotograma a fotograma, y para hacerlo de forma sincronizada espera al siguiente ciclo de procesamiento del pipeline, que ocurre cada 40 ms a 25 fps. Conocer esta latencia permite evaluar si el operador percibirá retraso al usar los modos de composición durante una retransmisión.

Se midió enviando la orden de cambio de modo de composición al mezclador y cronometrando el tiempo hasta recibir la confirmación de que el cambio se había aplicado. Se realizaron 32 mediciones en cada entorno cambiando entre los cuatro modos disponibles. La figura 5.5 recoge las medianas obtenidas.

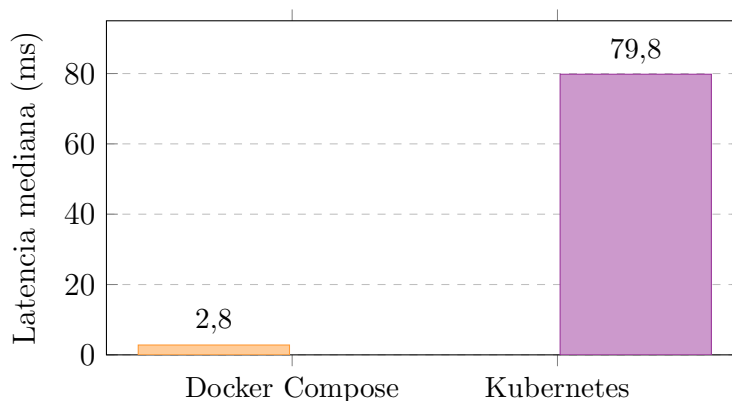


Figura 5.5: Latencia mediana de transición de modo de composición.

En Docker la transición tarda 2,8 ms, prácticamente igual que un corte de cámara. La CPU está libre y GStreamer alcanza el siguiente ciclo de fotograma sin esperar. En Kubernetes la latencia sube a 79,8 ms, lo que corresponde aproximadamente a dos ciclos completos de 40 ms a 25 fps: el plano de control de Minikube mantiene la CPU ocupada y GStreamer pierde el primer ciclo disponible. Este retardo es perceptible para el operador; la animación de transición se ejecuta correctamente, aunque comenzando 80 ms después de lo que ocurriría en Docker. En un despliegue en la nube, donde el plano de control no comparte recursos con *votocore*, la latencia sería comparable a la de Docker.

5.5. Estabilidad a largo plazo

En sistemas de procesamiento continuo de vídeo es habitual que, con el tiempo, la CPU vaya consumiendo cada vez más sin motivo aparente o que la memoria crezca de forma silenciosa hasta agotar los recursos del equipo. Estos problemas, conocidos como *CPU creep* y fuga de memoria [58], no aparecen en pruebas cortas y solo se detectan dejando el sistema en marcha durante un periodo prolongado. El objetivo de esta prueba es comprobar que Voctomix no presenta ninguno de estos comportamientos, tanto en Docker Compose como en Kubernetes.

Se arrancó el sistema completo con las cuatro cámaras activas y se dejó funcionando 31 minutos seguidos en cada entorno. Las fuentes de vídeo se generaron con FFmpeg directamente en el equipo para que nada externo pudiera fallar. Cada 5 segundos la telemetría anotaba el uso de CPU y RAM del sistema, acumulando 377 muestras en

Docker Compose y 372 en Kubernetes. Las figuras 5.6 y 5.7 muestran la evolución temporal del uso de CPU y RAM en cada entorno.

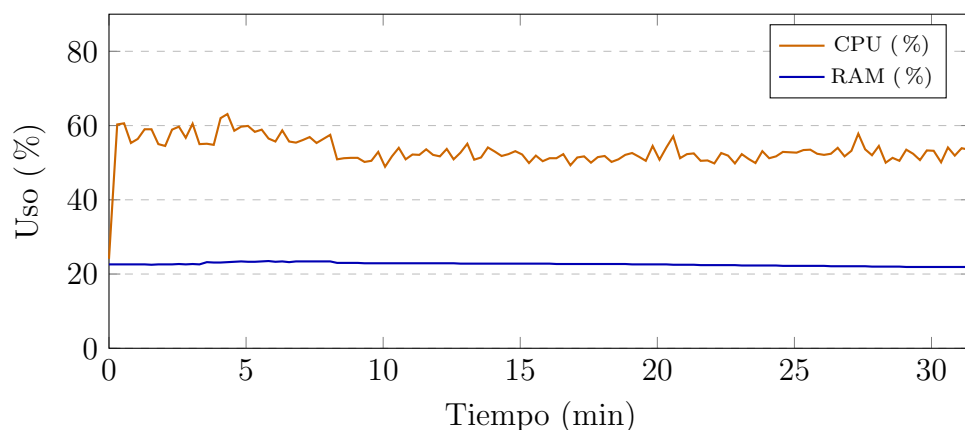


Figura 5.6: Uso de CPU y RAM durante la prueba de estabilidad en Docker Compose.

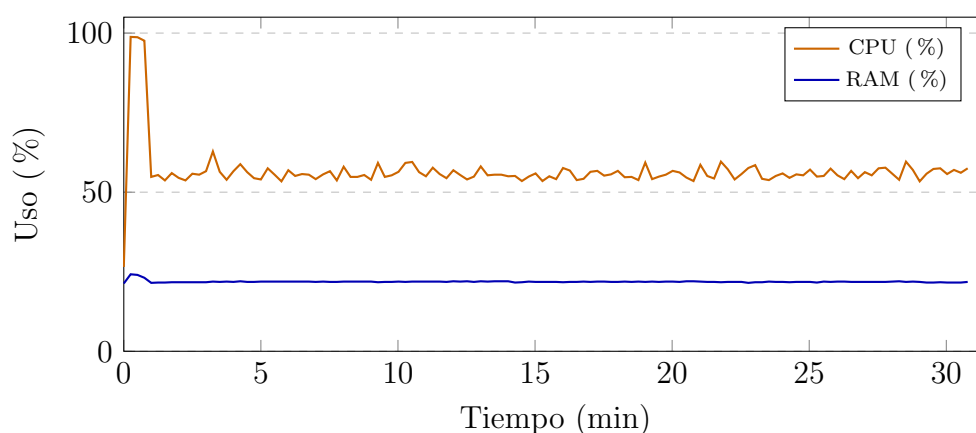


Figura 5.7: Uso de CPU y RAM durante la prueba de estabilidad en Kubernetes Minikube.

En Docker Compose la CPU presentó valores de hasta el 63 % durante los primeros ocho minutos de arranque y se estabilizó a continuación en la banda del 49–58 %, sin ninguna tendencia creciente a lo largo de toda la sesión. En Kubernetes el comportamiento fue análogo: pasado el primer minuto de inicialización, en el que la carga sube momentáneamente hasta casi el 100 % mientras los diez contenedores del pod arrancan de forma simultánea, la CPU se estabilizó igualmente en la banda del 54–60 %. La diferencia de aproximadamente 3 % respecto a Docker se debe al *overhead* añadido por el *kubelet* y el *runtime* de contenedores de Kubernetes, que ejecutan procesos de supervisión y gestión de forma continua. En ambos casos la RAM se mantuvo completamente plana durante toda la sesión, lo que confirma que no hay fuga de memoria en ninguno de los dos entornos. Durante toda la prueba no se registró ningún reinicio de contenedor ni ningún fallo, lo que confirma que el sistema es apto para retransmisiones de larga duración sin necesidad de supervisión continua, independientemente del entorno de despliegue utilizado.

5.6. Resiliencia y recuperación

Se midió cuánto tarda el sistema en recuperarse cuando falla una cámara. Para ello se mató el proceso principal del contenedor `cam1` con `kill -9`, simulando un fallo real, y se cronometró el tiempo hasta que el contenedor volvió a estar operativo.

En Docker Compose no existe penalización entre reinicios consecutivos, por lo que las 10 iteraciones se realizaron de forma consecutiva. En Kubernetes fue necesario esperar 12 minutos entre cada *kill*. Esto se debe al mecanismo *CrashLoopBackOff*: si un contenedor se reinicia demasiadas veces seguidas, Kubernetes lo penaliza con esperas cada vez más largas para evitar que un proceso con fallos consuma recursos sin control. El contador de penalización se reinicia después de 10 minutos de funcionamiento estable, así que esperar 12 minutos entre mediciones garantiza que cada cronometrado refleja el tiempo real de recuperación, sin distorsión acumulada. Se realizaron 10 iteraciones en Kubernetes. Las figuras 5.8 y 5.9 muestran el tiempo de recuperación en cada iteración.

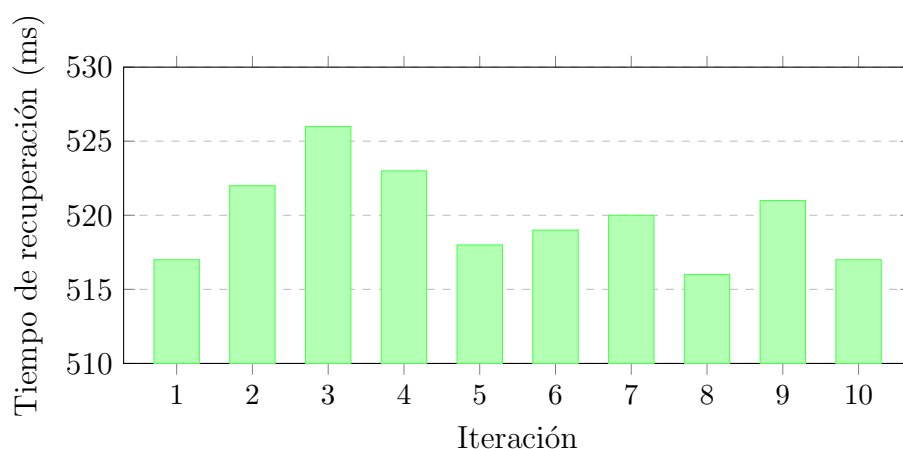


Figura 5.8: Tiempo de recuperación del contenedor `cam1` en cada iteración en Docker Compose.

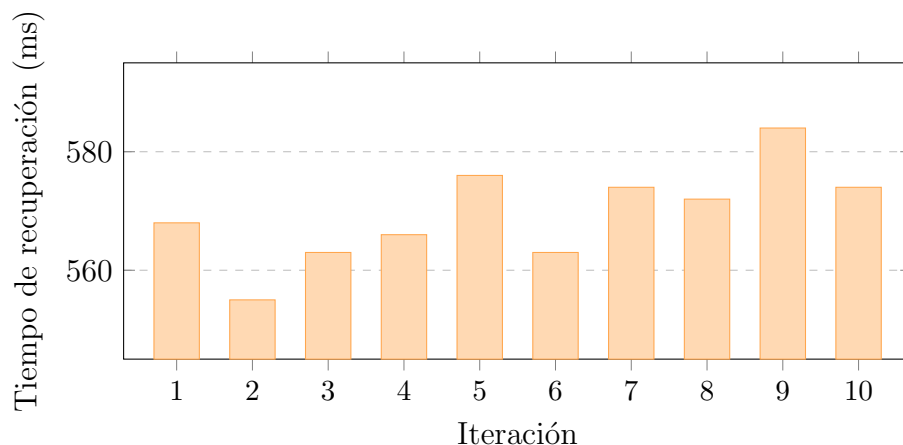


Figura 5.9: Tiempo de recuperación del contenedor `cam1` en cada iteración en Kubernetes (Minikube).

Los ejes Y de las figuras 5.8 y 5.9 no empiezan en cero: parten de 510 ms y 545 ms

respectivamente para que la pequeña variación entre iteraciones sea visible; en términos absolutos, todos los tiempos de Docker se sitúan entre 516 y 526 ms, y los de Kubernetes entre 555 y 584 ms. En Docker Compose la recuperación fue siempre de unos 520 ms, con apenas 10 ms de variación entre las 10 iteraciones. En Kubernetes fue algo más lento: la mediana es de 570 ms y la variación de 29 ms, un 10 % por encima de Docker. Esto se debe a que Kubernetes no detecta el fallo en el momento exacto en que ocurre, sino a través de ciclos de supervisión del *kubelet* que añaden entre 40 y 50 ms de latencia. En cualquier caso el resultado es bueno para una retransmisión en directo: si una cámara cae en plena emisión, el sistema la recupera solo en menos de un segundo, sin que el operador tenga que hacer nada. El mezclador puede seguir emitiendo con las demás cámaras durante ese tiempo, por lo que el espectador prácticamente no lo notaría.

6. Conclusiones y líneas futuras

En este capítulo se resumen las principales aportaciones del trabajo y se evalúa el grado de consecución de los objetivos planteados en el Capítulo 1. La sección 6.1 recoge las conclusiones del desarrollo e implementación del sistema. La sección 6.2 identifica las líneas de trabajo futuro que podrían ampliar las capacidades de Voctomix 2.0 o adaptarlo a nuevos casos de uso.

6.1. Conclusiones

Este trabajo ha desarrollado y validado Voctomix 2.0, un sistema modular de producción audiovisual en directo de código abierto que extiende la herramienta Voctomix original con funcionalidades de nivel profesional. A continuación se evalúa el grado de consecución de cada uno de los seis objetivos específicos planteados.

OE-1: Funcionalidades de producción profesional. Se han implementado los cuatro modos de composición de imagen con transiciones animadas y la rotulación dinámica con fundido configurable y desactivación automática en cada corte. El gestor de estados de emisión con tres modos (LIVE, PAUSE y NOSTREAM) permite interrumpir la señal de salida sin detener el procesamiento interno; la selección automática del audio según la fuente de vídeo activa (Audio Follows Video) elimina la gestión manual durante la producción.

OE-2: Servicio de telemetría. El servicio de telemetría registra en tiempo real los eventos de operación del mezclador (tipo **CHANGE**) y el estado periódico del sistema (tipo **STATE**, cada 5 segundos) en formato JSON Lines. Los eventos se publican simultáneamente en un *broker* RabbitMQ mediante AMQP, lo que permite su integración con sistemas externos de monitorización o análisis sin modificar el núcleo de la aplicación; los datos obtenidos han permitido además caracterizar el comportamiento del sistema en los cuatro escenarios de despliegue.

OE-3: Contenerización con Docker. El sistema completo se ha empaquetado en una pila de 11 contenedores Docker orquestada con Docker Compose, incluyendo voctocore, seis fuentes de vídeo, el *stream blanker*, el gestor de audio, el *broker* RabbitMQ y el servicio de telemetría. El sistema arranca ejecutando **make docker-up**: el comando levanta los 11 servicios, espera a que voctocore esté listo y lanza voctogui de forma nativa en el equipo, sin necesidad de instalar manualmente GStreamer ni Python.

OE-4: Despliegue en Kubernetes. El sistema se ha desplegado sobre un clúster local

Minikube mediante manifiestos declarativos que reproducen la misma funcionalidad que el escenario Docker Compose en una infraestructura orquestada. El orden de arranque controlado y las sondas de preparación garantizan que las dependencias entre servicios se respetan automáticamente. La arquitectura del sistema escala de forma natural desde Docker Compose hacia Kubernetes sin necesidad de modificar el código del núcleo ni de la interfaz gráfica.

OE-5: Validación en múltiples escenarios. El sistema ha sido validado funcionalmente en cuatro escenarios de despliegue: monopuesto nativo, dos PCs en red local, Docker Compose y Kubernetes. El análisis de rendimiento instrumentado se centró en los dos últimos, donde los resultados confirman que el sistema cumple los requisitos de rendimiento para producción en directo, con latencia de conmutación y de transición entre modos de composición muy por debajo del umbral tolerado, estabilidad a largo plazo sin fugas de memoria y correcto funcionamiento del sistema de telemetría.

OE-6: Despliegue en producción real. El sistema ha sido desplegado en un entorno de producción real dentro del proyecto europeo de investigación CyberNEMO (UPM), donde ha actuado como motor de producción audiovisual. Este despliegue acredita la madurez del sistema más allá del prototipo académico y lo posiciona como una contribución directamente aplicable a infraestructuras de investigación y producción audiovisual profesional.

La tabla 6.1 resume el grado de consecución de los objetivos y las secciones de la memoria donde se documentan.

Tabla 6.1: Consecución de los objetivos específicos del TFG.

Obj.	Descripción	Estado	Sección
OE-1	Composición, rotulación, <i>stream blanker</i> y AFV	Cumplido	§3.6, §3.3
OE-2	Telemetría JSON + RabbitMQ	Cumplido	§3.9
OE-3	Contenerización Docker Compose	Cumplido	§4.2
OE-4	Despliegue en Kubernetes	Cumplido	§4.3
OE-5	Validación multi-escenario	Cumplido	Cap. 5
OE-6	Despliegue real en CyberNEMO	Cumplido	§2.4

El trabajo ha demostrado que es posible construir una plataforma de producción audiovisual profesional íntegramente sobre *software* libre. GStreamer ha sido la pieza central: su modelo de *pipeline* modular ha permitido integrar cada nueva funcionalidad como un módulo independiente sin modificar el núcleo original, lo que facilita el mantenimiento y la extensión del sistema.

La separación entre *votocore* y *votogui* ha demostrado ser robusta en todos los entornos probados y escala de forma natural desde una red local hasta una infraestructura contenerizada y orquestada.

6.2. Líneas de trabajo futuro

El desarrollo de Voctomix 2.0 ha puesto de manifiesto varias áreas de mejora e investigación que quedan fuera del alcance de este Trabajo Fin de Grado pero que constituyen líneas de trabajo futuro de interés.

- **Validación con cámaras físicas en producción real.** La totalidad de las pruebas descritas en este trabajo se han realizado con fuentes sintéticas generadas mediante FFmpeg. La validación definitiva del sistema consiste en conectar cámaras físicas mediante tarjetas de captura y realizar una retransmisión en directo completa. Una producción real permitirá detectar posibles problemas de sincronización, *jitter* de red o estabilidad que no se manifiestan con fuentes sintéticas de frecuencia constante.
- **Migración del *pipeline* de salida de H.264 a H.265/HEVC.** Voctomix 2.0 emplea H.264 como códec de salida predeterminado y lo codifica en tiempo real en el procesador del equipo. La migración del *pipeline* de transcodificación a H.265/HEVC permitiría reducir el ancho de banda de la señal de programa aproximadamente a la mitad manteniendo la misma calidad perceptual [37], lo que facilitaría la distribución en entornos con conectividad limitada. El principal obstáculo es la velocidad de codificación, entre cinco y ocho veces más lenta que la de H.264 [37].
- **Escalado de la resolución de trabajo a 4K.** La resolución de trabajo actual del sistema es 1920×1080 píxeles a 25 fps, con un ancho de banda interno de 622 Mbit/s por fuente RAW I420. La ampliación a resolución 4K (3840×2160) cuadruplicaría ese ancho de banda hasta aproximadamente 2,5 Gbit/s por fuente, lo que requeriría redes internas de alta velocidad. GStreamer admite resoluciones arbitrarias, de modo que la adaptación se realizaría principalmente mediante cambios en la configuración de voctocore, aunque el consumo de CPU crecería proporcionalmente. Este escalado situaría el sistema a la par con los estándares de producción actuales en eventos de mediana y gran escala.
- **Interfaz gráfica con reconfiguración dinámica en tiempo real.** La arquitectura actual de voctogui requiere reiniciar el sistema completo para modificar el número de fuentes activas o la configuración de los modos de composición, lo que limita la adaptabilidad del sistema durante una producción en directo. Una extensión natural sería incorporar en el sistema comandos que permitan añadir y eliminar fuentes en caliente, modificar los parámetros de los *composites* en tiempo real y recargar la configuración sin interrumpir la señal de programa. En el lado del cliente, la interfaz gráfica debería reflejar dinámicamente esos cambios actualizando sus controles sin necesidad de reconexión. Esta mejora eliminaría uno de los principales puntos de fricción del sistema en producciones con configuraciones variables o imprevistos técnicos durante la emisión.
- **Ampliación del análisis de métricas del sistema.** Las pruebas de validación realizadas en este trabajo se han centrado en métricas de consumo de CPU, memoria RAM, energía y latencias de conmutación y de transición entre modos de composición. Una línea futura de interés consistiría en ampliar este análisis con métricas adicionales que permitan caracterizar con mayor precisión el comportamiento del sistema en distintos entornos de despliegue y orientar futuras optimizaciones tanto del rendimiento como de la fiabilidad.

Bibliografía

- [1] Ericsson. Ericsson mobility report. Technical report, Ericsson, 2024. <https://www.ericsson.com/en/mobility-report>.
- [2] C3VOC. Voctomix — GitHub repository. <https://github.com/voc/voctomix>, 2024. Accedido: abril 2026.
- [3] Alberto del Río, Álvaro Llorente, Sofia Ortiz-Arce, Maria Belesioti, George Pappas, Alejandro Muñoz, Luis M. Contreras, and Dimitris Christopoulos. Seamless user-generated content processing for smart media: Delivering QoE-aware live media with YOLO-based bib number recognition. *Electronics*, 14(20):4115, 2025.
- [4] dvswitch contributors. dvswitch — DV mixer for live events. <https://web.archive.org/web/20080708184246/http://dvswitch.alioth.debian.org/>, 2013. Accedido: abril 2026.
- [5] StudioCoast Pty Ltd. vMix live production software. <https://www.vmix.com>, 2024. Accedido: abril 2026.
- [6] OBS Project. OBS Studio — open broadcaster software. <https://obsproject.com>, 2024. Accedido: abril 2026.
- [7] European Commission. CyberNEMO: End-to-end cybersecurity to NEMO meta-OS. <https://cordis.europa.eu/project/id/101168182>, 2024. CORDIS, programa Horizon Europe, ID 101168182. Accedido: junio 2026.
- [8] TM Broadcast. Guía para no perderse sobre la producción remota. <https://tmbroadcast.es/claves-produccion-remota/>, 2023. Accedido: abril 2026.
- [9] Blackmagic Design. ATEM production switchers. <https://www.blackmagicdesign.com/products/atem>, 2024. Accedido: abril 2026.
- [10] Society of Motion Picture and Television Engineers. SMPTE ST 259M / 292M / 424M: Serial digital interface standards. <https://www.smpte.org>, 2023. Accedido: abril 2026.
- [11] GStreamer Project. GStreamer application development manual. <https://gstreamer.freedesktop.org/documentation/>, 2024. Accedido: abril 2026.
- [12] Jon Postel. User Datagram Protocol. RFC 768, IETF, 1980. <https://www.rfc-editor.org/rfc/rfc768>.
- [13] Jon Postel. Transmission Control Protocol. RFC 793, IETF, 1981. <https://www.rfc-editor.org/rfc/rfc793>.
- [14] Society of Motion Picture and Television Engineers. SMPTE ST 2022-6:2012: Transport of high bit rate media signals over IP networks (HBRMT). Standard, SMPTE, 2012. <https://www.smpte.org/standards/document-index/st>.

- [15] Society of Motion Picture and Television Engineers. SMPTE ST 2110-20:2022: Professional media over managed IP networks: Uncompressed active video. Standard, SMPTE, 2022. <https://www.smpte.org/standards/document-index/st>.
- [16] IEEE. IEEE Std 1588-2019: IEEE standard for a precision clock synchronization protocol for networked measurement and control systems. Ieee standard, IEEE, 2019. <https://standards.ieee.org/ieee/1588/6825/>.
- [17] Maxim P. Sharabayko, Maria A. Sharabayko, Jean Dube, Jeongseok Kim, and Joonwoong Kim. SRT: Secure reliable transport protocol. Internet-Draft draft-sharabayko-srt-latest, IETF, January 2024. <https://datatracker.ietf.org/doc/draft-sharabayko-srt/>.
- [18] Henning Schulzrinne, Stephen Casner, Ron Frederick, and Van Jacobson. RTP: A transport protocol for real-time applications. RFC 3550, IETF, 2003. <https://www.rfc-editor.org/rfc/rfc3550>.
- [19] Henning Schulzrinne, Anup Rao, Robert Lanphier, Magnus Westerlund, and Martin Stiemerling. Real-Time Streaming Protocol version 2.0. RFC 7826, IETF, 2016. <https://www.rfc-editor.org/rfc/rfc7826>.
- [20] Society of Motion Picture and Television Engineers. SMPTE ST 377-1: Material exchange format (MXF) — file format specification. SMPTE Standard ST 377-1:2019, SMPTE, 2019.
- [21] Canon Inc. Canon develops XF-AVC: New professional video recording format. <https://global.canon/en/news/2015/apr08e.html>, 2015. Accedido: junio 2026.
- [22] IETF / Matroska.org. Matroska media container specification. <https://www.matroska.org/technical/elements.html>, 2024. Accedido: abril 2026.
- [23] Apple Inc. About Apple ProRes. <https://support.apple.com/en-us/102207>, 2024. Accedido: abril 2026.
- [24] Society of Motion Picture and Television Engineers. SMPTE ST 2019: VC-3 intra-frame video coding. SMPTE Standard ST 2019, Society of Motion Picture and Television Engineers, 2016.
- [25] Avid Technology. Avid DNxHD codec family. <https://www.avid.com/resource-center/dnxhd-and-dnxhr-codecs>, 2023. Accedido: abril 2026.
- [26] ISO/IEC. ISO/IEC 15444-1:2019: Information technology — JPEG 2000 image coding system: Core coding system. International standard, ISO/IEC, 2019. <https://www.iso.org/standard/78321.html>.
- [27] ISO/IEC. ISO/IEC 21122: Information technology — JPEG XS low-latency lightweight image coding system. International Standard ISO/IEC 21122, International Organization for Standardization, Geneva, Switzerland, 2022.
- [28] Roy T. Fielding, Mark Nottingham, and Julian Reschke. HTTP semantics. RFC 9110, IETF, 2022. <https://www.rfc-editor.org/rfc/rfc9110>.
- [29] Haivision. Broadcast transformation report 2025. <https://www.haivision.com>, 2025. Accedido: abril 2026.

- [30] Roger Pantos and William May. HTTP live streaming. RFC 8216, IETF, 2017. <https://www.rfc-editor.org/rfc/rfc8216>.
- [31] ISO/IEC. ISO/IEC 23009-1:2022: Information technology — dynamic adaptive streaming over HTTP (DASH) — part 1: Media presentation description and segment formats. International standard, ISO/IEC, 2022. <https://www.iso.org/standard/83314.html>.
- [32] ISO/IEC. ISO/IEC 13818-1:2023: Information technology — generic coding of moving pictures and associated audio information: Systems. International standard, ISO/IEC, 2023. <https://www.iso.org/standard/83239.html>.
- [33] ISO/IEC. ISO/IEC 14496-12:2022: Information technology — coding of audio-visual objects — ISO base media file format. International standard, ISO/IEC, 2022. <https://www.iso.org/standard/83102.html>.
- [34] Society of Motion Picture and Television Engineers. OV 2067-0:2017 — interoperable master format (IMF) overview. SMPTE Standard OV 2067-0:2017, SMPTE, 2017.
- [35] Apple Inc. *QuickTime File Format Specification*, 2016. <https://developer.apple.com/library/archive/documentation/QuickTime/QTFF/>. Accedido: junio 2026.
- [36] ITU-T. Recommendation ITU-T H.264: Advanced video coding for generic audiovisual services. Itu-t recommendation, International Telecommunication Union, 2021. <https://www.itu.int/rec/T-REC-H.264/en>.
- [37] ITU-T. Recommendation ITU-T H.265: High efficiency video coding. Itu-t recommendation, International Telecommunication Union, 2021. <https://www.itu.int/rec/T-REC-H.265/en>.
- [38] Zhuoyuan Chen, Yuxin Guo, Yao Jin, and Bo Peng. Performance comparison of VVC, AV1, HEVC, and AVC for high resolutions. *Electronics*, 13(5):953, 2024. <https://www.mdpi.com/2079-9292/13/5/953>.
- [39] International Telecommunication Union. H.266: Versatile Video Coding. ITU-T Recommendation H.266, ITU-T, 2020.
- [40] Julian Spittka, Koen Vos, and Jean-Marc Valin. RTP Payload Format for Opus Speech and Audio Codec. RFC 7587, IETF, 2015. <https://www.rfc-editor.org/rfc/rfc7587>.
- [41] Advanced Television Systems Committee. Digital Audio Compression Standard (AC-3, E-AC-3). ATSC Standard ATSC A/52:2012, Advanced Television Systems Committee, 2012. <https://www.atsc.org/wp-content/uploads/2015/03/A52-201212-17.pdf>. Accedido: junio 2026.
- [42] European Telecommunications Standards Institute. Digital Audio Compression (AC-4) Standard; Part 1: Introduction to AC-4. ETSI Technical Specification ETSI TS 103 190-1 V1.3.1, European Telecommunications Standards Institute, 2018. https://www.etsi.org/deliver/etsi_ts/103100_103199/10319001/01.03.01_60/ts_10319001v010301p.pdf. Accedido: junio 2026.

- [43] Chaos Communication Club. 36th Chaos Communication Congress (36C3). https://events.ccc.de/congress/2019/wiki/index.php/Main_Page, 2019. Accedido: junio 2026.
- [44] Olympic Broadcasting Services. OBS at the Tokyo 2020 Olympic Games: Remote production report. <https://www.obs.tv>, 2021. Accedido: abril 2026.
- [45] Appear. Telefónica servicios audiovisuales and Appear selected by LaLiga to power Spain's new contribution & distribution network from Madrid. <https://www.appear.net/blog/2025/08/27/laliga-distribution-network/>, 2025. Accedido: junio 2026.
- [46] Telefónica, S.A. RTVE y Telefónica hacen la primera realización a distancia sin unidad móvil gracias al Edge Computing. <https://tmbroadcast.es/rtve-telefonica-completan-primera-realizacion-distancia-sin-unidad-movil-edge-computing/>, December 2019. Nota de prensa. Accedido: junio 2026.
- [47] European Commission. NEMO: Next generation meta operating system. <https://cordis.europa.eu/project/id/101070118>, 2022. CORDIS, programa Horizon Europe, ID 101070118. Accedido: junio 2026.
- [48] FFmpeg Developers. FFmpeg — a complete, cross-platform solution to record, convert and stream audio and video. <https://ffmpeg.org>, 2024. Accedido: abril 2026.
- [49] International Telecommunication Union – Radiocommunication Sector. Recommendation ITU-R BT.709-6: Parameter values for the HDTV standards for production and international programme exchange. Recommendation BT.709-6, ITU-R, Geneva, Switzerland, 2015.
- [50] Broadcom / VMware. RabbitMQ documentation. <https://www.rabbitmq.com/docs>, 2024. Accedido: abril 2026.
- [51] OASIS. OASIS Advanced Message Queuing Protocol (AMQP) version 1.0. Oasis standard, OASIS, 2012. <https://docs.oasis-open.org/amqp/core/v1.0/amqp-core-overview-v1.0.html>.
- [52] Dirk Merkel. Docker: Lightweight Linux containers for consistent development and deployment. *Linux Journal*, 2014(239):2, 2014. <https://www.linuxjournal.com/content/docker-lightweight-linux-containers-consistent-development-and-deployment>.
- [53] Docker Inc. Docker Compose overview. <https://docs.docker.com/compose/>, 2024. Accedido: abril 2026.
- [54] Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. Borg, omega, and Kubernetes: Lessons learned from three container-management systems over a decade. *ACM Queue*, 14(1):70–93, 2016.
- [55] Kubernetes Authors. Minikube: Run Kubernetes locally. <https://minikube.sigs.k8s.io/docs/>, 2024. Accedido: abril 2026.
- [56] Kashif Nizam Khan, Mikael Hirki, Timo Niemi, Jukka K. Nurminen, and Zhonghong Ou. RAPL in action: Experiences in using RAPL for power measurements. *ACM Transactions on Modeling and Performance Evaluation of Computing Systems*, 3(2):9:1–9:26, 2018.

- [57] International Telecommunication Union – Radiocommunication Sector. Recommendation ITU-R BT.1359-1: Relative timing of sound and vision for broadcasting. Recommendation BT.1359-1, ITU-R, Geneva, Switzerland, 1998.
- [58] Yennun Huang, Chandra Kintala, Nick Kolettis, and N. D. Fulton. Software rejuvenation: Analysis, module and applications. In *Proceedings of the 25th International Symposium on Fault-Tolerant Computing (FTCS)*, pages 381–390. IEEE, 1995.
- [59] Apache Software Foundation. Apache License, version 2.0. <https://www.apache.org/licenses/LICENSE-2.0>, 2004. Accedido: junio 2026.
- [60] Daron Acemoglu and Pascual Restrepo. Automation and new tasks: How technology displaces and reinstates labor. *Journal of Economic Perspectives*, 33(2):3–30, 2019.
- [61] National Institute of Standards and Technology. NIST cybersecurity framework 2.0. NIST Special Publication CSWP 29, U.S. Department of Commerce, 2024.
- [62] International Energy Agency. Data centres and data transmission networks. <https://www.iea.org/energy-system/buildings/data-centres-and-data-transmission-networks>, 2023. Accedido: junio 2026.

Anexo A. Aspectos éticos, económicos, sociales y ambientales

A1. INTRODUCCIÓN

Este Trabajo Fin de Grado desarrolla y optimiza Voctomix 2.0, un sistema de código abierto para la producción remota de vídeo en tiempo real. El proyecto amplía la herramienta Voctomix con nuevas funcionalidades: composición de vídeo multicámara, *overlays* dinámicos, telemetría basada en RabbitMQ y despliegue con Docker y Kubernetes, con el objetivo de que cualquier organización pueda realizar retransmisiones de calidad profesional sin necesidad de *hardware* dedicado.

El sistema nació de una necesidad real: muchos grupos universitarios, asociaciones y entidades públicas necesitan cubrir eventos en directo pero no pueden permitirse el equipamiento *broadcast* habitual. Al estar construido sobre herramientas libres, como GStreamer, Python, Docker, RabbitMQ y Kubernetes, el proyecto es auditable, reproducible y puede evolucionar más allá del trabajo individual, alineándose con los principios del *software* libre y la ingeniería abierta.

Este anexo revisa los aspectos éticos, económicos, sociales y ambientales del proyecto.

A2. DESCRIPCIÓN DE IMPACTOS RELEVANTES RELACIONADOS CON EL PROYECTO

A continuación se identifican y justifican los principales impactos del proyecto, agrupados por dimensión:

Impacto en el sector audiovisual y académico

Voctomix 2.0 está pensado para organizaciones que necesitan producción audiovisual pero no tienen presupuesto *broadcast*: grupos de investigación universitarios como CyberNEMO (UPM), asociaciones estudiantiles, organismos públicos y entidades sin ánimo de lucro. Con él pueden hacer retransmisiones multicámara con transiciones animadas, *overlays*, rotulación dinámica y grabación simultánea, usando equipos convencionales, sin *hardware* dedicado.

Aspectos éticos

El sistema está construido sobre *software* libre: Voctomix tiene licencia MIT, GStreamer es LGPL 2.1 y Docker y Kubernetes usan Apache 2.0 [59]. Que todo el código sea abierto

y auditable significa que no hay dependencia de ningún proveedor propietario.

Dicho esto, el modelo de producción remota tiene implicaciones que conviene señalar. Al poder gestionar toda la producción desde un equipo centralizado, se reduce la necesidad de personal técnico presente en el lugar del evento: operadores de cámara, técnicos de audio o encargados de la mezcla que antes viajaban *in situ*. Esto no significa que esos puestos vayan a desaparecer, pero sí que el sector tendrá que adaptarse, y es responsabilidad de quienes adoptan estas tecnologías que ese cambio no derive en precariedad laboral [60].

Por otro lado, al exponer puertos de control por red, como el puerto 9999 de voctocore, la consola de RabbitMQ o la *Application Programming Interface* (API) de telemetría, aumenta el número de puntos de acceso que un atacante podría intentar explotar. Un acceso no autorizado podría interferir con una retransmisión en directo. Por ello, quien despliega el sistema debe aplicar autenticación en RabbitMQ, restringir los puertos mediante cortafuegos y no exponer el sistema a Internet sin protección adicional, siguiendo las recomendaciones del *National Institute of Standards and Technology* (NIST) Cybersecurity Framework [61].

Respecto a la privacidad, la telemetría registra únicamente eventos de operación del mezclador en ficheros locales, sin recopilar datos personales de operadores ni audiencia. Cuando el sistema se utiliza para retransmitir imágenes de personas, el operador debe cumplir el *Reglamento General de Protección de Datos* (RGPD) y la *Ley Orgánica de Protección de Datos* (LOPD), así como respetar los derechos de imagen y de propiedad intelectual sobre el contenido emitido.

Aspectos económicos

La ventaja económica principal de Voctomix 2.0 es que hace posible el modelo de producción remota (REMI): el núcleo de procesamiento corre en un servidor centralizado y no hace falta desplazar a todo el equipo técnico al lugar del evento. Eso reduce directamente los costes de desplazamiento, alojamiento y logística, que en producciones de varios días o en localizaciones lejanas suelen ser el gasto más importante.

A eso se suma el ahorro en *hardware*: una instalación multicámara profesional con equipos dedicados (mezclador Blackmagic ATEM, matrices de vídeo SDI, sistemas de grabación profesional) puede superar los 5.000 € en adquisición [9], sin contar mantenimiento ni formación. Voctomix 2.0 replica esas funcionalidades sobre *hardware* convencional con un coste de despliegue prácticamente nulo, al ser todo *software* libre.

La arquitectura contenerizada permite además reutilizar la misma instalación para distintos eventos, reduciendo el coste por producción conforme crece el volumen de retransmisiones.

Aspectos sociales

El impacto social más claro del proyecto es que acerca la producción audiovisual profesional a organizaciones que normalmente no pueden permitírsela. Al eliminar la barrera del *hardware* dedicado, conferencias académicas, jornadas de puertas abiertas, actos culturales y organismos públicos pueden ofrecer retransmisiones de calidad a sus audiencias. El hecho de publicar el código bajo licencia libre fomenta además que otros puedan mejorarlo y adaptarlo a sus propias necesidades.

Aspectos medioambientales

Desarrollo y optimización de un sistema modular de código abierto
para la producción remota de vídeo en tiempo real

El modelo REMI reduce los desplazamientos de personal y equipos, y con ello las emisiones de CO₂ asociadas al transporte. Sustituir *hardware* especializado por *software* sobre equipos de propósito general también reduce la fabricación de dispositivos dedicados y alarga la vida útil del equipamiento existente.

El lado negativo es que el servidor que ejecuta voctocore trabaja a alta carga de CPU durante toda la retransmisión. Las mediciones realizadas en este trabajo muestran un consumo neto, medido con el sensor RAPL [56], que crece de 25,1 W con una cámara activa a 32,2 W con cuatro, siempre por encima del consumo en reposo del servidor. El procesamiento de vídeo RAW en tiempo real consume más energía que una solución con *hardware* dedicado de bajo consumo, algo habitual en infraestructuras de servidor [62].

Aun así, el balance global es positivo: el ahorro en emisiones por reducir desplazamientos es mayor que el aumento de consumo eléctrico del servidor, especialmente en producciones con muchas personas implicadas o localizaciones geográficamente dispersas.

A3. ANÁLISIS DETALLADO DE ALGUNO DE LOS IMPACTOS

Se analizan en detalle los dos impactos más relevantes: el económico, por ser la principal ventaja diferencial del modelo REMI, y el ético, por implicar riesgos que hay que gestionar activamente.

Impacto económico, reducción de costes en producción remota

1. **Reducción de personal desplazado** En una producción multicámara convencional, todo el equipo técnico viaja al lugar del evento. Con el modelo REMI, el realizador, el técnico de audio y el operador de gráficos trabajan desde una sala de control centralizada; solo los operadores de cámara necesitan estar presentes *in situ*.
2. **Eliminación de *hardware* dedicado** Voctomix 2.0 sustituye mezcladores, matrices de vídeo y sistemas de grabación profesional por *software* ejecutado en *hardware* convencional. El coste de adquisición pasa de varios miles de euros a prácticamente cero, con la única inversión en equipos de red y ordenadores de propósito general.
3. **Escalabilidad sin sobre coste** La arquitectura contenerizada permite levantar el sistema completo, incluyendo telemetría y mensajería, con un único comando y sin licencias adicionales. La misma instalación sirve para todos los eventos de una organización.

Impacto ético, ciberseguridad y responsabilidad profesional

1. **Superficie de ataque en red** El sistema expone varios puertos TCP: control en 9999, entradas de cámara en 10000–10005, salidas en 11000–15000, consola RabbitMQ en 15672 y API de telemetría en 8080. En una red no controlada, cada puerto accesible es un vector de ataque potencial. El ingeniero que despliega el sistema debe aplicar reglas de cortafuegos que restrinjan el acceso a los equipos autorizados.
2. **Autenticación y control de acceso** El protocolo de control TCP de voctocore no incluye autenticación en la capa de aplicación: cualquier cliente que llegue al puerto 9999 puede enviar comandos al mezclador. En una red local controlada esto es aceptable, pero si hay acceso remoto conviene añadir un túnel cifrado (*Virtual*

Private Network (VPN) o *Secure Shell* (SSH) port forwarding) para proteger el canal de control.

3. **Responsabilidad ante el desplazamiento laboral** La adopción de sistemas de producción remota debe ir acompañada de medidas de formación y reconversión para el personal técnico afectado, de modo que la ganancia en eficiencia no se consiga a costa de empeorar las condiciones laborales en el sector audiovisual.

Consideraciones sobre privacidad

El sistema de telemetría registra los eventos de operación del mezclador en ficheros locales y los publica en un *broker* RabbitMQ interno. El operador debe asegurarse de que: (i) el acceso al *broker* está protegido con autenticación; (ii) los registros no contienen datos personales; (iii) el sistema no retransmite contenido protegido por derechos de autor sin la autorización correspondiente. Al mantener toda la telemetría en infraestructura local bajo control del operador, cumplir el RGPD no requiere modificaciones adicionales.

A4. CONCLUSIONES

Desde el punto de vista ético, social, económico y ambiental, Voctomix 2.0 tiene un balance global positivo, aunque hay riesgos que deben gestionarse activamente.

En el plano ético, el uso de *software* libre garantiza que el código es transparente y auditable, y la telemetría local protege la privacidad de los usuarios. Los dos riesgos principales son la exposición de puertos de red, que exige configurar bien la seguridad, y el posible impacto sobre el empleo técnico presencial, que obliga a que la adopción de estas herramientas sea responsable.

Económicamente, la mayor ventaja es la reducción de costes de desplazamiento que permite el modelo REMI, junto con la eliminación del *hardware* dedicado. Esto hace el sistema especialmente útil para organizaciones con presupuesto limitado.

Socialmente, el proyecto acerca la producción audiovisual profesional a comunidades académicas y organizaciones que de otro modo no tendrían acceso a ella.

Medioambientalmente, reducir los desplazamientos implica una bajada real en emisiones de CO₂, y ese ahorro supera el aumento en consumo eléctrico del servidor. El balance ambiental es positivo, siendo el ahorro en transporte el factor determinante.

En conjunto, los riesgos identificados (exposición de puertos, impacto sobre el empleo presencial y mayor consumo eléctrico del servidor) son gestionables con medidas conocidas de seguridad, formación y eficiencia, y no cuestionan la viabilidad del modelo REMI que habilita este proyecto.

Anexo B. Presupuesto económico

B1. TABLA RESUMEN DEL PRESUPUESTO ECONÓMICO

COSTE DE MANO DE OBRA (coste directo)		Horas	Precio/hora	Total	
		320	25 €	8.000,00 €	
COSTE DE RECURSOS MATERIALES (coste directo)		Precio de compra	Uso en meses	Amortización (en años)	Total
PC1: Ordenador personal del alumno		1.500,00 €	8	5	200,00 €
PC2: Equipo de laboratorio ETSIT-UPM (i9-10900X, 128 GB DDR4)		6.000,00 €	8	5	800,00 €
COSTE TOTAL DE RECURSOS MATERIALES					1.000,00 €
GASTOS GENERALES (costes indirectos)	25 %	sobre CD			2.250,00 €
BENEFICIO INDUSTRIAL	6 %	sobre CD + CI			675,00 €
SUBTOTAL PRESUPUESTO					11.925,00 €
IVA APLICABLE				21 %	2.504,25 €
TOTAL PRESUPUESTO					14.429,25 €

Tabla 6.2: Presupuesto económico del proyecto.

B2. ELABORACIÓN DEL PRESUPUESTO ECONÓMICO

A continuación se detalla el presupuesto económico del proyecto. El coste de mano de obra se ha calculado a partir de los 12 créditos ECTS del TFG, a razón de 25 horas por crédito (mínimo estipulado: 300 horas), más 20 horas adicionales correspondientes a reuniones con el tutor, revisiones y preparación de la defensa, totalizando 320 horas. El precio por hora de 25€ corresponde al coste estándar de un ingeniero junior en el sector tecnológico en España. Los recursos materiales incluyen el ordenador personal del alumno (PC1, 1.500€) y el equipo de laboratorio de la *Escuela Técnica Superior de Ingenieros de Telecomunicación* (ETSIT)-UPM empleado para las pruebas de validación (PC2: Intel Core i9-10900X, 128 GB DDR4, NVMe 932 GB, precio estimado de mercado en el momento de adquisición: 6.000€). Ambos equipos se amortizan en 5 años con 8 meses de uso efectivo durante el desarrollo del proyecto.

Anexo C. Evolución del repositorio

C1. INTRODUCCIÓN

Este anexo reúne en un solo lugar los ficheros principales del proyecto, para consultarlos sin necesidad de acceder al repositorio. C2 compara la estructura de directorios antes y después del desarrollo; C3 muestra la interfaz gráfica original; C4 recoge los ficheros de configuración INI; C5 los ficheros de infraestructura Docker; C6 los manifiestos de Kubernetes, y C7 las herramientas con las que se obtuvieron los datos del Capítulo 5.

C2. EVOLUCIÓN DE LA ESTRUCTURA DEL REPOSITORIO

Los dos árboles muestran el repositorio tal como estaba al descargarlo (izquierda) y como quedó tras el desarrollo (derecha). Los ficheros marcados con [N] son los que se incorporaron en este trabajo. Los árboles muestran los ficheros principales; se omiten módulos auxiliares (gestión de argumentos, reloj de sincronización, captura de fotogramas, atajos de teclado, construcción de la interfaz y previsualización de vídeo) y ficheros `__init__.py` por brevedad.

Estado inicial*(voc/voctomix, clonado)*

```

voctomix/
|-- voctocore/
|   |-- lib/
|   |   |-- audiomix.py
|   |   |-- commands.py
|   |   |-- composites.py
|   |   |-- config.py
|   |   |-- controlserver.py
|   |   |-- loghandler.py
|   |   |-- pipeline.py
|   |   |-- response.py
|   |   |-- tcpmulticonnection.py
|   |   |-- tcpsingleconnection.py
|   |   |-- videomix.py
|   |   '-- sources/
|   |       |-- avsource.py
|   |       |-- decklinkavsource.py
|   |       |-- imgvsource.py
|   |       '-- tcpavsource.py
|   |-- tests/
|   |-- voctocore.py
|   '-- default-config.ini
|-- voctogui/
|   |-- lib/
|   |   |-- audioleveldisplay.py
|   |   |-- config.py
|   |   |-- connection.py
|   |   |-- loghandler.py
|   |   |-- toolbar/
|   |   |   |-- buttons.py
|   |   |   |-- helpers.py
|   |   |   |-- misc.py
|   |   |   |-- mix.py
|   |   |   |-- output.py
|   |   |   '-- preview.py
|   |   '-- ui.py
|   |-- ui/
|   |-- voctogui.py
|   '-- default-config.ini
|-- vocto/
|   '-- composite_commands.py
|-- example-scripts/
|   |-- control-server/
|   '-- ffmpeg/
|-- README.md
'-- LICENSE.txt

```

Estado final*(Voctomix 2.0)*

```

voctomix/
|-- voctocore/
|   |-- lib/
|   |   |-- audiomix.py
|   |   |-- avpreviewoutput.py
|   |   |-- avrawoutput.py
|   |   |-- commands.py
|   |   |-- composites.py
|   |   |-- config.py
|   |   |-- controlserver.py
|   |   |-- errors/
|   |   |-- loghandler.py
|   |   |-- overlay.py [N]
|   |   |-- pipeline.py
|   |   |-- response.py
|   |   |-- scene.py [N]
|   |   |-- streamblanker.py [N]
|   |   |-- tcpmulticonnection.py
|   |   |-- tcpsingleconnection.py
|   |   |-- transitions.py [N]
|   |   |-- videomix.py
|   |   '-- sources/
|   |       |-- avsource.py
|   |       |-- decklinkavsource.py
|   |       |-- imgvsource.py
|   |       '-- tcpavsource.py
|   |-- tests/
|   |-- voctocore.py
|   '-- default-config.ini
|-- voctogui/
|   |-- lib/
|   |   |-- audioleveldisplay.py
|   |   |-- config.py
|   |   |-- connection.py
|   |   |-- gui_state_exporter.py [N]
|   |   |-- loghandler.py
|   |   |-- toolbar/
|   |   |   |-- buttons.py
|   |   |   |-- helpers.py
|   |   |   |-- misc.py
|   |   |   |-- mix.py
|   |   |   |-- output.py
|   |   |   |-- overlay.py [N]
|   |   |   |-- preview.py
|   |   |   '-- streamblank.py [N]
|   |   '-- ui.py
|   |-- ui/
|   |-- voctogui.py
|   '-- default-config.ini
|-- vocto/
|   '-- composite_commands.py
|-- example-scripts/
|   |-- control-server/
|   '-- ffmpeg/
|       '-- telemetry_service.py [N]
|-- docker-compose.yml [N]
|-- Dockerfile [N]
|-- k8s_escenario/ [N]
|-- tools/ [N]
|-- images/ [N]
|-- 1pc_escenario1/ [N]
|-- 2pc_escenario2/ [N]
|-- Makefile [N]
|-- README.md
'-- LICENSE.txt

```

C3. EVOLUCIÓN DE LA INTERFAZ GRÁFICA

La figura siguiente muestra la interfaz original, tal como aparecía al clonar el repositorio y antes de ninguna modificación. La comparación con la figura 3.2 del Capítulo 3 permite ver el alcance de los cambios realizados.



Figura 6.1: Interfaz gráfica original de Voctomix al clonar el repositorio.

C4. FICHEROS DE CONFIGURACIÓN DEL SISTEMA

El sistema utiliza dos ficheros de configuración INI. El de **votocore** define el formato de vídeo y audio, las fuentes activas, los modos de composición, las transiciones y los *overlays*. El de **votogui** indica dónde está el servidor, el tamaño de las previsualizaciones y los atajos de teclado. Se muestran los parámetros funcionales; se omiten los campos de posición y *tooltip* de los botones de la barra de herramientas por brevedad.

voctocore/default-config.ini

```
[mix]
videocaps = video/x-raw,format=I420,width=1920,height=1080,framerate=25/1,pixel-aspect
-ratio=1/1,interlace-mode=progressive,colorimetry=bt709
audiocaps = audio/x-raw,format=S16LE,channels=2,layout=interleaved,rate=48000

sources = cam1,cam2,cam3,cam4,break,intro
backgrounds=background1,background2
audiostreams = 1

[source.cam1]
deinterlace = assume-progressive
[source.cam2]
deinterlace = assume-progressive
```

```

[source.cam3]
deinterlace = assume-progressive
[source.cam4]
deinterlace = assume-progressive
[source.break]
deinterlace = assume-progressive
[source.intro]
deinterlace = assume-progressive

[logo1]
file = ../images/logos/logo1.png

[logo2]
file = ../images/logos/logo2.png

[source.background]
kind = img
file = ../images/backgrounds/bg.png

[source.background1]
kind=test
pattern=black
composites=fs

[source.background2]
kind=img
file = ../images/backgrounds/bg.png
composites=sbs,pip,lec

[stream-blanker]
enabled=true
sources=pause,nostream
volume=1.0

[source.stream-blanker]
kind=tcp

[source.stream-blanker-pause]
kind=tcp

[source.stream-blanker-nostream]
kind=tcp

[output-buffers]
mix_out = 10000

[previews]
enabled = true
live = true
deinterlace = false
videocaps=video/x-raw,width=1024,height=576,framerate=25/1

[mirrors]
enabled = true

[overlay]
path = ../images
user-auto-off = true
auto-off = true
blend-time=300

[composites]
fs.a          = *
fs.b          = *
fs.alpha-b    = 0
fs.noswap     = true

fs-pip.a      = *
fs-pip.b      = 0.86/0.85 0.0
fs-pip.alpha-b = 0
fs-pip.inter  = true
fs-pip.mirror = true

```

```

fs-sbs.a          = *
fs-sbs.b          = 1.0/0.5 0.0
fs-sbs.alpha-b    = 0
fs-sbs.inter      = true

fs-lec.a          = *
fs-lec.b          = 1.0 0.0
fs-lec.alpha-b    = 0
fs-lec.crop-b     = 0.31/0
fs-lec.inter      = true
fs-lec.mirror     = true

pip.a             = *
pip.b             = 0.73/0.72 0.26
pip.noswap        = true
pip.mirror        = true

sbs.a             = 0.008/0.25 0.49
sbs.b             = 0.503/0.25 0.49

lec.a             = 0.006/0.01 0.75
lec.b             = 0.76/0.01 0.23
lec.crop-b        = 0/0
lec.mirror        = true

lec_43.a          = 0.006/0.01 0.75
lec_43.b          = 0.76/0.01 0.23
lec_43.crop-a     = 0/0
lec_43.crop-b     = 0/0
lec_43.mirror     = true

fs-b.a            = *
fs-b.b            = *
fs-b.noswap       = true

fs-b-sbs.a        = 0.0/0.5 0.0
fs-b-sbs.alpha-a  = 0.0
fs-b-sbs.b        = *
fs-b-sbs.inter    = true

fs-b-lec.a        = 0.0/1.0 0.0
fs-b-lec.b        = *
fs-b-lec.inter    = true
fs-b-lec.mirror   = true

oao.a             = 0.3/0.2 0.4
oao.alpha-a       = 0.5
oao.b             = 0.2/0.3 0.6
oao.inter         = true
oao.noswap        = true

[transitions]
fs-fs             = 750, fs / fs-b
fs-pip            = 750, fs-pip / pip
fs-sbs            = 750, fs-sbs / sbs
fs-b-pip          = 750, fs-b / pip
fs-b-sbs          = 750, fs-b-sbs / sbs
fs-lec            = 750, fs-lec / lec
fs-b-lec          = 750, fs-b-lec / lec
pip-pip           = 750, pip / sbs / pip
sbs-sbs           = 750, sbs / oao / sbs
_sbs-sbs          = 750, ^sbs / ^oao / sbs

fs-pip_           = 750, |fs-pip / |pip
fs-b-pip_         = 750, fs-b / |pip
fs-lec_           = 750, |fs-lec / |lec
fs-b-lec_         = 750, |fs-b-lec / |lec
pip-pip_          = 750, |pip / sbs / |pip

def               = 750, * / *

```


voctogui/default-config.ini

```
[server]
host=localhost

[previews]
width=320
use=true

[mainvideo]
playaudio=false

[videodisplay]
; system=gl
; system=xv
system=x

[misc]
close=true
cut=true
fullscreen=true

[mainwindow]
; width=1700
; height=1000
; forcefullscreen=true

[toolbar.sources.a]
cam1.name = CAM1
cam1.key = F1
cam2.name = CAM2
cam2.key = F2
cam3.name = CAM3
cam3.key = F3
cam4.name = CAM4
cam4.key = F4
break.name = BREAK
break.key = F5
intro.name = INTRO
intro.key = F6

[toolbar.sources.b]
cam1.name = CAM1
cam1.key = 1
cam2.name = CAM2
cam2.key = 2
cam3.name = CAM3
cam3.key = 3
cam4.name = CAM4
cam4.key = 4
break.name = BREAK
break.key = 5
intro.name = INTRO
intro.key = 6

[toolbar.composites]
fs.name = FULL SCREEN
fs.key = F5
sbs.name = SIDE BY SIDE
sbs.key = F6
pip.name = PIP
pip.key = F7
lec.name = LECTURE
lec.key = F8

[toolbar.mods]
mirror.name = MIRROR
mirror.replace = lec->|lec, pip->|pip
mirror.key = F9

[toolbar.mix]
retake.name = RETAKE
retake.key = BackSpace
```

```
cut.name      = CUT  
cut.key       = Return  
trans.name    = TRANS  
trans.key     = space
```

C5. FICHEROS DE INFRAESTRUCTURA DOCKER

El Dockerfile construye la imagen base instalando GStreamer, FFmpeg, Python y la librería pika. El docker-compose.yml arranca los 11 servicios del sistema. La clave de la arquitectura es `network_mode: "service:votocore"`: con esta opción todos los contenedores de cámara comparten la red de votocore y se comunican por localhost, sin modificar ninguna línea del código original.

Dockerfile

```
FROM ubuntu:22.04

ENV DEBIAN_FRONTEND=noninteractive

RUN apt-get update && apt-get install -y \
    python3 \
    python3-pip \
    python3-gi \
    python3-gi-cairo \
    python3-scipy \
    python3-numpy \
    gir1.2-gstreamer-1.0 \
    gir1.2-gst-plugins-base-1.0 \
    gir1.2-gtk-3.0 \
    gstreamer1.0-tools \
    gstreamer1.0-plugins-base \
    gstreamer1.0-plugins-good \
    gstreamer1.0-plugins-bad \
    gstreamer1.0-plugins-ugly \
    gstreamer1.0-libav \
    ffmpeg \
    netcat-openbsd \
    iproute2 \
    && pip3 install --no-cache-dir pika==1.3.2 \
    && apt-get clean \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /opt/votomix

COPY . /opt/votomix/

HEALTHCHECK --interval=5s --timeout=3s --retries=12 \
    --start-period=20s \
    CMD bash -c "echo '' | nc -zw1 localhost ${VOTOCORE_PORT:-9999}" || exit 1
```

docker-compose.yml

```
x-video-env: &video-env
  WIDTH:    ${WIDTH:-1920}
  HEIGHT:   ${HEIGHT:-1080}
  FRAMERATE: ${FRAMERATE:-25}
  AUDIORATE: ${AUDIORATE:-48000}

# Dedicated bridge network: isolates the telemetry service from the
# votocore network stack, removing the single point of failure that
# would take down all services if votocore crashed.
networks:
  votomix_net:
    driver: bridge

services:
  # 0. RABBITMQ (Message Broker - event decoupling)
  rabbitmq:
    image: rabbitmq:3-management
    container_name: rabbitmq
    networks:
      - votomix_net
```

```
ports:
  - "5672:5672"
  - "15672:15672"
environment:
  RABBITMQ_DEFAULT_USER: ${RABBITMQ_USER:-voctomix}
  RABBITMQ_DEFAULT_PASS: ${RABBITMQ_PASS:-voctomix123}
healthcheck:
  test: ["CMD", "rabbitmq-diagnostics", "ping"]
  interval: 10s
  timeout: 5s
  retries: 6
  start_period: 30s

# 1. VOCTOCORE (Central Server)
voctocore:
  build: .
  container_name: voctocore
  command: python3 voctocore/voctocore.py
  volumes:
    - ./:/opt/voctomix
    - ./images:/opt/images
  working_dir: /opt/voctomix
  networks:
    - voctomix_net
  ports:
    - "9998:9998"
    - "9998:9998/udp"
    - "9999:9999"
    - "10000:10000"
    - "10001:10001"
    - "10002:10002"
    - "10003:10003"
    - "10004:10004"
    - "10005:10005"
    - "11000:11000"
    - "12000:12000"
    - "13000-13005:13000-13005"
    - "14000-14005:14000-14005"
    - "15000:15000"
  healthcheck:
    test: ["CMD", "bash", "-c", "echo '' | nc -zw1 localhost 9999"]
    interval: 5s
    timeout: 3s
    retries: 12
    start_period: 20s

# 2. TELEMETRY SERVICE (Telemetry and JSON logging)
telemetry:
  image: voctomix-voctocore:latest
  container_name: telemetry
  command: python3 example-scripts/ffmpeg/telemetry_service.py
  environment:
    - TELEMETRY_USE_UDP=1
    - HOST_IP=${HOST_IP}
    - VOCTOCORE_HOST=voctocore
    - RABBITMQ_HOST=rabbitmq
    - RABBITMQ_USER=${RABBITMQ_USER:-voctomix}
    - RABBITMQ_PASS=${RABBITMQ_PASS:-voctomix123}
    - SAVE_LOGS=${SAVE_LOGS:-false}
  user: "${COMPOSE_UID:-1000}:${COMPOSE_GID:-1000}"
  volumes:
    - ./:/opt/voctomix
  working_dir: /opt/voctomix
  networks:
    - voctomix_net
  ports:
    - "8080:8080"
  depends_on:
    voctocore:
      condition: service_healthy
    rabbitmq:
      condition: service_healthy
```

```
# 3. CAMERA 1
cam1:
  build: .
  container_name: cam1
  environment:
    <<: *video-env
    CAM1_SOURCE: ${CAM1_SOURCE:-https://...}
    CAM1_PORT:   ${CAM1_PORT:-10000}
  command:
    - sh
    - -c
    - >-
      ffmpeg -y -nostdin
      -i "$$CAM1_SOURCE"
      -ac 2
      -filter_complex "[0:v] scale=$$WIDTH:$$HEIGHT,fps=$$FRAMERATE [v];
                      [0:a] aresample=$$AUDIORATE [a]"
      -map "[v]" -map "[a]"
      -pix_fmt yuv420p -c:v rawvideo -c:a pcm_s16le
      -f matroska tcp://localhost:$$CAM1_PORT
  volumes:
    - ./:/opt/voctomix
  working_dir: /opt/voctomix
  network_mode: "service:voctocore"
  restart: on-failure
  depends_on:
    voctocore:
      condition: service_healthy

# cam2, cam3: identicos a cam1 con sus respectivos puertos y fuentes.

# 4. CAMERA 4 (con filtro de colorimetria BT.709 explicito)
cam4:
  build: .
  container_name: cam4
  environment:
    <<: *video-env
    CAM4_SOURCE: ${CAM4_SOURCE:-https://...}
    CAM4_PORT:   ${CAM4_PORT:-10003}
  command:
    - sh
    - -c
    - >-
      ffmpeg -y -nostdin -stream_loop -1
      -i "$$CAM4_SOURCE"
      -filter_complex "[0:v] format=yuv420p,
                      scale=$$WIDTH:$$HEIGHT:out_range=tv,fps=$$FRAMERATE,
                      setparams=color_primaries=bt709:color_trc=bt709:colourspace=bt709 [v];
                      [0:a] aresample=$$AUDIORATE [a]"
      -map "[v]" -map "[a]"
      -color_primaries bt709 -color_trc bt709 -colourspace bt709 -color_range tv
      -c:v rawvideo -c:a pcm_s16le
      -f matroska tcp://localhost:$$CAM4_PORT
  volumes:
    - ./:/opt/voctomix
  working_dir: /opt/voctomix
  network_mode: "service:voctocore"
  restart: on-failure
  depends_on:
    voctocore:
      condition: service_healthy

# 5. BREAK (video de pausa con audio independiente)
break:
  build: .
  container_name: break
  environment:
    <<: *video-env
    BREAK_VIDEO: ${BREAK_VIDEO:-/opt/voctomix/videos/video_pausa.mp4}
    BREAK_AUDIO: ${BREAK_AUDIO:-/opt/voctomix/videos/musica_pausa.mp3}
    BREAK_PORT:  ${BREAK_PORT:-10004}
  command:
    - sh
```

```
- -c
- >-
ffmpeg -y -nostdin -stream_loop -1 -i "$$BREAK_VIDEO"
-re -stream_loop -1 -i "$$BREAK_AUDIO"
-filter_complex "[0:v] format=yuv420p,scale=$$WIDTH:$$HEIGHT:out_range=tv,
fps=$$FRAMERATE [v]; [1:a] aresample=$$AUDIORATE,pan=stereo|c0=c0|c1=c1 [a]"
-map "[v]" -map "[a]"
-c:v rawvideo -c:a pcm_s16le
-f matroska tcp://localhost:$$BREAK_PORT
network_mode: "service:votocore"
restart: on-failure
depends_on:
  votocore:
    condition: service_healthy

# 6. INTRO (bucle con pista de silencio generada en tiempo real)
intro:
  build: .
  container_name: intro
  environment:
    <<: *video-env
    INTRO_VIDEO: ${INTRO_VIDEO:-/opt/votomix/videos/intro.mp4}
    INTRO_PORT: ${INTRO_PORT:-10005}
  command:
    - bash
    - -c
    - |
      while true; do
        ffmpeg -loglevel quiet -nostats -re \
          -i "$$INTRO_VIDEO" \
          -f lavfi -i anullsrc=r=$$AUDIORATE:cl=stereo \
          -pix_fmt yuv420p -s "${WIDTH}x${HEIGHT}" -r "$$FRAMERATE" \
          -c:v rawvideo -c:a pcm_s16le \
          -map 0:v:0 -map 1:a:0 -shortest \
          -f matroska tcp://127.0.0.1:$$INTRO_PORT 2>/dev/null
        sleep 1
      done
  network_mode: "service:votocore"
  restart: on-failure
  depends_on:
    votocore:
      condition: service_healthy

# 7. STREAM BLANKER (PAUSE + NOSTREAM en paralelo)
stream_blanker:
  build: .
  container_name: stream_blanker
  environment:
    <<: *video-env
    PAUSE_VIDEO: ${PAUSE_VIDEO:-/opt/votomix/videos/starting_soon.mp4}
    OFFLINE_VIDEO: ${OFFLINE_VIDEO:-/opt/votomix/videos/stream_offline.mp4}
    BLANKER_PAUSE_PORT: ${BLANKER_PAUSE_PORT:-17000}
    BLANKER_OFFLINE_PORT: ${BLANKER_OFFLINE_PORT:-17001}
  command:
    - bash
    - -c
    - |
      ffmpeg -nostats -loglevel warning -stream_loop -1 -re \
        -i "$$PAUSE_VIDEO" \
        -pix_fmt yuv420p -s "${WIDTH}x${HEIGHT}" -r "$$FRAMERATE" \
        -c:v rawvideo -an -f matroska tcp://127.0.0.1:$$BLANKER_PAUSE_PORT &
      ffmpeg -nostats -loglevel warning -stream_loop -1 -re \
        -i "$$OFFLINE_VIDEO" \
        -pix_fmt yuv420p -s "${WIDTH}x${HEIGHT}" -r "$$FRAMERATE" \
        -c:v rawvideo -an -f matroska tcp://127.0.0.1:$$BLANKER_OFFLINE_PORT &
      wait
  network_mode: "service:votocore"
  restart: on-failure
  depends_on:
    votocore:
      condition: service_healthy

# 8. DYNAMIC AUDIO CONTROLLER (musica de fondo continua)
```

```
audio_manager:
  build: .
  container_name: audio_manager
  environment:
    AUDIORATE:      ${AUDIORATE:-48000}
    BACKGROUND_MUSIC: ${BACKGROUND_MUSIC:-/opt/voctomix/videos/musica_pausa.mp3}
    AUDIO_PORT:      ${AUDIO_PORT:-18000}
  command:
    - sh
    - -c
    - >-
      ffmpeg -nostats -loglevel warning -re -stream_loop -1
      -i "$$BACKGROUND_MUSIC"
      -c:a pcm_s16le -ar $$AUDIORATE -ac 2 -vn
      -f matroska tcp://127.0.0.1:$$AUDIO_PORT
  network_mode: "service:voctocore"
  restart: on-failure
  depends_on:
    voctocore:
      condition: service_healthy
```

C6. MANIFIESTOS DE DESPLIEGUE EN KUBERNETES

El despliegue en Kubernetes usa cinco ficheros. `configmap.yaml` guarda todos los parámetros en un solo lugar. `pvc.yaml` reserva espacio en disco para los registros de telemetría. `rabbitmq.yaml` arranca el broker de mensajería. `studio.yaml` define el pod principal con todos los contenedores y el servicio de red para exponerlos al exterior. Las contraseñas se guardan en un `Secret` independiente que no se reproduce aquí por seguridad; el fichero `secret.yaml.example` del repositorio sirve de plantilla. La duplicidad entre `CAM2_SOURCE` y `CAM3_SOURCE` en el listado siguiente está presente en el fichero original del repositorio y se reproduce tal cual.

k8s/configmap.yaml

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: voctomix-config
data:
  WIDTH: "1920"
  HEIGHT: "1080"
  FRAMERATE: "25"
  AUDIORATE: "48000"
  VOCTOCORE_HOST: "localhost"
  VOCTOCORE_PORT: "9999"
  VOCTOCORE_CLOCK_PORT: "9998"
  TELEMETRY_HTTP_PORT: "8080"
  TELEMETRY_USE_UDP: "1"
  VOCTOMIX_BASE_DIR: "/opt/voctomix"
  SAVE_LOGS: "false"
  RABBITMQ_HOST: "rabbitmq"
  RABBITMQ_PORT: "5672"
  RABBITMQ_USER: "voctomix"
  CAM1_PORT: "10000"
  CAM2_PORT: "10001"
  CAM3_PORT: "10002"
  CAM4_PORT: "10003"
  BREAK_PORT: "10004"
  INTRO_PORT: "10005"
  BLANKER_PAUSE_PORT: "17000"
  BLANKER_OFFLINE_PORT: "17001"
  AUDIO_PORT: "18000"
  BG_PORT: "16000"
  BG_IMAGE: "/opt/voctomix/images/backgrounds/bg.png"
  CAM1_SOURCE: "http://cdn.media.ccc.de/.../talk1_hd.mp4"
  CAM2_SOURCE: "http://cdn.media.ccc.de/.../talk2_hd.mp4"
  CAM3_SOURCE: "http://cdn.media.ccc.de/.../talk2_hd.mp4"
  CAM4_SOURCE: "https://www.w3schools.com/html/mov_bbb.mp4"
  BREAK_VIDEO: "/opt/voctomix/videos/video_cuenta_regresiva_10s.mp4"
  BREAK_AUDIO: "/opt/voctomix/videos/musica_pausa.mp3"
  INTRO_VIDEO: "/opt/voctomix/videos/intro.mp4"
  PAUSE_VIDEO: "/opt/voctomix/videos/SLIDES_video_starting_soon.mp4"
  OFFLINE_VIDEO: "/opt/voctomix/videos/stream_offline.mp4"
  BACKGROUND_MUSIC: "/opt/voctomix/videos/musica_pausa.mp3"
```

k8s/pvc.yaml

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: voctomix-sessions-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
```


k8s/rabbitmq.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: rabbitmq
spec:
  replicas: 1
  selector:
    matchLabels:
      app: rabbitmq
  template:
    metadata:
      labels:
        app: rabbitmq
    spec:
      containers:
        - name: rabbitmq
          image: rabbitmq:3-management
          ports:
            - containerPort: 5672
            - containerPort: 15672
          env:
            - name: RABBITMQ_DEFAULT_USER
              valueFrom:
                secretKeyRef:
                  name: voctomix-secret
                  key: RABBITMQ_DEFAULT_USER
            - name: RABBITMQ_DEFAULT_PASS
              valueFrom:
                secretKeyRef:
                  name: voctomix-secret
                  key: RABBITMQ_DEFAULT_PASS
          readinessProbe:
            exec:
              command: ["rabbitmq-diagnostics", "ping"]
            initialDelaySeconds: 30
            timeoutSeconds: 10
            periodSeconds: 15
            failureThreshold: 6
---
apiVersion: v1
kind: Service
metadata:
  name: rabbitmq
spec:
  selector:
    app: rabbitmq
  ports:
    - name: amqp
      port: 5672
      targetPort: 5672
    - name: management
      port: 15672
      targetPort: 15672
```

k8s/studio.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: studio
spec:
  replicas: 1
  selector:
    matchLabels:
      app: studio
  template:
    metadata:
      labels:
        app: studio
    spec:
      initContainers:
        - name: wait-for-rabbitmq
          image: busybox
          command:
            - sh
            - -c
            - until nc -zw1 rabbitmq 5672; do sleep 2; done

      containers:
        - name: voctocore
          image: ${VOCTOMIX_IMAGE:-martin1210235/voctomix:latest}
          imagePullPolicy: Always
          command: ["bash", "-c",
            "cd /opt/voctomix/voctocore && python3 voctocore.py"]
          envFrom:
            - configMapRef:
                name: voctomix-config
            - secretRef:
                name: voctomix-secret
          volumeMounts:
            - name: sessions
              mountPath: /opt/voctomix/sessions
          readinessProbe:
            tcpSocket:
              port: 9999
            initialDelaySeconds: 20
            periodSeconds: 5
            failureThreshold: 12

        - name: telemetry
          image: ${VOCTOMIX_IMAGE:-martin1210235/voctomix:latest}
          imagePullPolicy: Always
          command: ["python3",
            "example-scripts/ffmpeg/telemetry_service.py"]
          envFrom:
            - configMapRef:
                name: voctomix-config
            - secretRef:
                name: voctomix-secret
          volumeMounts:
            - name: sessions
              mountPath: /opt/voctomix/sessions

        - name: cam1
          image: ${VOCTOMIX_IMAGE:-martin1210235/voctomix:latest}
          imagePullPolicy: Always
          command:
            - sh
            - -c
            - >-
              ffmpeg -y -nostdin
              -i "${CAM1_SOURCE}"
              -ac 2
              -filter_complex "[0:v] scale=${WIDTH}:${HEIGHT},
                fps=${FRAMERATE} [v]; [0:a] aresample=${AUDIORATE} [a]"
              -map "[v]" -map "[a]"
              -pix_fmt yuv420p -c:v rawvideo -c:a pcm_s16le
```

```

        -f matroska tcp://localhost:${CAM1_PORT}
envFrom:
  - configMapRef:
      name: voctomix-config
- name: cam2
  image: ${VOCTOMIX_IMAGE:-martin1210235/voctomix:latest}
  imagePullPolicy: Always
  command:
    - sh
    - -c
    - >-
      ffmpeg -y -nostdin
      -i "${CAM2_SOURCE}"
      -ac 2
      -filter_complex "[0:v] scale=${WIDTH}:${HEIGHT},fps=${FRAMERATE} [v];
        [0:a] aresample=${AUDIORATE} [a]"
      -map "[v]" -map "[a]"
      -pix_fmt yuv420p -c:v rawvideo -c:a pcm_s16le
      -f matroska tcp://localhost:${CAM2_PORT}
  envFrom:
    - configMapRef:
        name: voctomix-config
- name: cam3
  image: ${VOCTOMIX_IMAGE:-martin1210235/voctomix:latest}
  imagePullPolicy: Always
  command:
    - sh
    - -c
    - >-
      ffmpeg -y -nostdin
      -i "${CAM3_SOURCE}"
      -ac 2
      -filter_complex "[0:v] scale=${WIDTH}:${HEIGHT},fps=${FRAMERATE} [v];
        [0:a] aresample=${AUDIORATE} [a]"
      -map "[v]" -map "[a]"
      -pix_fmt yuv420p -c:v rawvideo -c:a pcm_s16le
      -f matroska tcp://localhost:${CAM3_PORT}
  envFrom:
    - configMapRef:
        name: voctomix-config
- name: cam4 # conversion de colorimetria BT.709 explicita
  image: ${VOCTOMIX_IMAGE:-martin1210235/voctomix:latest}
  imagePullPolicy: Always
  command:
    - sh
    - -c
    - >-
      ffmpeg -y -nostdin -stream_loop -1
      -i "${CAM4_SOURCE}"
      -filter_complex "[0:v] format=yuv420p,
        scale=${WIDTH}:${HEIGHT}:out_range=tv,fps=${FRAMERATE},
        setparams=color_primaries=bt709:color_trc=bt709:
        colorspace=bt709 [v]; [0:a] aresample=${AUDIORATE} [a]"
      -map "[v]" -map "[a]"
      -color_primaries bt709 -color_trc bt709
      -colorspace bt709 -color_range tv
      -c:v rawvideo -c:a pcm_s16le
      -f matroska tcp://localhost:${CAM4_PORT}
  envFrom:
    - configMapRef:
        name: voctomix-config
- name: break
  image: ${VOCTOMIX_IMAGE:-martin1210235/voctomix:latest}
  imagePullPolicy: Always
  command:
    - sh
    - -c
    - >-
      ffmpeg -y -nostdin -stream_loop -1

```

```

-i "${BREAK_VIDEO}"
-re -stream_loop -1 -i "${BREAK_AUDIO}"
-filter_complex "[0:v] format=yuv420p,scale=${WIDTH}:${HEIGHT}:out_range
=tv,fps=${FRAMERATE},setparams=color_primaries=bt709:color_trc=bt709
:colorspace=bt709 [v]; [1:a] aresample=${AUDIORATE},pan=stereo|c0=c0
|c1=c1 [a]"
-map "[v]" -map "[a]"
-c:v rawvideo -c:a pcm_s16le
-f matroska tcp://localhost:${BREAK_PORT}
envFrom:
- configMapRef:
  name: voctomix-config

- name: intro
image: ${VOCTOMIX_IMAGE:-martin1210235/voctomix:latest}
imagePullPolicy: Always
command:
- bash
- -c
- |
while true; do
  ffmpeg -loglevel quiet -nostats -re \
    -i "${INTRO_VIDEO}" \
    -f lavfi -i anullsrc=r=${AUDIORATE}:cl=stereo \
    -pix_fmt yuv420p -s "${WIDTH}x${HEIGHT}" -r "${FRAMERATE}" \
    -c:v rawvideo -c:a pcm_s16le \
    -map 0:v:0 -map 1:a:0 \
    -shortest \
    -f matroska tcp://127.0.0.1:${INTRO_PORT} 2>/dev/null
  sleep 1
done
envFrom:
- configMapRef:
  name: voctomix-config

- name: stream-blanker
image: ${VOCTOMIX_IMAGE:-martin1210235/voctomix:latest}
imagePullPolicy: Always
command:
- bash
- -c
- |
ffmpeg -nostats -loglevel warning -stream_loop -1 -re \
  -i "${PAUSE_VIDEO}" \
  -pix_fmt yuv420p -s "${WIDTH}x${HEIGHT}" -r "${FRAMERATE}" \
  -c:v rawvideo -an \
  -f matroska tcp://127.0.0.1:${BLANKER_PAUSE_PORT} &
ffmpeg -nostats -loglevel warning -stream_loop -1 -re \
  -i "${OFFLINE_VIDEO}" \
  -pix_fmt yuv420p -s "${WIDTH}x${HEIGHT}" -r "${FRAMERATE}" \
  -c:v rawvideo -an \
  -f matroska tcp://127.0.0.1:${BLANKER_OFFLINE_PORT} &
wait
envFrom:
- configMapRef:
  name: voctomix-config

- name: audio-manager
image: ${VOCTOMIX_IMAGE:-martin1210235/voctomix:latest}
imagePullPolicy: Always
command:
- sh
- -c
- >-
ffmpeg -nostats -loglevel warning -re -stream_loop -1
-i "${BACKGROUND_MUSIC}"
-c:a pcm_s16le -ar ${AUDIORATE} -ac 2 -vn
-f matroska tcp://127.0.0.1:${AUDIO_PORT}
envFrom:
- configMapRef:
  name: voctomix-config

```

volumes:

```
- name: sessions
  persistentVolumeClaim:
    claimName: voctomix-sessions-pvc
---
apiVersion: v1
kind: Service
metadata:
  name: studio
spec:
  type: NodePort
  selector:
    app: studio
  ports:
    - name: control
      port: 9999
      targetPort: 9999
      nodePort: 30999
    - name: clock-udp
      protocol: UDP
      port: 9998
      targetPort: 9998
      nodePort: 30998
    - name: telemetry
      port: 8080
      targetPort: 8080
      nodePort: 30808
    - name: program-out
      port: 15000
      targetPort: 15000
      nodePort: 31500
    - name: mix-out
      port: 11000
      targetPort: 11000
      nodePort: 31100
    - name: mix-preview
      port: 12000
      targetPort: 12000
      nodePort: 31200
    - name: preview-0
      port: 14000
      targetPort: 14000
      nodePort: 31400
    - name: preview-1
      port: 14001
      targetPort: 14001
      nodePort: 31401
    - name: preview-2
      port: 14002
      targetPort: 14002
      nodePort: 31402
    - name: preview-3
      port: 14003
      targetPort: 14003
      nodePort: 31403
    - name: preview-4
      port: 14004
      targetPort: 14004
      nodePort: 31404
    - name: preview-5
      port: 14005
      targetPort: 14005
      nodePort: 31405
```

C7. HERRAMIENTAS DE MEDICIÓN EXPERIMENTAL

Estas son las tres herramientas con las que se obtuvieron los datos del Capítulo 5. `measure_composite_latency.py` mide cuánto tarda el mezclador en aplicar cada cambio de fuente o modo de composición. `measure_resilience.py` fuerza el fallo de un contenedor y mide el tiempo hasta que el sistema lo recupera. `analyze_stability.py` lee los registros de telemetría y genera las gráficas de CPU y RAM.

tools/measure_composite_latency.py

```
#!/usr/bin/env python3

import socket
import time
import argparse
import json
import numpy as np
from datetime import datetime

MODES = ["fs", "sbs", "pip", "lec"]
SRC_A = "cam1"
SRC_B = "cam2"

def send_recv(sock, cmd, expected_prefix, timeout=3.0):
    sock.sendall((cmd + "\n").encode())
    buf = ""
    deadline = time.monotonic() + timeout
    while time.monotonic() < deadline:
        try:
            chunk = sock.recv(4096).decode(errors="replace")
            buf += chunk
            for line in buf.split("\n"):
                line = line.strip()
                if line.startswith(expected_prefix):
                    return line
            except socket.timeout:
                continue
    return None

def measure(host, port, n_measurements):
    results = []
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    sock.settimeout(0.5)
    sock.connect((host, port))

    for i in range(n_measurements):
        mode = MODES[i % len(MODES)]
        cmd = f"transition {mode}({SRC_A},{SRC_B})"
        expected = f"composite {mode}"

        t0 = time.monotonic()
        response = send_recv(sock, cmd, expected)
        t1 = time.monotonic()

        if response is None:
            continue

        ms = (t1 - t0) * 1000
        results.append({"mode": mode, "latency_ms": round(ms, 3)})
        time.sleep(0.15)

    sock.close()
    return results

def stats(results):
    if not results:
```

```

        return {}
    vals = [r["latency_ms"] for r in results]
    return {
        "n": len(vals),
        "min_ms": round(float(np.min(vals)), 2),
        "median_ms": round(float(np.median(vals)), 2),
        "mean_ms": round(float(np.mean(vals)), 2),
        "max_ms": round(float(np.max(vals)), 2),
        "std_ms": round(float(np.std(vals)), 2),
        "p95_ms": round(float(np.percentile(vals, 95)), 2),
    }

def histogram(results, bins=None):
    if bins is None:
        bins = [1.0, 1.5, 2.0, 2.5, 3.0, 3.5, 4.0, 4.5, 5.0, 6.0, 8.0]
    vals = [r["latency_ms"] for r in results]
    hist = {}
    for lo, hi in zip(bins[:-1], bins[1:]):
        hist[f"{lo}-{hi}"] = sum(1 for v in vals if lo <= v < hi)
    return hist

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--host", default="localhost")
    parser.add_argument("--port", type=int, default=9999)
    parser.add_argument("--n", type=int, default=32)
    parser.add_argument("--label", default="docker")
    args = parser.parse_args()

    results = measure(args.host, args.port, args.n)
    s = stats(results)
    h = histogram(results)

    output = {
        "label": args.label,
        "timestamp": datetime.now().isoformat(),
        "host": args.host,
        "port": args.port,
        "stats": s,
        "histogram": h,
        "raw": results,
    }
    out_path = f"sessions/composite_latency_{args.label}.json"
    with open(out_path, "w") as f:
        json.dump(output, f, indent=2)

if __name__ == "__main__":
    main()

```

tools/measure_resilience.py

```
#!/usr/bin/env python3

import subprocess
import time
import json
import argparse
import numpy as np
from datetime import datetime

def container_is_healthy(name):
    out = subprocess.run(
        ["docker", "inspect", "--format",
         "{{.State.Status}} {{.State.Health.Status}}", name],
        capture_output=True, text=True
    ).stdout.strip()
    return out == "running healthy"

def measure_recovery(container, n):
    results = []

    for i in range(n):
        # Kill PID 1 inside the container to simulate a real crash.
        # docker stop would set desired-state=stopped and skip restart policy.
        subprocess.run(["docker", "exec", container, "kill", "-9", "1"],
                       capture_output=True)
        t_stop = time.monotonic()

        deadline = t_stop + 60
        recovered = False
        while time.monotonic() < deadline:
            time.sleep(0.5)
            if container_is_healthy(container):
                t_healthy = time.monotonic()
                recovered = True
                break

        if not recovered:
            continue

        ms = round((t_healthy - t_stop) * 1000)
        results.append(ms)
        time.sleep(2)

    return results

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--container", default="cam1")
    parser.add_argument("--n", type=int, default=10)
    args = parser.parse_args()

    results = measure_recovery(args.container, args.n)

    if not results:
        return

    vals = np.array(results)
    output = {
        "container": args.container,
        "timestamp": datetime.now().isoformat(),
        "stats": {
            "n": int(len(vals)),
            "min_ms": int(np.min(vals)),
            "median_ms": float(np.median(vals)),
            "mean_ms": float(np.mean(vals)),
            "max_ms": int(np.max(vals)),
            "p95_ms": float(np.percentile(vals, 95)),
        },
    }
```



```
        "raw_ms": list(map(int, vals)),
    }
    out_path = f"sessions/resilience_{args.container}.json"
    with open(out_path, "w") as f:
        json.dump(output, f, indent=2)

if __name__ == "__main__":
    main()
```

tools/analyze_stability.py

```
#!/usr/bin/env python3

import sys
import json
import os
import argparse
from datetime import datetime

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker
import numpy as np

DEFAULT_OUTPUT = os.path.join(
    os.path.dirname(__file__), "..", "memoria_tfg", "figuras",
    "estabilidad_30min.png"
)

def parse_jsonl(path, max_minutes=None):
    timestamps, cpu, ram = [], [], []
    t0 = None
    with open(path, "r", encoding="utf-8") as f:
        for line in f:
            line = line.strip()
            if not line:
                continue
            try:
                obj = json.loads(line)
            except json.JSONDecodeError:
                continue
            if obj.get("type") != "state":
                continue
            ts_str = obj.get("timestamp", "")
            health = obj.get("system_health", {})
            cpu_val = health.get("cpu_usage_percent")
            ram_val = health.get("ram_usage_percent")
            if ts_str and cpu_val is not None and ram_val is not None:
                try:
                    ts = datetime.fromisoformat(ts_str)
                    if t0 is None:
                        t0 = ts
                    elapsed_min = (ts - t0).total_seconds() / 60.0
                    if max_minutes is not None and elapsed_min > max_minutes:
                        break
                    timestamps.append(ts)
                    cpu.append(float(cpu_val))
                    ram.append(float(ram_val))
                except ValueError:
                    continue
    return timestamps, cpu, ram

def to_minutes(timestamps):
    if not timestamps:
        return []
    t0 = timestamps[0]
    return [(t - t0).total_seconds() / 60.0 for t in timestamps]

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("session", help="Path to JSONL session file")
    parser.add_argument("--output", default=None)
    parser.add_argument("--max-minutes", type=float, default=None)
    parser.add_argument("--label", default="Docker Compose")
    args = parser.parse_args()

    out_path = os.path.abspath(
```

```
args.output if args.output else DEFAULT_OUTPUT
)

timestamps, cpu, ram = parse_jsonl(args.session, args.max_minutes)
if not timestamps:
    sys.exit(1)

minutes = to_minutes(timestamps)

fig, ax = plt.subplots(figsize=(13 / 2.54, 6 / 2.54))
ax.plot(minutes, cpu, color="#CC5500", linewidth=1.2, label="CPU (%)")
ax.plot(minutes, ram, color="#1a6faf", linewidth=1.2, label="RAM (%)")
ax.set_xlabel("Tiempo transcurrido (min)", fontsize=9)
ax.set_ylabel("Uso del sistema (%)", fontsize=9)
ax.set_xlim(0, max(minutes) * 1.01)
ax.set_ylim(0, 85)
ax.xaxis.set_major_locator(ticker.MultipleLocator(5))
ax.yaxis.set_major_locator(ticker.MultipleLocator(10))
ax.grid(True, linestyle="--", linewidth=0.5, alpha=0.7)
ax.legend(fontsize=8, loc="upper right")
ax.tick_params(labelsize=8)
fig.tight_layout(pad=0.4)

os.makedirs(os.path.dirname(out_path), exist_ok=True)
fig.savefig(out_path, dpi=300, bbox_inches="tight")

if __name__ == "__main__":
    main()
```